Основы системного администрирования Linux: от командной строки до веб-сервера

Базовый курс на основе Debian

Чайкин Евгений

Содержание

Введение **12**
О чём и для кого эта книга . . . 12
Кто такой системный администратор . . . 12
За что он отвечает . . . 13
Как выглядит обычный рабочий день . . . 14
Сисадмин, DevOps и программист . . . 14
Почему Linux . . . 15
Почему WordPress . . . 15
Как главы этой книги связаны с реальной работой . . . 15
Что нужно для старта . . . 16
Как пользоваться курсом . . . 16

Работа в консоли **17**
Почему текст, а не кнопки . . . 17
Комбинируемость . . . 17
Автоматизируемость . . . 18
Минимальные требования к каналу связи . . . 18
Почему серверы без графической оболочки . . . 18
Tab: автодополнение . . . 19
Двойной Tab: список вариантов . . . 19
Стрелки: история команд . . . 19
Поиск по истории: Ctrl+R . . . 20
Команда history . . . 20
Горячие клавиши . . . 20
Ctrl+C: ваш аварийный выход . . . 21
Ctrl+D: мягкий выход . . . 21
Ctrl+L: быстрая очистка экрана . . . 21
Копирование и вставка . . . 21
Два буфера обмена в Linux . . . 22
Перемещение по строке . . . 22
Ошибки — это нормально . . . 22
Многозадачность: несколько окон . . . 23
Итоги . . . 24

Этап 1. Первое знакомство с Linux **25**
Что такое Linux . . . 25
Подключение по SSH . . . 25
Подключаемся из Windows . . . 25
Первое подключение . . . 26
Отключение . . . 26

Командная строка: первый взгляд . . . 26
Первые команды . . . 27
whoami — кто я . . . 27
hostname — имя компьютера . . . 27
pwd — где я нахожусь . . . 27
ls — что здесь лежит . . . 27
cd — перейти в другой каталог . . . 28
clear — очистить экран . . . 28
exit — выйти . . . 28
Как получить помощь . . . 28
man — встроенное руководство . . . 29
–help — краткая справка . . . 29
Практическое задание . . . 29
Итоги . . . 30

Этап 2. Файловая система **31**
Всё — файл . . . 31
Структура каталогов . . . 31
Абсолютные и относительные пути . . . 32
Навигация . . . 32
cd — перемещение по каталогам (подробнее) . . . 32
ls — просмотр содержимого каталога (подробнее) . . . 33
Создание файлов и каталогов . . . 34
touch — создать пустой файл . . . 34
mkdir — создать каталог . . . 34
Практика: структура каталогов для проекта . . . 34
Копирование, перемещение, удаление . . . 35
cp — копирование . . . 35
mv — перемещение и переименование . . . 35
rm — удаление . . . 35
Просмотр содержимого файлов . . . 36
cat — вывести файл целиком . . . 36
less — постраничный просмотр . . . 36
head — начало файла . . . 36
tail — конец файла . . . 37
Врезка: кодировки и переводы строк . . . 37
Кодировки . . . 37
Переводы строк . . . 37
Как проверить . . . 38
Практическое задание . . . 38
Итоги . . . 40

Этап 3. Права доступа и владение файлами **41**
Зачем нужны права доступа . . . 41
Владелец, группа, остальные . . . 41
Три типа прав: чтение, запись, выполнение . . . 42
Чтение строки прав . . . 42
Примеры . . . 42
Числовой (восьмеричный) формат . . . 43
chmod — изменение прав . . . 43
Символьный формат . . . 43
Числовой формат . . . 44

Почему `chmod 777` — плохая идея . . . 44
Разные права для файлов и каталогов . . . 45
chown и chgrp — смена владельца и группы . . . 45
chown — смена владельца . . . 45
chgrp — смена группы . . . 46
Практика: права для проекта WordPress . . . 46
Задание 1. Изучите текущие права . . . 47
Задание 2. Сделайте конфиги доступными только владельцу . . . 47
Задание 3. Создайте скрипт и сделайте его исполняемым . . . 47
Задание 4. Ограничьте доступ к каталогу logs . . . 48
Специальные биты (обзор) . . . 48
Практическое задание . . . 48
Итоги . . . 49

Этап 4. Текстовые редакторы: nano и vim **50**
Зачем нужен текстовый редактор в терминале . . . 50
nano — основной редактор . . . 50
Установка . . . 50
Открытие файла . . . 51
Интерфейс nano . . . 51
Навигация . . . 52
Редактирование текста . . . 52
Поиск и замена . . . 52
Сохранение и выход . . . 53
Полезные приёмы . . . 53
Практика: редактируем конфигурацию проекта . . . 54
vim — обзор . . . 55
Зачем знать vim . . . 55
Два режима vim . . . 55
Минимальный набор действий . . . 56
Шпаргалка по vim . . . 56
Как сменить редактор по умолчанию . . . 57
Практическое задание . . . 57
Итоги . . . 59

Этап 5. Coreutils и конвейеры: обработка текста **60**
Зачем это нужно . . . 60
Врезка: перенаправление ввода-вывода . . . 60
Три стандартных потока . . . 60
Перенаправление в файл . . . 61
Перенаправление ошибок . . . 61
Конвейер (pipe) . . . 62
Подготовка: пример лог-файла nginx . . . 62
wc — подсчёт строк, слов и символов . . . 63
sort — сортировка строк . . . 64
uniq — удаление повторяющихся строк . . . 64
cut — извлечение столбцов . . . 65
По разделителю (самый частый вариант) . . . 65
По символам . . . 66
grep — поиск строк по шаблону . . . 66
Простой поиск . . . 66
Полезные флаги . . . 66

grep в конвейере . . . 67
Врезка: регулярные выражения (базовый уровень) . . . 67
Основные метасимволы . . . 68
Примеры с grep . . . 68
Расширенные регулярные выражения . . . 68
sed — потоковая замена текста . . . 69
Примеры . . . 69
sed в конвейере . . . 69
awk — обработка текста по столбцам . . . 70
Базовый синтаксис . . . 70
Примеры . . . 70
Фильтрация в awk . . . 70
Указание разделителя . . . 71
Комбинирование: решаем реальные задачи . . . 71
Задача 1: найти 3 самых активных IP-адреса . . . 71
Задача 2: подсчитать количество запросов с ошибками . . . 71
Задача 3: список уникальных URL, которые запрашивали . . . 72
Задача 4: сохранить только ошибки в отдельный файл . . . 72
Задача 5: общий объём переданных данных . . . 72
Практическое задание . . . 72
Итоги . . . 74

Этап 6. Основы bash-скриптов **75**
Зачем нужны скрипты . . . 75
Врезка: переменные окружения и PATH . . . 75
Что такое переменные окружения . . . 75
Создание и экспорт переменных . . . 76
PATH — как система находит команды . . . 77
Файл ~/.bashrc . . . 77
Первый скрипт . . . 78
Создание . . . 78
Shebang . . . 78
Запуск . . . 78
Альтернативный способ запуска . . . 79
Переменные в скриптах . . . 79
Объявление и использование . . . 79
Фигурные скобки . . . 79
Кавычки . . . 79
Подстановка команд . . . 80
Аргументы скрипта . . . 80
Условия (if) . . . 81
Базовый синтаксис . . . 81
Примеры условий . . . 81
if / else / elif . . . 81
Проверка результата команды . . . 82
Циклы . . . 82
for — перебор списка . . . 82
while — цикл с условием . . . 83
Коды возврата . . . 83
Что это такое . . . 83
exit в скриптах . . . 84
Практика: скрипт резервного копирования . . . 84

Практическое задание . . . 86
Итоги . . . 87

Этап 7. Управление пакетами (apt) и репозитории 88
Что такое пакеты . . . 88
Что такое зависимости . . . 88
Репозитории . . . 89
Что это такое . . . 89
Секции репозитория . . . 89
Обновление списка пакетов . . . 90
Поиск и информация о пакетах . . . 90
apt search — поиск по имени и описанию . . . 90
apt show — подробная информация . . . 90
dpkg -l — список установленных пакетов . . . 91
Проверка, установлен ли пакет . . . 91
Установка пакетов . . . 91
apt install . . . 91
Установка нескольких пакетов . . . 92
Обновление пакетов . . . 92
Обновление списка + обновление пакетов . . . 92
apt upgrade vs apt full-upgrade . . . 92
Удаление пакетов . . . 92
apt remove — удаление программы . . . 92
apt purge — полное удаление . . . 93
apt autoremove — удаление ненужных зависимостей . . . 93
Сводная таблица команд apt . . . 93
Практика: утилита tree . . . 93
Практическое задание . . . 94
Итоги . . . 95

Этап 8. Пользователи, группы и sudo 96
Зачем в Linux несколько пользователей . . . 96
Пользователь root . . . 96
Как устроены пользователи . . . 97
UID и файл /etc/passwd . . . 97
Системные и обычные пользователи . . . 97
Пароли и /etc/shadow . . . 98
Группы . . . 98
Зачем нужны группы . . . 98
GID и файл /etc/group . . . 98
Основная и дополнительные группы . . . 99
Управление пользователями . . . 99
Создание пользователя: adduser . . . 99
Смена пароля: passwd . . . 99
Изменение пользователя: usermod . . . 100
Удаление пользователя: deluser . . . 100
Отключение пользователя: usermod -L . . . 100
Управление группами . . . 101
Создание и удаление групп . . . 101
Добавление пользователя в группу . . . 101
Проверка принадлежности . . . 101
Переключение пользователей: su . . . 101

sudo: выполнение команд от имени root . . . 102
Зачем нужен sudo . . . 102
Группа sudo . . . 102
Файл /etc/sudoers и visudo . . . 102
Практика: настраиваем пользователя для проекта . . . 103
Шаг 1. Создаём группу и пользователя . . . 103
Шаг 2. Добавляем webmaster в группы . . . 103
Шаг 3. Меняем владельца каталога проекта . . . 103
Шаг 4. Настраиваем права для группы . . . 104
Шаг 5. Проверяем от имени webmaster . . . 104
Справочник команд . . . 105
Практические задания . . . 105
Итоги . . . 106

Этап 9. Процессы и сервисы (systemd) 107
Что такое процесс . . . 107
Просмотр процессов . . . 108
ps — снимок процессов . . . 108
pstree — дерево процессов . . . 108
top — интерактивный мониторинг . . . 109
Управление процессами . . . 109
Сигналы . . . 109
kill — отправка сигнала . . . 109
Фоновые процессы . . . 110
Что такое systemd . . . 110
systemctl: управление сервисами . . . 111
Статус сервиса . . . 111
Запуск, остановка, перезапуск . . . 111
Автозапуск при загрузке . . . 112
Список сервисов . . . 112
journalctl: чтение логов . . . 112
Логи конкретного сервиса . . . 112
Последние записи . . . 112
Следить в реальном времени . . . 112
Фильтрация по времени . . . 113
Устройство unit-файла . . . 113
Где хранятся unit-файлы . . . 113
Разбор ssh.service . . . 113
Практика: оборачиваем backup.sh в systemd-сервис . . . 114
Шаг 1. Подготавливаем скрипт . . . 114
Шаг 2. Создаём service-файл . . . 115
Шаг 3. Создаём timer-файл . . . 115
Шаг 4. Запускаем таймер . . . 116
Шаг 5. Проверяем . . . 116
Справочник команд . . . 116
Процессы . . . 117
Фоновые процессы . . . 117
systemctl . . . 117
journalctl . . . 117
Практические задания . . . 118
Итоги . . . 119

Этап 10. Сеть: ip, ss, firewall (ufw), основы DNS **120**
Зачем разбираться в сети . . . 120
Врезка: основы IP-сетей . . . 120
IP-адреса . . . 120
Маска подсети и CIDR . . . 121
Шлюз (gateway) . . . 121
Порты . . . 121
TCP и UDP . . . 122
ip: сетевые интерфейсы и адреса . . . 122
Посмотреть IP-адреса . . . 122
Посмотреть маршруты . . . 123
Посмотреть интерфейсы . . . 123
ss: сетевые соединения . . . 123
Какие порты слушают . . . 123
Установленные соединения . . . 124
Проверка соединений . . . 124
ping — доступность хоста . . . 124
curl — HTTP-запрос . . . 124
DNS: разрешение имён . . . 125
Что такое DNS . . . 125
/etc/resolv.conf — настройки DNS . . . 125
/etc/hosts — локальные записи . . . 125
Проверка DNS . . . 125
ufw: файрвол . . . 126
Зачем нужен файрвол . . . 126
Установка и включение . . . 126
Статус и правила . . . 126
Разрешение и запрет . . . 127
Удаление правил . . . 127
Отключение файрвола . . . 127
Практика: настраиваем сеть для проекта . . . 127
Шаг 1. Узнаём IP-адрес сервера . . . 127
Шаг 2. Проверяем шлюз и доступ в интернет . . . 127
Шаг 3. Смотрим, что слушает сеть . . . 128
Шаг 4. Настраиваем файрвол . . . 128
Шаг 5. Проверяем DNS . . . 128
Справочник команд . . . 128
Сетевые интерфейсы и адреса . . . 128
Соединения и диагностика . . . 128
DNS . . . 129
Файрвол (ufw) . . . 129
Практические задания . . . 129
Итоги . . . 130

Этап 11. Установка и настройка nginx + PHP-FPM + WordPress **131**
Что мы будем делать . . . 131
Врезка: клиент-серверная модель и HTTP . . . 132
Клиент и сервер . . . 132
HTTP-запрос . . . 132
HTTP-ответ . . . 132
Статика и динамика . . . 132
Установка nginx . . . 133

Устанавливаем и проверяем . . . 133
Структура каталогов nginx . . . 133
Настройка серверного блока для WordPress . . . 133
Создаём каталог для сайта . . . 133
Создаём конфигурацию сайта . . . 134
Включаем сайт . . . 135
Установка PHP-FPM . . . 135
Устанавливаем PHP и необходимые модули . . . 135
Как nginx взаимодействует с PHP-FPM . . . 135
Проверяем связку nginx + PHP-FPM . . . 136
Установка MariaDB . . . 136
Устанавливаем и защищаем . . . 136
Создаём базу данных и пользователя . . . 137
Установка WordPress . . . 137
Скачиваем и распаковываем . . . 137
Настраиваем права . . . 137
Настраиваем wp-config.php . . . 138
Завершаем установку через браузер . . . 138
Проверка всего стека . . . 139
Справочник команд . . . 139
nginx . . . 139
PHP-FPM . . . 139
MariaDB . . . 140
WordPress . . . 140
Практические задания . . . 140
Итоги . . . 141

Этап 12. Логирование и мониторинг 142
Зачем следить за логами . . . 142
Системный журнал: journalctl (расширение) . . . 142
Приоритеты сообщений . . . 142
Логи текущей загрузки . . . 143
Размер журнала и ротация . . . 143
Логи nginx . . . 143
Где лежат логи . . . 144
Формат access.log . . . 144
Формат error.log . . . 144
Анализ access.log с помощью coreutils . . . 144
Сколько запросов получил сервер . . . 145
Топ IP-адресов (кто чаще всего обращается) . . . 145
Распределение по кодам ответа . . . 145
Найти все ошибки 404 (страница не найдена) . . . 145
Найти серверные ошибки (500) . . . 146
Самые популярные страницы . . . 146
Запросы за последний час . . . 146
Найти ботов . . . 146
Анализ error.log . . . 146
Ротация логов (logrotate) . . . 147
Зачем нужна ротация . . . 147
Конфигурация для nginx . . . 147
Практика: анализируем логи WordPress . . . 148
Шаг 1. Смотрим системный журнал . . . 148

Шаг 2. Генерируем трафик для анализа . . . 148
Шаг 3. Анализируем access.log . . . 148
Шаг 4. Проверяем error.log . . . 148
Шаг 5. Проверяем ротацию . . . 148
Справочник команд . . . 149
journalctl (расширение) . . . 149
Анализ логов nginx . . . 149
Практические задания . . . 149
Итоги . . . 151

Этап 13. Git: основы контроля версий 152
Зачем нужен контроль версий . . . 152
Установка и настройка . . . 152
Установка . . . 153
Первоначальная настройка . . . 153
Три зоны Git . . . 153
Первый репозиторий . . . 154
git status — текущее состояние . . . 154
Что не хранить в репозитории: .gitignore . . . 155
Синтаксис .gitignore . . . 156
Первый коммит . . . 156
git log — история коммитов . . . 157
git diff — что изменилось . . . 157
Справочник основных команд . . . 158
Ветвление и слияние . . . 159
Создание ветки . . . 159
Работа в ветке . . . 160
Слияние . . . 160
Справочник команд ветвления . . . 161
Конфликты слияния . . . 161
Создаём конфликт . . . 161
Разрешаем конфликт . . . 162
Откат изменений . . . 163
Удалённый репозиторий: GitLab . . . 163
Регистрация на GitLab . . . 164
Настройка SSH-ключа . . . 164
Создание проекта на GitLab . . . 165
Подключение и отправка . . . 165
Получение изменений и клонирование . . . 166
Справочник команд для работы с удалёнными репозиториями . . . 166
Практические задания . . . 166
Итоги . . . 168

Этап 14. Docker: контейнеризация проекта 169
Зачем нужны контейнеры . . . 169
Контейнер vs. виртуальная машина . . . 169
Врезка: форматы конфигурационных файлов . . . 170
INI . . . 170
JSON . . . 170
YAML . . . 171
Сравнение форматов . . . 171
Установка Docker . . . 172

Подготовка . . . 172
Добавление репозитория Docker . . . 172
Установка . . . 172
Настройка доступа без sudo . . . 172
Проверка . . . 173
Образы и контейнеры . . . 173
Docker Hub . . . 173
Основные команды для работы с образами и контейнерами . . . 174
Справочник команд . . . 175
Запуск nginx в контейнере . . . 175
Проброс порта . . . 175
Монтирование файлов (volumes) . . . 176
Dockerfile: собственный образ . . . 176
Создание Dockerfile . . . 177
Сборка образа . . . 177
Запуск из собственного образа . . . 178
.dockerignore . . . 178
Справочник инструкций Dockerfile . . . 178
Управление контейнерами . . . 178
Жизненный цикл контейнера . . . 178
Просмотр логов . . . 179
Выполнение команд внутри контейнера . . . 179
Перезапуск контейнера . . . 179
Информация о контейнере . . . 180
Справочник команд управления . . . 180
Очистка . . . 180
Практические задания . . . 180
Итоги . . . 182

Этап 15. Docker Compose: оркестрация в контейнерах 183
Зачем нужен Docker Compose . . . 183
Установка . . . 184
Подготовка структуры проекта . . . 184
Шаг 1: база данных (MariaDB) . . . 184
Где хранятся данные тома . . . 185
Файл .env . . . 186
Запуск и проверка . . . 186
Шаг 2: WordPress . . . 187
Шаг 3: nginx . . . 188
Запуск и проверка . . . 190
Управление стеком . . . 191
Основные команды . . . 191
stop vs. down . . . 191
Просмотр логов . . . 191
Выполнение команд внутри контейнера . . . 192
Практические задания . . . 192
Итоги . . . 193

Этап 16. CI/CD: автоматизация сборки и деплоя 194
Зачем автоматизировать . . . 194
Что такое CI/CD . . . 194
CI/CD-системы . . . 195

Как устроен GitLab CI/CD . . . 195
Установка GitLab Runner . . . 195
Шаг 1: добавление репозитория и установка . . . 195
Шаг 2: регистрация Runner в GitLab . . . 196
Шаг 3: доступ Runner к Docker . . . 197
Врезка. DooD и DinD — два способа дать CI/CD доступ к Docker . . . 197
Первый конвейер . . . 198
Полный конвейер: проверка и деплой . . . 199
Два этапа . . . 200
Проверка конфигурации nginx . . . 201
Деплой только из main . . . 201
Окружение (environment) . . . 201
Переменные окружения и секреты . . . 201
Настройка переменных . . . 201
Использование переменных в конвейере . . . 202
Конвейер с уведомлениями . . . 203
Ключевое слово needs и граф зависимостей . . . 203
Финальная версия конвейера . . . 204
Что происходит после push . . . 205
Работа с ветками . . . 205
Отладка конвейера . . . 206
Runner не подхватывает задачу . . . 206
Ошибка «docker: not found» . . . 206
Ошибка в .gitlab-ci.yml . . . 207
Справочник: структура .gitlab-ci.yml . . . 207
Практические задания . . . 208
Итоги . . . 209

Введение

О чём и для кого эта книга

Эта книга — практический курс по Linux (на основе Debian) для тех, кто только начинает путь в IT. Если вы работали только в Windows, никогда не открывали командную строку и не настраивали серверы — вы именно тот читатель, для которого она написана.

Мы пройдём путь от первого подключения к серверу по SSH до развёртывания веб-сайта в контейнерах Docker с автоматическим деплоем. Через все главы проходит сквозной проект — настоящий сайт на **nginx + PHP-FPM + WordPress**, который вы соберёте своими руками: сначала вручную, потом в контейнерах, потом с автоматизацией через CI/CD.

Свежая версия книги, исходные markdown-файлы и история изменений доступны в репозитории: https://github.com/strannick-ru/linux-book

Курс охватывает три уровня: пользователь Linux (командная строка, файлы, права, скрипты), системный администратор (пакеты, сервисы, сеть, веб-сервер, логи) и начала DevOps (Git, Docker, CI/CD). Каждая глава опирается на предыдущие, поэтому читайте последовательно.

Но прежде чем перейти к практике, давайте разберёмся: кто такой системный администратор, чем он занимается и почему именно Linux — главный инструмент в этой профессии.

Кто такой системный администратор

Системный администратор (сисадмин) — это специалист, который обеспечивает жизнь серверной системы: развёртывает её, поддерживает, защищает, чинит и постепенно улучшает.

За что он отвечает

Всё начинается с настройки. Сисадмин берёт пустую машину и превращает её в рабочий сервер: устанавливает Linux, настраивает сеть, создаёт пользователей, поднимает сервисы — веб-сервер, базу данных, почту. После этого сервер начинает работать — и начинается основная часть работы: поддержка.

Сайт должен открываться, база данных — отвечать на запросы, веб-сервер — не падать, контейнеры — стартовать после перезагрузки, сертификаты — не истекать. Администратор следит, чтобы всё это было доступно. Звучит просто, но на практике это требует постоянного внимания.

Когда что-то ломается — а ломается рано или поздно всё — сисадмин ищет причину. Сайт перестал открываться, база тормозит, диск переполнен, SSH не пускает, контейнер упал. Нужно прочитать логи, проверить сервисы, сеть, права, конфиги — и найти, что именно пошло не так. Очень большая часть работы — это именно диагностика, а не первичная настройка.

Параллельно администратор занимается безопасностью: следит за обновлениями, настраивает файрвол, ограничивает доступ, контролирует секреты и пароли. Работающая система, в которую может зайти кто угодно, — это не рабочая система. Сюда же входят резервные копии — причём бэкап, из которого нельзя восстановиться, это не бэкап. Администратор не просто делает копии, а проверяет, что из них действительно можно поднять рабочую систему.

Хороший сисадмин не любит повторять одно и то же вручную. Он автоматизирует рутину: пишет скрипты, настраивает задания по расписанию, использует шаблоны конфигов. Он наблюдает за системой — следит за загрузкой процессора и памяти, местом на дисках, сроками действия сертификатов — и старается заметить проблему до того, как её заметят пользователи. Он документирует: записывает, как устроена система и почему были приняты те или иные решения. Документация — это не бюрократия, а возможность разобраться в происходящем, когда автор настройки в отпуске, уволился или просто забыл детали.

И наконец — сисадмин общается с людьми. Это часто забывают, но администратор разговаривает с разработчиками, менеджерами, коллегами. Ему нужно уметь объяснить: что сломалось, насколько это критично, что уже сделано и почему нельзя «просто открыть всё наружу».

Работа администратора — это смесь инженерии, диагностики, безопасности, автоматизации и ответственности за стабильность.

Как выглядит обычный рабочий день

У сисадмина нет «типичного дня» — каждый день другой. Но есть общий ритм.

Утро обычно начинается с проверки: всё ли работает, не было ли ночных сбоев, хватает ли места на дисках, нет ли тревожных записей в логах. Дальше — плановая работа: обновить пакеты на сервере, настроить автоматические бэкапы, перенести сервис в контейнеры, написать документацию, улучшить мониторинг — добавить алерты на заканчивающееся место на диске. Но в любой момент может прилететь сообщение «сайт не открывается» — и всё плановое откладывается.

Аварийная работа — это всегда диагностика. Диск переполнен и база не может записать данные. Сертификат просрочен и браузер показывает предупреждение. После обновления сервис не стартует. Прежде чем чинить, нужно понять, что именно сломалось. Сисадмин читает логи, проверяет статус сервисов, смотрит сетевые соединения, сравнивает конфиги с тем, что было раньше. Навык диагностики — один из самых ценных и один из самых сложных в освоении. Он приходит с практикой.

Именно баланс между плановой и аварийной работой определяет профессию. Хороший администратор тратит больше времени на плановую — налаживает мониторинг, автоматизирует рутину, пишет документацию — чтобы аварийных ситуаций было как можно меньше.

Сисадмин, DevOps и программист

В IT много ролей, и их границы размыты. Вот как они соотносятся:

Программист (разработчик) пишет код приложения — ту программу, которой пользуются люди. Его фокус — логика приложения, интерфейс, данные.

Системный администратор обеспечивает инфраструктуру, на которой этот код работает. Его фокус — серверы, сеть, безопасность, доступность.

DevOps-инженер стоит на пересечении. Он понимает и код, и инфраструктуру, а его главная задача — автоматизация: сделать так, чтобы путь от написанного кода до работающего сервиса был максимально быстрым и надёжным.

Раньше в компаниях было чёткое разделение: разработчики пишут код, а сисадмины его разворачивают на серверах. Разработчик говорит: «Вот новая версия, деплойте». Сисадмин отвечает: «У вас ничего не работает, пишите нормально». Каждая сторона знала только свою часть — и это порождало проблемы.

DevOps (от *Development* + *Operations*) — подход, при котором разработка и эксплуатация объединяются. DevOps-инженер автоматизирует тестирование, сборку, деплой, мониторинг. Типичные его инструменты: CI/CD, Docker, Ansible, Terraform, Kubernetes.

На практике граница между сисадмином и DevOps-инженером размытая. Многие сисадмины занимаются автоматизацией, а DevOps-инженеры — настройкой серверов. Это скорее спектр, чем два отдельных мира. Специалист, который начинал как сисадмин, со временем осваивает Docker, CI/CD, инфраструктуру как код — и постепенно двигается в сторону DevOps.

Навыки системного администрирования — это фундамент, на котором строится всё остальное. Именно поэтому этот курс начинается с основ Linux и администрирования, а заканчивается контейнерами и CI/CD.

Почему Linux

Более 90% серверов в мире работают на Linux. Облачные платформы (AWS, Google Cloud, Yandex Cloud), веб-хостинги, суперкомпьютеры, контейнеры — почти всё построено на Linux. Даже Android использует ядро Linux.

Для системного администратора и DevOps-инженера Linux — основной рабочий инструмент. Знание Linux — это не «хорошо бы», а обязательное условие.

Windows-серверы тоже существуют, но их ниша уже: корпоративная инфраструктура с Active Directory, серверы для .NET-приложений. Большинство веб-сервисов, баз данных и современных приложений работают именно на Linux.

Почему WordPress

В этом курсе мы не изучаем Linux в вакууме. Красной нитью через все главы проходит практический проект — развёртывание веб-сайта на **nginx + PHP-FPM + WordPress**.

WordPress — самая популярная CMS (система управления контентом) в мире: примерно каждый третий сайт в интернете работает на нём. Развернуть WordPress — реальная задача, с которой сисадмин сталкивается на практике. А стек, необходимый для его работы, включает все классические компоненты: nginx — один из самых популярных веб-серверов, PHP-FPM — обработчик PHP-скриптов и MariaDB — база данных, в которой WordPress хранит содержимое сайта.

Как главы этой книги связаны с реальной работой

Этот курс — обзорный. Он даёт начальные знания по каждой теме, а не исчерпывающее руководство. Но вот что важно: темы, которые мы разбираем — это не учебная экзотика. Это реальные задачи, из которых состоит бóльшая часть повседневной работы администратора.

Глава	Тема	Реальная задача
1–2	Знакомство с Linux, файловая система	Базовая ориентация на сервере — без этого невозможно всё остальное
3	Права доступа	Безопасность: кто имеет доступ к каким файлам
4	Текстовые редакторы	Редактирование конфигов — ежедневная работа
5	Coreutils и конвейеры	Диагностика: поиск по логам, анализ данных при разборе инцидентов
6	Bash-скрипты	Автоматизация: бэкапы, деплой, проверки состояния
7	Пакеты	Установка и обновление программ — регулярная плановая задача
8	Пользователи и группы	Управление доступом: кто может заходить на сервер
9	Systemd	Управление сервисами: запуск, остановка, автозапуск
10	Сеть	Сетевая настройка и файрвол — без этого не настроить и не продиагностировать сервис
11	nginx + PHP-FPM + WordPress	Развёртывание веб-стека — одна из самых типичных задач

Глава	Тема	Реальная задача
12	Логирование	Наблюдение за системой: логи — первое, куда смотрит сисадмин
13	Git	Контроль версий конфигов: возможность откатить и понять, кто что менял
14–15	Docker, Docker Compose	Контейнеризация — современный способ развёртывания сервисов
16	CI/CD	Автоматический деплой — шаг в сторону DevOps

Один и тот же проект — WordPress на nginx — проходит через все уровни курса. К концу у вас будет не только набор знаний, но и конкретный результат: работающий сайт, который обновляется автоматически после каждого push в репозиторий.

Что нужно для старта

- Компьютер с любой операционной системой: Windows, Linux или macOS. SSH-клиент встроен во все три — ничего дополнительно устанавливать не нужно.
- Доступ к серверу с Debian 13 — виртуальная машина от преподавателя или собственный ноутбук с установленным Debian.
- Желание разобраться.

Как пользоваться курсом

Читайте последовательно. Каждая глава опирается на предыдущие. Если перескочить к настройке nginx, не разобравшись с правами доступа и сервисами — будет непонятно.

Выполняйте практические задания. В конце каждой главы есть задания. Они не факультативные — именно через практику закрепляются навыки.

Не бойтесь ошибок. Вы работаете на учебном сервере или виртуальной машине. Если что-то сломается — не страшно. Ошибки — нормальная часть обучения, а умение их исправлять — один из ключевых навыков администратора.

Работа в консоли

Чёрный экран, мигающий курсор, никаких кнопок — для человека, привыкшего к Windows, командная строка выглядит как шаг назад. Зачем набирать текст, если можно кликнуть мышкой?

У графического интерфейса есть очевидные плюсы: он привычен, нагляден, подсказывает возможные действия — не нужно ничего запоминать, всё перед глазами. Но когда люди пробуют всерьёз поработать в консоли, многие с удивлением обнаруживают, что текстовый интерфейс по-своему удобен — а для ряда задач попросту быстрее. Вот почему.

Почему текст, а не кнопки

Комбинируемость

Одна из ключевых идей командной строки Unix — принцип маленьких инструментов. Каждая утилита делает что-то одно: `grep` ищет текст, `sort` сортирует строки, `wc` считает их количество. По отдельности каждая не слишком впечатляет. Но их можно соединять в цепочки — передавать вывод одной команды на вход другой через символ `|` (он называется «пайп», от английского *pipe* — труба).

Например, вы хотите узнать, с каких IP-адресов чаще всего заходят на ваш сайт:

```
awk '{print $1}' /var/log/nginx/access.log | sort | uniq -c | sort -rn | head
↪  -20
```

Выглядит пугающе. Но никто не пишет такое целиком, в одно движение. Администратор строит эту команду как домик из кубиков, добавляя по одному элементу и проверяя результат на каждом шаге.

Ход мысли примерно такой: «Мне нужны IP-адреса — это первое поле в каждой строке лога. Вытащу-ка их… Хорошо, вижу список. Теперь посчитать, сколько раз встречается каждый. Отлично. Отсортировать по убыванию… И взять двадцатку самых частых».

Каждый шаг — одна простая утилита. `awk` вытаскивает нужное поле, `sort` сортирует, `uniq -c` считает повторения, `head -20` обрезает вывод. Пайп `|` просто передаёт результат на вход следующей. Вы собираете решение по частям, как конструктор, — и на каждом этапе видите, что получается.

Мы подробно разберём каждую из этих утилит в главе 5. Пока достаточно понять принцип: не нужно держать в голове всю команду целиком — нужно уметь думать «шагами».

Эта комбинируемость — огромная сила консоли. Но у неё есть и обратная сторона. Когда задача усложняется, однострочники превращаются в нечитаемые конструкции с вложенными кавычками и подстановками, которые через неделю не разберёт даже автор. Умение почувствовать момент, когда пора перестать изощряться в bash и написать нормальный скрипт на Python — отдельный и важный навык. Мы ещё вернёмся к этому позже.

Автоматизируемость

Команду можно записать, повторить, вставить в скрипт. То, что вы сделали руками на одном сервере, скрипт воспроизведёт на десяти — одинаково и без ошибок. Эту же команду можно положить в систему контроля версий, запустить по расписанию, передать коллеге. Текст — универсальный формат, с которым удобно работать и людям, и программам.

С графическим интерфейсом так не получится: попробуйте описать коллеге последовательность из двадцати кликов по меню.

Минимальные требования к каналу связи

У консоли есть ещё одно преимущество, о котором редко пишут в учебниках, — она почти ничего не требует от канала связи. SSH-сессия — это буквально килобайты трафика. Вы передаёте нажатия клавиш в одну сторону и получаете текст в другую. Всё.

Это звучит как мелочь — до тех пор, пока вам не приходится чинить сервер из зала ожидания аэропорта с полумёртвым Wi-Fi или из купе поезда, где мобильный интернет пропадает каждые три минуты. SSH-сессия будет работать даже там, где соединение с удалённым рабочим столом давно бы разорвалось.

Больше того: подключиться к серверу по SSH можно даже с телефона, установив приложение-терминал. Это неудобно — набирать команды на экранной клавиатуре не пожелаешь и врагу. Но когда коллеги просят срочно посмотреть, что случилось с сервером, а вы в дороге с одним телефоном в кармане — этот вариант серьёзно выручит.

Почему серверы без графической оболочки

Серверы, как правило, устанавливают без графического интерфейса. Не из аскетизма — причины практические.

Большинство серверных программ в Linux управляются из командной строки. Графические интерфейсы к ним существуют, но это отдельные проекты, которые разрабатываются независимо от самой программы. GUI может отставать от текущей версии, не поддерживать часть настроек, реализовывать логику по-своему. По сути, вам приходится учить не одну вещь, а две: саму программу и её графическую обёртку — со своими особенностями, ограничениями и багами. А когда что-то идёт не так, всё равно приходится открывать консоль и разбираться через конфиги и логи.

Кроме того, консоль автоматизируется (скрипты, контроль версий, повторяемость), а SSH работает там, где графический интерфейс давно бы отвалился.

Всё это звучит убедительно, но пока абстрактно. Давайте перейдём к конкретике — приёмам, благодаря которым работа в консоли становится быстрой и приятной.

Tab: автодополнение

Это самый важный приём в командной строке. Вместо того чтобы вводить длинные имена файлов и команд целиком, вы нажимаете клавишу **Tab** — и оболочка дописывает за вас.

Попробуйте. Введите начало команды и нажмите Tab:

```
hos Tab
```

Оболочка дополнит до `hostname`.

Это работает и для путей к файлам. Введите:

```
cd /ho Tab
```

Оболочка дополнит до `cd /home/`. Если в `/home/` один каталог — дополнит и его:

```
cd /home/stu Tab
```

Получится `cd /home/student/`.

Двойной Tab: список вариантов

Если совпадений несколько, одно нажатие Tab не сработает. Нажмите Tab **дважды** — оболочка покажет все варианты:

```
ls /u Tab Tab


usr/  udev/
```

Допишите ещё одну букву, чтобы убрать неоднозначность, и снова нажмите Tab:

```
ls /us Tab
```

Получится `ls /usr/`.

> **Совет.** Привыкайте нажимать Tab после каждых 2–3 букв. Это не лень — это нормальная практика. Автодополнение не только экономит время, но и защищает от опечаток: если Tab не дополняет — значит, вы ввели что-то не то.

Стрелки: история команд

Оболочка запоминает все команды, которые вы вводили. Нажмите **стрелку вверх** — появится предыдущая команда. Ещё раз вверх — команда перед ней. **Стрелка вниз** — обратно к более новым командам.

Это полезно, когда нужно повторить или слегка изменить предыдущую команду. Вместо того чтобы набирать заново:

```
docker compose -f docker-compose.yml up -d
```

— просто нажмите стрелку вверх, найдите нужную команду и нажмите Enter.

Поиск по истории: Ctrl+R

Стрелки удобны для последних 3–5 команд. Если нужно найти команду, которую вы вводили час назад — используйте **Ctrl+R**.

Нажмите `Ctrl+R` — появится приглашение:

```
(reverse-i-search)`':
```

Начните вводить часть команды — например, `docker`:

```
(reverse-i-search)`docker': docker compose up -d
```

Оболочка покажет последнюю команду, содержащую `docker`. Нажмите `Ctrl+R` ещё раз, чтобы перейти к более ранним совпадениям. Когда нашли нужную команду — нажмите **Enter**, чтобы выполнить, или **стрелку вправо**, чтобы отредактировать перед выполнением.

Чтобы выйти из поиска без выполнения — нажмите `Ctrl+C`.

Команда history

Чтобы увидеть всю историю команд:

```
history
```

Оболочка покажет пронумерованный список:

```
  1  whoami
  2  hostname
  3  pwd
  4  ls /
  5  cd /tmp
  6  cd
```

Чтобы посмотреть последние 10 команд:

```
history 10
```

Горячие клавиши

В командной строке есть набор сочетаний клавиш, которые ускоряют работу. Вот самые важные — выучите их в первую очередь:

Сочетание	Действие
`Ctrl+C`	Прервать выполнение текущей команды
`Ctrl+D`	Завершить сеанс (аналог команды `exit`)
`Ctrl+L`	Очистить экран (аналог команды `clear`)
`Ctrl+A`	Перейти в начало строки
`Ctrl+E`	Перейти в конец строки
`Ctrl+W`	Удалить слово перед курсором
`Ctrl+U`	Удалить всё от курсора до начала строки
`Ctrl+K`	Удалить всё от курсора до конца строки

Ctrl+C: ваш аварийный выход

Ctrl+C — самое важное сочетание. Запомните его первым.

Если команда работает слишком долго, если вы случайно запустили что-то не то, если экран заполнился непонятным текстом — нажмите `Ctrl+C`. Это безопасно: команда прерывается, и вы возвращаетесь к обычному приглашению $.

```
ping 8.8.8.8
```

```
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=118 time=5.42 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=118 time=5.31 ms
^C
--- 8.8.8.8 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss
```

Команда `ping` работает бесконечно, пока вы не остановите её. `^C` в выводе — это и есть ваше нажатие Ctrl+C.

Ctrl+D: мягкий выход

Ctrl+D означает «конец ввода». Если вы находитесь в командной строке, Ctrl+D завершает сеанс — как команда `exit`. Если вы внутри какой-то программы (например, `python3` или `mariadb`) — Ctrl+D выходит из неё.

Ctrl+L: быстрая очистка экрана

Когда экран заполнен выводом предыдущих команд и хочется начать с чистого листа — нажмите **Ctrl+L**. Это аналог команды `clear`, только быстрее — не нужно ничего набирать.

Копирование и вставка

В обычном приложении `Ctrl+C` — это «копировать». Но в терминале `Ctrl+C` — это «прервать команду». Поэтому для копирования и вставки используются другие сочетания.

Терминалов существует много: Windows Terminal, GNOME Terminal, Konsole, xterm и другие. Почти во всех работает одна и та же схема:

Действие	Сочетание клавиш
Копировать	`Ctrl+Shift+C`
Вставить	`Ctrl+Shift+V`

Во многих терминалах также работает вставка правым кликом мыши.

Два буфера обмена в Linux

В Linux есть особенность, к которой быстро привыкаешь и без которой потом сложно жить: **два независимых буфера обмена**.

Основной буфер — работает через `Ctrl+Shift+C` / `Ctrl+Shift+V`, как описано выше. Это аналог обычного буфера обмена в Windows.

Буфер выделения (selection buffer) — работает автоматически. Вы **выделяете** текст мышью — он тут же попадает в буфер. Чтобы вставить — нажимаете **среднюю кнопку мыши** (колёсико). Никаких дополнительных клавиш.

Два буфера работают независимо. Вы можете скопировать одну строку через `Ctrl+Shift+C`, выделить мышью другую, и вставить любую из них: `Ctrl+Shift+V` вставит первую, средняя кнопка мыши — вторую.

На практике буфер выделения используется чаще: увидели нужную строку в логе — выделили мышью, перешли в другое окно — нажали среднюю кнопку. Два движения вместо четырёх.

> **Примечание.** Буфер выделения — это особенность оконной системы X11 в Linux. Он работает, если вы подключаетесь с Linux-машины или используете терминал с поддержкой X11. В Windows Terminal буфер выделения недоступен, но основной буфер работает как обычно.

Перемещение по строке

Набрали длинную команду и заметили ошибку в начале? Не нужно стирать всё. Используйте клавиши:

- **Стрелки влево/вправо** — перемещение по одному символу.
- **Ctrl+A** — прыжок в начало строки.
- **Ctrl+E** — прыжок в конец строки.
- **Ctrl+стрелка влево/вправо** — перемещение по словам (работает в большинстве терминалов).

Это работает точно так же, как перемещение курсора в текстовом редакторе, только вместо мыши вы используете клавиатуру.

Ошибки — это нормально

Новички часто боятся ввести «не ту команду» и что-то сломать. Это понятный страх, но на практике сломать что-то серьёзное одной командой сложно — особенно если вы работаете от обычного пользователя, а не от root.

Вот что произойдёт, если вы ошибётесь:

Опечатка в имени команды:

```
hotsname
```

```
-bash: hotsname: command not found
```

Оболочка просто скажет, что такой команды нет. Ничего не сломается.

Несуществующий файл:

```
cat /etc/nofile
```

```
cat: /etc/nofile: No such file or directory
```

Ничего страшного — файл не найден, команда завершилась.

Нет прав доступа:

```
cat /etc/shadow
```

```
cat: /etc/shadow: Permission denied
```

Система не даст вам прочитать файл, к которому у вас нет доступа. Никакого вреда.

> **Правило.** В Linux обычный пользователь не может повредить систему. Самое серьёзное, что может случиться — вы удалите или испортите собственные файлы в домашнем каталоге. Но даже это легко исправить на учебном сервере. Ломайте, экспериментируйте, пробуйте — так учатся все.

Многозадачность: несколько окон

Опытные администраторы часто работают с несколькими сеансами одновременно: в одном окне редактируют конфиг, в другом смотрят логи, в третьем перезапускают сервис.

Самый простой способ — открыть несколько вкладок в терминале. В большинстве терминалов:

- **Ctrl+Shift+T** — новая вкладка.
- **Ctrl+Tab** или **Ctrl+Page Down** — переключение между вкладками.

В каждой вкладке откройте отдельное SSH-соединение с сервером. Теперь вы можете одновременно следить за логами и вводить команды:

```
Вкладка 1:  nano /etc/nginx/conf.d/default.conf
Вкладка 2:  sudo systemctl restart nginx
Вкладка 3:  tail -f /var/log/nginx/error.log
```

> **Примечание.** Позже, в главе 9, мы познакомимся с программой `tmux`, которая позволяет делить один экран на несколько панелей прямо на сервере — без открытия дополнительных вкладок.

Итоги

В этой главе вы узнали приёмы, которые делают работу в консоли удобной:

- **Tab** — автодополнение команд и путей. Нажимайте после каждых 2–3 букв.
- **Стрелки вверх/вниз** — история команд. Не набирайте одно и то же дважды.
- **Ctrl+R** — поиск по истории. Находит команду по фрагменту.
- **Ctrl+C** — аварийный выход. Прерывает любую команду.
- **Ctrl+L** — очистка экрана. Быстрее, чем `clear`.
- **Ctrl+A** / **Ctrl+E** — прыжок в начало и конец строки.
- **Копирование/вставка** — через `Ctrl+Shift+C/V`, а в Linux ещё и через выделение мышью + среднюю кнопку (два независимых буфера обмена).

Эти приёмы кажутся мелочью, но именно из них складывается ощущение «мне тут удобно». Попробуйте использовать их с самого начала — и вы быстро поймёте, почему администраторы предпочитают консоль.

Этап 1. Первое знакомство с Linux

Что такое Linux

Когда говорят «Linux», обычно имеют в виду операционную систему — программу, которая управляет компьютером: распределяет память, работает с дисками, обеспечивает запуск других программ. Если точнее, Linux — это **ядро** операционной системы. Само по себе ядро бесполезно для пользователя, поэтому вокруг него собирают набор программ, утилит и настроек. Такой готовый набор называется **дистрибутивом**.

Дистрибутивов существуют десятки: Ubuntu, Fedora, Arch, CentOS и другие. В этом курсе мы используем **Debian 13 (trixie)**. Debian — один из старейших дистрибутивов, известный своей стабильностью. Именно на его основе построен Ubuntu — самый популярный дистрибутив для настольных компьютеров. На серверах Debian используется повсеместно.

Linux работает не только на серверах. Ядро Linux лежит в основе Android, его используют на суперкомпьютерах, в маршрутизаторах и умных устройствах. Но для нас главное — это серверы. Подавляющее большинство веб-сайтов, баз данных и облачных сервисов работают именно на Linux.

Подключение по SSH

В этом курсе мы работаем с Linux через командную строку — без графического интерфейса. Это именно тот режим, в котором работают системные администраторы и DevOps-инженеры.

Для подключения к серверу используется **SSH** (Secure Shell) — протокол безопасного удалённого доступа. Данные, включая пароль, передаются в зашифрованном виде.

Подключаемся из Windows

В Windows 10 и 11 SSH-клиент уже встроен. Откройте **Windows Terminal** (или PowerShell) и введите команду:

```
ssh student@192.168.1.100
```

Здесь `student` — имя пользователя на сервере, а `192.168.1.100` — IP-адрес сервера. Оба значения вам сообщит преподаватель.

> **Адрес и имя пользователя в примерах.** Во всех примерах этого курса мы используем `student@192.168.1.100`. В вашем случае адрес и имя пользователя могут отличаться — подставляйте свои.

Первое подключение

При первом подключении к серверу SSH спросит, доверяете ли вы этому хосту:

```
The authenticity of host '192.168.1.100' can't be established.
ED25519 key fingerprint is SHA256:xxxxxxxxxxxxxxxxxxxxxxxxxxx.
Are you sure you want to continue connecting (yes/no/[fingerprint])?
```

Введите `yes` и нажмите Enter. Это нормальное поведение — SSH запоминает сервер, и при следующих подключениях этот вопрос больше не появится.

После этого введите пароль. **При вводе пароля символы не отображаются** — ни звёздочки, ни точки. Это не ошибка, просто вводите пароль и нажимайте Enter.

Отключение

Чтобы завершить сеанс и отключиться от сервера, введите:

```
exit
```

Или нажмите `Ctrl+D`.

Командная строка: первый взгляд

После успешного подключения вы увидите **приглашение командной строки** (prompt) — примерно такое:

```
student@debian:~$
```

Разберём его по частям:

Часть	Значение
`student`	имя пользователя, под которым вы вошли в систему
`@`	разделитель
`debian`	имя компьютера (хоста)
`:`	разделитель
`~`	текущий каталог (~ означает «домашний каталог пользователя»)
`$`	признак обычного пользователя

Символ $ в конце приглашения означает, что вы работаете как обычный пользователь. У администратора (root) вместо $ стоит #. Пока запомните: **видите $ — всё нормально**. Работа от имени root рассматривается в главе 08.

Первые команды

Пришло время попробовать. Вводите команды после символа $ и нажимайте Enter.

whoami — кто я

```
whoami
```

```
student
```

Команда выводит имя текущего пользователя. Полезно, когда вы не уверены, под какой учётной записью работаете.

hostname — имя компьютера

```
hostname
```

```
debian
```

Показывает сетевое имя машины. На сервере это может быть что-то вроде `web-server-01` или `db-prod-03`.

pwd — где я нахожусь

```
pwd
```

```
/home/student
```

pwd расшифровывается как *print working directory* — «показать рабочий каталог». В Linux нет дисков `C:` и `D:`, как в Windows. Вся файловая система начинается с корня `/`, а каталоги разделяются символом `/` (прямой слеш, а не обратный \, как в Windows).

`/home/student` — это ваш **домашний каталог**, личное пространство. Именно сюда вы попадаете при входе в систему.

ls — что здесь лежит

```
ls
```

Команда выводит содержимое текущего каталога. Если каталог пуст, команда не выведет ничего — это нормально. В свежей системе домашний каталог обычно пустой.

Попробуйте посмотреть, что лежит в корне файловой системы:

```
ls /
```

```
bin  boot  dev  etc  home  lib  media  mnt  opt  proc  root  run  sbin  srv
↪  sys  tmp  usr  var
```

Здесь вы видите основные системные каталоги. Подробно мы разберём их в следующей главе.

cd — перейти в другой каталог

```
cd /tmp
```

Команда **cd** (*change directory*) меняет текущий каталог. После выполнения обратите внимание — приглашение командной строки изменилось:

```
student@debian:/tmp$
```

Вместо ~ теперь показывается `/tmp`. Убедитесь командой `pwd`:

```
pwd
```

```
/tmp
```

Чтобы вернуться в домашний каталог, введите `cd` без аргументов:

```
cd
```

```
student@debian:~$
```

clear — очистить экран

```
clear
```

Экран очищается. Команды и их вывод никуда не пропадают — просто экран прокручивается вверх. Можно прокрутить обратно, чтобы увидеть историю.

exit — выйти

```
exit
```

Завершает текущий сеанс. Если вы подключены по SSH — соединение закроется.

Как получить помощь

Запомнить все команды и их параметры невозможно — и не нужно. Важнее уметь быстро найти нужную информацию.

man — встроенное руководство

В Linux есть встроенная система справки — **man** (от слова *manual*). Чтобы прочитать руководство по любой команде, введите:

```
man ls
```

Откроется страница с подробным описанием команды `ls`: что она делает, какие параметры принимает, примеры использования.

Навигация по man-странице:

Клавиша	Действие
`Пробел` или `Page Down`	следующая страница
`b` или `Page Up`	предыдущая страница
стрелки `вверх` / `вниз`	прокрутка по строкам
`/текст` + Enter	поиск текста на странице
`n`	следующее совпадение поиска
`q`	выход из man

Самое важное: **`q` — выход**. Новички часто пугаются, попав в man, потому что не знают, как выйти. Просто нажмите `q`.

–help — краткая справка

Большинство команд поддерживают флаг `--help`, который выводит краткую справку прямо в терминал:

```
ls --help
```

Это удобнее, чем man, когда нужно быстро вспомнить нужный параметр.

> **Совет.** Не пытайтесь запомнить всё сразу. Даже опытные администраторы регулярно заглядывают в man-страницы и ищут информацию в интернете. Главный навык — умение находить ответ, а не держать всё в голове.

Практическое задание

1. Подключитесь к серверу по SSH из Windows Terminal.
2. Выполните команды `whoami`, `hostname`, `pwd` и запишите результат.
3. Выполните `ls /` и посмотрите, какие каталоги есть в корне системы.
4. Перейдите в каталог `/var` командой `cd /var`, убедитесь командой `pwd`.
5. Посмотрите содержимое `/var` командой `ls`.
6. Вернитесь в домашний каталог (`cd` без аргументов), проверьте с помощью `pwd`.
7. Откройте man-страницу для команды `hostname` (`man hostname`), найдите описание, выйдите клавишей `q`.
8. Отключитесь от сервера командой `exit`.

Итоги

В этой главе вы:

- узнали, что Linux — это ядро, а Debian — дистрибутив;
- подключились к серверу по SSH;
- познакомились с приглашением командной строки;
- освоили базовые команды: `whoami`, `hostname`, `pwd`, `ls`, `cd`, `clear`, `exit`;
- научились пользоваться справкой: `man` и `--help`.

В следующей главе мы подробно разберём файловую систему Linux: как она устроена, как создавать, копировать, перемещать и удалять файлы и каталоги.

Этап 2. Файловая система

Всё — файл

В Linux есть фундаментальный принцип: **всё является файлом**. Обычные файлы, каталоги, устройства (диски, клавиатура), сетевые сокеты — система представляет их как файлы. Это звучит странно, но на практике означает одно: для работы со всем этим используются одни и те же инструменты.

Ещё одно важное отличие от Windows: в Linux нет дисков `C:`, `D:` и так далее. Вся файловая система начинается с единственного корня — `/`. Все диски, флешки и сетевые хранилища «подключаются» (монтируются) как каталоги внутри этого дерева.

Структура каталогов

В главе 01 мы выполнили `ls /` и увидели набор каталогов в корне. Это не случайный набор — в Linux существует стандарт **FHS** (Filesystem Hierarchy Standard), который определяет, что где лежит.

Вот основные каталоги, с которыми вы будете сталкиваться:

Каталог	Назначение
`/`	корень файловой системы, начало всего дерева
`/home`	домашние каталоги пользователей (`/home/student`, `/home/admin` и т.д.)
`/root`	домашний каталог суперпользователя root
`/etc`	конфигурационные файлы системы и программ
`/var`	изменяемые данные: логи, почта, базы данных
`/tmp`	временные файлы (очищается при перезагрузке)
`/usr`	программы и библиотеки, установленные в системе
`/bin`	основные исполняемые файлы (команды)
`/sbin`	системные команды (обычно для администратора)
`/opt`	дополнительное программное обеспечение
`/dev`	файлы устройств (диски, терминалы и др.)
`/proc`	виртуальная файловая система с информацией о процессах и ядре

Привязка к проекту. Когда мы будем устанавливать nginx и WordPress, вы увидите эти каталоги в деле: - конфигурация nginx — в `/etc/nginx/` - логи nginx — в `/var/log/nginx/` - файлы сайта — в `/var/www/`

Пока просто запомните, что конфиги живут в `/etc`, логи — в `/var/log`, а данные сайтов — в `/var/www`.

Абсолютные и относительные пути

В Linux есть два способа указать расположение файла или каталога.

Абсолютный путь начинается с `/` и описывает полный маршрут от корня:

```
/home/student/documents/notes.txt
```

Относительный путь отсчитывается от текущего каталога и не начинается с `/`:

```
documents/notes.txt
```

Если вы находитесь в `/home/student`, оба пути указывают на один и тот же файл.

Для относительных путей используются специальные обозначения:

Обозначение	Значение
`.`	текущий каталог
`..`	родительский каталог (на уровень вверх)
`~`	домашний каталог текущего пользователя

Примеры:

```
# Подняться на уровень вверх
cd ..

# Подняться на два уровня вверх
cd ../..

# Перейти в соседний каталог (вверх, потом вниз)
cd ../other-directory
```

Навигация

cd — перемещение по каталогам (подробнее)

В главе 01 мы уже использовали `cd`. Вот полезные приёмы:

```
# Перейти в каталог по абсолютному пути
cd /var/log

# Подняться на уровень вверх
cd ..

# Вернуться в домашний каталог
cd
# или
cd ~

# Вернуться в предыдущий каталог (где были до последнего cd)
cd -
```

Последний вариант (`cd -`) особенно удобен: он работает как кнопка «назад» и переключает вас между двумя каталогами.

ls — просмотр содержимого каталога (подробнее)

В главе 01 мы использовали `ls` без параметров. Теперь разберём полезные флаги.

`ls -l` — подробный (длинный) формат:

```
ls -l /etc
```

```
drwxr-xr-x 2 root root 4096 янв 15 10:30 apt
-rw-r--r-- 1 root root 2981 фев  3 09:15 bash.bashrc
-rw-r--r-- 1 root root  367 мар 10 14:22 hostname
```

Разберём вывод по столбцам:

Столбец	Пример	Значение
1	`drwxr-xr-x`	тип файла и права доступа (подробно — в главе 03)
2	`2`	количество жёстких ссылок
3	`root`	владелец
4	`root`	группа
5	`4096`	размер в байтах
6	`янв 15 10:30`	дата последнего изменения
7	`apt`	имя файла или каталога

Первый символ в столбце прав показывает тип: - `d` — каталог (directory) - `-` — обычный файл - `l` — символическая ссылка (аналог ярлыка в Windows)

`ls -a` — показать скрытые файлы:

```
ls -a ~
```

```
.  ..  .bash_history  .bashrc  .profile
```

В Linux файлы, имя которых начинается с точки (`.`), считаются **скрытыми**. Обычный `ls` их не показывает. Это не защита — просто способ не засорять вывод служебными файлами.

`ls -la` — подробный формат + скрытые файлы (комбинация флагов):

```
ls -la ~
```

`ls -lh` — подробный формат с читаемыми размерами (human-readable):

```
ls -lh /var/log
```

```
-rw-r----- 1 root adm  15K мар 10 14:00 auth.log
-rw-r--r-- 1 root root 248K мар  9 06:25 dpkg.log
```

Вместо `15360` показывается `15K`, вместо `253952` — `248K`. Размеры выводятся в килобайтах (K), мегабайтах (M) или гигабайтах (G).

> **Совет.** Флаги можно комбинировать: `ls -la`, `ls -lah`. Порядок флагов не важен: `ls -la` и `ls -al` — одно и то же.

Создание файлов и каталогов

touch — создать пустой файл

```
touch myfile.txt
```

Если файл уже существует, `touch` обновит его дату последнего изменения, но не изменит содержимое.

mkdir — создать каталог

```
mkdir projects
```

Если нужно создать вложенную структуру каталогов, где промежуточные каталоги ещё не существуют, используйте флаг `-p`:

```
mkdir -p projects/website/config
```

Без `-p` команда выдаст ошибку, если каталог `projects/website` не существует. С `-p` она создаст всю цепочку.

Практика: структура каталогов для проекта

Подготовим структуру каталогов для нашего будущего сайта на WordPress. Пока мы создадим её в домашнем каталоге — для тренировки:

```
cd ~
mkdir -p wordpress-project/config
mkdir -p wordpress-project/logs
mkdir -p wordpress-project/site
```

Проверим результат:

```
ls -la wordpress-project/
```

```
drwxr-xr-x 2 student student 4096 мар 10 15:00 config
drwxr-xr-x 2 student student 4096 мар 10 15:00 logs
drwxr-xr-x 2 student student 4096 мар 10 15:00 site
```

Создадим несколько файлов для экспериментов:

```
touch wordpress-project/config/nginx.conf
touch wordpress-project/config/php.ini
touch wordpress-project/logs/access.log
touch wordpress-project/logs/error.log
touch wordpress-project/site/index.html
```

Копирование, перемещение, удаление

cp — копирование

Копирование файла:

```
cp wordpress-project/config/nginx.conf
↪  wordpress-project/config/nginx.conf.backup
```

Копирование каталога целиком — обязательно с флагом `-r` (recursive):

```
cp -r wordpress-project/config wordpress-project/config-backup
```

Без `-r` команда `cp` откажется копировать каталог и выдаст ошибку.

mv — перемещение и переименование

Переименование файла (перемещение в тот же каталог, но с другим именем):

```
mv wordpress-project/site/index.html wordpress-project/site/index.php
```

Перемещение файла в другой каталог:

```
mv wordpress-project/site/index.php wordpress-project/
```

Перемещение с одновременным переименованием:

```
mv wordpress-project/index.php wordpress-project/site/main.php
```

> **Важно.** `mv` в Linux не спрашивает подтверждения. Если файл с таким именем уже существует в целевом каталоге — он будет перезаписан без предупреждения.

rm — удаление

Удаление файла:

```
rm wordpress-project/site/main.php
```

Удаление каталога со всем содержимым — флаг `-r` (recursive):

```
rm -r wordpress-project/config-backup
```

Удаление пустого каталога (альтернатива):

```
rmdir wordpress-project/logs
```

`rmdir` удалит каталог только если он пуст. Если внутри есть файлы — будет ошибка.

> **Осторожно с `rm -rf`!**
>
> Команда `rm -rf` удаляет каталог со всем содержимым **без запроса подтверждения** и **без возможности восстановления**. В Linux нет «Корзины» в командной строке. Удалённые файлы восстановить крайне сложно.

Особенно опасно:

```
# НИКОГДА НЕ ДЕЛАЙТЕ ЭТО:
rm -rf /
```

Эта команда уничтожит всю файловую систему. Современные системы защищены от этого конкретного случая, но вариации вроде `rm -rf /*` по-прежнему опасны.

Правило: **всегда дважды проверяйте путь перед выполнением `rm -rf`**.

Просмотр содержимого файлов

cat — вывести файл целиком

```
cat /etc/hostname
```

```
debian
```

`cat` выводит всё содержимое файла разом. Для коротких файлов это удобно, но для больших файлов (например, логов на тысячи строк) экран заполнится текстом, и вы увидите только конец файла.

less — постраничный просмотр

```
less /etc/bash.bashrc
```

`less` открывает файл в режиме чтения, как `man`. Навигация такая же:

Клавиша	Действие
`Пробел` / `Page Down`	следующая страница
`b` / `Page Up`	предыдущая страница
стрелки `вверх` / `вниз`	прокрутка по строкам
`/текст` + Enter	поиск
`n`	следующее совпадение
`q`	выход

Используйте `less` для просмотра больших файлов — логов, конфигураций, исходного кода.

head — начало файла

```
head /etc/passwd
```

Выводит первые 10 строк файла. Можно указать другое количество:

```
head -n 5 /etc/passwd
```

Выведет первые 5 строк.

tail — конец файла

```
tail /etc/passwd
```

Выводит последние 10 строк. Аналогично можно указать количество:

```
tail -n 3 /etc/passwd
```

`tail` особенно полезен для просмотра логов — свежие записи добавляются в конец файла:

```
tail /var/log/syslog
```

> **Полезный приём.** Команда `tail -f` позволяет следить за файлом в реальном времени — новые строки будут появляться на экране по мере их добавления. Это незаменимо при отладке работающих сервисов:
>
> ```
> tail -f /var/log/syslog
> ```
>
> Для выхода нажмите `Ctrl+C`.

Врезка: кодировки и переводы строк

Если вы когда-нибудь переносили текстовые файлы с Windows на Linux (или наоборот), то могли столкнуться со странными символами или «слипшимися» строками. Вот почему это происходит.

Кодировки

Кодировка — это таблица соответствия между символами и числами. Компьютер хранит только числа, и кодировка говорит ему, какое число соответствует какой букве.

В Windows исторически использовались разные кодировки для разных языков: CP1251 для русского, CP1252 для западноевропейских и так далее. В Linux стандартом является **UTF-8** — универсальная кодировка, которая поддерживает символы всех языков мира.

Если открыть файл в неправильной кодировке, вместо русского текста будут «кракозябры». Решение — убедиться, что и отправитель, и получатель используют одну кодировку. Сегодня лучший выбор — всегда использовать UTF-8.

Переводы строк

В конце каждой строки текстового файла стоит невидимый символ перевода строки. Проблема в том, что Windows и Linux используют **разные символы**:

Система	Символ перевода строки	Обозначение
Linux / macOS	LF (Line Feed)	`\n`
Windows	CR+LF (Carriage Return + Line Feed)	`\r\n`

Когда вы переносите файл из Windows в Linux, лишний символ `\r` (CR) может вызвать проблемы. Например, bash-скрипт, написанный в Windows, может не запускаться на Linux с ошибкой `\r: command not found`.

Как проверить

Команда `file` определяет тип файла и показывает информацию о кодировке и формате переводов строк:

```
file /etc/hostname
```

```
/etc/hostname: ASCII text
```

Если файл создан в Windows, вы увидите:

```
script.sh: Bash script, ASCII text, with CRLF line terminators
```

Фраза **CRLF line terminators** означает, что файл использует Windows-формат переводов строк.

> **Совет.** Если вы редактируете файлы в Windows и переносите их на Linux-сервер, используйте редактор, который умеет сохранять в формате LF (например, VS Code — в строке состояния можно переключить CRLF на LF).

Практическое задание

1. Перейдите в домашний каталог. Создайте структуру каталогов:

   ```
   ~/project/
   ├── backup/
   ├── config/
   ├── data/
   │   ├── images/
   │   └── uploads/
   └── logs/
   ```

   Подсказка: используйте `mkdir -p`.

2. Создайте файлы:
   - `~/project/config/app.conf`
   - `~/project/data/readme.txt`
   - `~/project/logs/app.log`
3. Скопируйте файл `app.conf` в каталог `backup/` с именем `app.conf.bak`.
4. Переименуйте `readme.txt` в `README.txt`.
5. Скопируйте каталог `config/` целиком в `backup/` (чтобы получился `backup/config/`).
6. Удалите файл `app.log`.
7. Посмотрите содержимое файла `/etc/os-release` командой `cat`. Какой дистрибутив и какая версия установлены?
8. Посмотрите первые 5 строк файла `/etc/passwd` с помощью `head`.
9. Выполните `file /etc/passwd` — какой тип файла и кодировку показывает команда?

10. Проверьте результат: выполните `ls -laR ~/project/` (флаг `-R` выводит содержимое всех подкаталогов рекурсивно).

Итоги

В этой главе вы:

- узнали принцип «всё — файл» и структуру каталогов Linux (FHS);
- разобрались с абсолютными и относительными путями;
- научились навигации: `cd` с аргументами `..`, `-`, `~`;
- освоили подробный вывод `ls`: флаги `-l`, `-a`, `-h`;
- научились создавать файлы и каталоги: `touch`, `mkdir -p`;
- работали с файлами: `cp`, `mv`, `rm`, `rmdir`;
- просматривали содержимое файлов: `cat`, `less`, `head`, `tail`;
- узнали про кодировки (UTF-8 vs CP1251) и переводы строк (LF vs CRLF).

В следующей главе мы подробно разберём **права доступа** — кто может читать, изменять и запускать файлы в Linux.

Этап 3. Права доступа и владение файлами

Зачем нужны права доступа

Linux — многопользовательская система. На одном сервере могут работать десятки пользователей, и каждый из них не должен случайно (или намеренно) читать, изменять или удалять чужие файлы. Для этого в Linux существует система **прав доступа**.

Права доступа определяют, **кто** и **что** может делать с каждым файлом и каталогом.

> **Привязка к проекту.** Когда мы будем настраивать nginx и WordPress, права доступа станут критически важны. Веб-сервер nginx работает от имени специального пользователя `www-data`. Файлы сайта должны быть доступны ему для чтения, но при этом посторонние пользователи не должны иметь возможности их изменять. Неправильные права — одна из самых частых причин ошибок «403 Forbidden» и проблем с безопасностью.

Владелец, группа, остальные

Каждый файл в Linux имеет:

- **владельца** (user, u) — обычно того, кто создал файл;
- **группу** (group, g) — набор пользователей, которым нужен общий доступ;
- **остальных** (others, o) — все, кто не является владельцем и не входит в группу.

Посмотрим на знакомый вывод `ls -l`:

```
-rw-r--r-- 1 student student 0 мар 10 15:00 notes.txt
```

Здесь: - третий столбец (`student`) — **владелец** файла; - четвёртый столбец (`student`) — **группа** файла.

При создании файла владельцем становится текущий пользователь, а группой — его основная группа (обычно совпадает с именем пользователя).

Три типа прав: чтение, запись, выполнение

Для каждой категории (владелец, группа, остальные) устанавливаются три вида прав:

Право	Буква	Для файла	Для каталога
Чтение	`r` (read)	просмотр содержимого	просмотр списка файлов (`ls`)
Запись	`w` (write)	изменение содержимого	создание и удаление файлов внутри
Выполнение	`x` (execute)	запуск как программы	вход в каталог (`cd`)

Обратите внимание на важный нюанс: **для каталогов права работают иначе, чем для файлов**. В частности:

- `r` без `x` на каталоге позволяет увидеть имена файлов, но не их атрибуты (размер, дату и т.д.) и не позволяет войти внутрь;
- `x` без `r` позволяет войти в каталог и обратиться к файлу по имени, но не позволяет получить список файлов;
- `w` без `x` бесполезен — нельзя создавать файлы в каталоге, в который нельзя войти.

На практике каталоги почти всегда имеют права `r` и `x` вместе.

Чтение строки прав

Вернёмся к выводу `ls -l` и разберём первый столбец:

```
-rw-r--r--
```

Эта строка из 10 символов состоит из четырёх частей:

```
-  rw-  r--  r--
│   │    │    │
│   │    │    └── права для остальных (others): r-- = только чтение
│   │    └─────── права для группы (group):     r-- = только чтение
│   └──────────── права для владельца (user):   rw- = чтение и запись
└─────────────── тип файла:                     -   = обычный файл
```

Каждая тройка — это три позиции: `r`, `w`, `x`. Если право есть — стоит соответствующая буква, если нет — прочерк (`-`).

Примеры

Строка прав	Тип	Владелец	Группа	Остальные	Описание
`-rwxr-xr-x`	файл	rwx	r-x	r-x	Исполняемый файл, все могут запускать
`-rw-r--r--`	файл	rw-	r–	r–	Обычный файл, все могут читать
`-rw-------`	файл	rw-	—	—	Только владелец может читать и писать
`drwxr-xr-x`	каталог	rwx	r-x	r-x	Обычный каталог, все могут заходить и смотреть
`drwx------`	каталог	rwx	—	—	Каталог только для владельца

Числовой (восьмеричный) формат

Помимо буквенного представления, права можно записать числами. Каждому праву соответствует число:

Право	Число
`r` (чтение)	4
`w` (запись)	2
`x` (выполнение)	1
нет права	0

Числа складываются для каждой тройки:

Комбинация	Сумма	Значение
`rwx`	4+2+1 = **7**	все права
`rw-`	4+2+0 = **6**	чтение и запись
`r-x`	4+0+1 = **5**	чтение и выполнение
`r--`	4+0+0 = **4**	только чтение
`---`	0+0+0 = **0**	нет прав

Три цифры подряд задают права для владельца, группы и остальных:

Числовой формат	Буквенный	Типичное применение
`755`	`rwxr-xr-x`	исполняемые файлы, каталоги
`644`	`rw-r--r--`	обычные файлы (конфиги, тексты)
`700`	`rwx------`	личный каталог, приватные скрипты
`600`	`rw-------`	файлы с паролями, ключи SSH

> **Совет.** Запомните две комбинации: **644** для файлов и **755** для каталогов и скриптов. Это самые распространённые наборы прав.

chmod — изменение прав

Команда `chmod` (change mode) изменяет права доступа. Она работает в двух форматах.

Символьный формат

```
chmod <кто><действие><права> файл
```

Где: - **кто**: `u` (владелец), `g` (группа), `o` (остальные), `a` (все) - **действие**: `+` (добавить), `-` (убрать), `=` (установить точно) - **права**: `r`, `w`, `x`

Примеры:

```
# Добавить право на выполнение для владельца
chmod u+x script.sh

# Убрать право на запись у группы и остальных
chmod go-w config.txt
```

```
# Добавить чтение для всех
chmod a+r readme.txt

# Установить точные права: владелец — всё, группа и остальные — только чтение
chmod u=rwx,go=r file.txt
```

Числовой формат

```
chmod <число> файл
```

Примеры:

```
# Стандартные права для файла: владелец rw, остальные r
chmod 644 config.txt

# Стандартные права для каталога или скрипта
chmod 755 scripts/

# Только владелец может читать и писать
chmod 600 secret.key

# Применить рекурсивно ко всему каталогу и его содержимому
chmod -R 755 wordpress-project/
```

Осторожно с `-R`. Флаг `-R` (recursive) применяет права ко всем файлам и подкаталогам внутри. Это удобно, но помните: обычным файлам редко нужен бит `x`, а каталогам он нужен почти всегда. Поэтому `chmod -R 755` на каталог с файлами — не всегда хорошая идея (файлы станут исполняемыми).

Почему `chmod 777` — плохая идея

Если вы будете искать решения проблем в интернете, вы неизбежно встретите совет: «просто сделайте `chmod -R 777`». Не следуйте ему. Разберёмся, почему.

Права `777` означают `rwxrwxrwx` — **любой** пользователь системы может читать, изменять и выполнять файл. Это полное отсутствие защиты.

Чем это опасно на практике:

- **Безопасность.** Если на веб-сервере файлы WordPress имеют права `777`, любой процесс (включая взломанный скрипт) сможет изменить код сайта, подложить вредоносный файл или прочитать конфигурацию с паролями к базе данных.
- **Маскировка проблем.** `chmod 777` не устраняет причину ошибки — он просто снимает все ограничения. Настоящая проблема (неправильный владелец, не тот пользователь у сервиса) остаётся скрытой и может всплыть позже.
- **Привычка.** Если привыкнуть «чинить» всё через `777`, вы никогда не разберётесь, как права доступа работают на самом деле.

Что делать вместо этого:

1. Определите, **какой пользователь** и **какой процесс** должен обращаться к файлу (например, `www-data` для nginx).

2. Назначьте правильного **владельца** (`chown`) и **группу** (`chgrp`).
3. Выставьте **минимально необходимые права**: обычно `644` для файлов и `755` для каталогов.

Правильная настройка требует чуть больше усилий, но зато вы точно понимаете, кто и к чему имеет доступ, — а это основа безопасного администрирования.

Разные права для файлов и каталогов

На практике часто нужно выставить одни права на каталоги (например, `755`) и другие на файлы (`644`). Это типичная ситуация при настройке веб-сайта.

Если структура простая и файлы лежат в одном каталоге, можно обойтись двумя командами:

```
# Сначала выставляем права на сам каталог
chmod 755 wordpress-project/site
# Затем на файлы внутри
chmod 644 wordpress-project/site/*
```

Но если каталогов много и они вложенные, этот способ не сработает. Правильное решение — команда `find`, которую мы подробно разберём в главе 05. Здесь покажем паттерн, который вам пригодится в реальной работе:

```
# Выставить 755 на все каталоги внутри /var/www/site
find /var/www/site -type d -exec chmod 755 {} +

# Выставить 644 на все файлы внутри /var/www/site
find /var/www/site -type f -exec chmod 644 {} +
```

Первая команда находит все каталоги (`-type d`) и применяет к ним `chmod 755`. Вторая находит все обычные файлы (`-type f`) и применяет `chmod 644`.

Не нужно сейчас запоминать синтаксис `find` — просто знайте, что такой приём существует. Когда дойдём до главы 05, вы поймёте, как он работает.

chown и chgrp — смена владельца и группы

chown — смена владельца

```
chown admin notes.txt
```

Сменить владельца и группу одновременно:

```
chown admin:developers notes.txt
```

Рекурсивно — для каталога и всего его содержимого:

```
chown -R admin:developers project/
```

chgrp — смена группы

```
chgrp developers notes.txt
```

> **Важно.** Менять владельца файла может только **root** (суперпользователь). Обычный пользователь может менять группу файла — но только на группу, в которой он сам состоит. Подробнее о root, sudo и управлении пользователями — в главе 08.
>
> Пока вы работаете под своей учётной записью, вы можете менять права (`chmod`) только на свои файлы. Для `chown` вам понадобится `sudo` — эту команду мы разберём позже.

Практика: права для проекта WordPress

Вернёмся к структуре каталогов из главы 02. Если вы её удалили — создайте заново:

```
cd ~
mkdir -p wordpress-project/{config,logs,site}
touch wordpress-project/config/nginx.conf
touch wordpress-project/config/php.ini
touch wordpress-project/site/index.html
```

> **Что за фигурные скобки?** Запись `{config,logs,site}` — это механизм bash, который называется **brace expansion** (раскрытие скобок). Оболочка автоматически разворачивает такую запись в несколько слов. То есть команда
>
> ```
> mkdir -p wordpress-project/{config,logs,site}
> ```
>
> равносильна трём командам:
>
> ```
> mkdir -p wordpress-project/config
> mkdir -p wordpress-project/logs
> mkdir -p wordpress-project/site
> ```
>
> Фигурные скобки — просто удобное сокращение, чтобы не набирать одно и то же несколько раз. Внутри скобок элементы перечисляются через запятую **без пробелов**.

Посмотрим текущие права:

```
ls -la wordpress-project/
```

```
drwxr-xr-x 5 student student 4096 мар 10 16:00 .
drwxr-xr-x 3 student student 4096 мар 10 16:00 ..
drwxr-xr-x 2 student student 4096 мар 10 16:00 config
drwxr-xr-x 2 student student 4096 мар 10 16:00 logs
drwxr-xr-x 2 student student 4096 мар 10 16:00 site
```

Задание 1. Изучите текущие права

```
ls -la wordpress-project/config/
```

Обратите внимание: файлы `nginx.conf` и `php.ini` имеют права `644` (`rw-r--r--`). Это значит, что владелец может читать и писать, а все остальные — только читать.

Задание 2. Сделайте конфиги доступными только владельцу

Конфигурационные файлы могут содержать пароли. Уберём доступ для группы и остальных:

```
chmod 600 wordpress-project/config/nginx.conf
chmod 600 wordpress-project/config/php.ini
```

Проверим:

```
ls -l wordpress-project/config/
```

```
-rw------- 1 student student 0 мар 10 16:00 nginx.conf
-rw------- 1 student student 0 мар 10 16:00 php.ini
```

Теперь файлы доступны только владельцу.

Задание 3. Создайте скрипт и сделайте его исполняемым

Создадим простой скрипт (содержимое пока не важно — подробно о скриптах в главе 06):

```
echo '#!/bin/bash' > wordpress-project/backup.sh
echo 'echo "Backup started"' >> wordpress-project/backup.sh
```

> **Что делают > и >>?** Символ > записывает вывод команды в файл (создаёт или перезаписывает), а >> добавляет в конец файла, не затирая содержимое. Подробно мы разберём перенаправление ввода-вывода в главе 05.

Попробуем запустить:

```
./wordpress-project/backup.sh
```

```
bash: ./wordpress-project/backup.sh: Отказано в доступе
```

Файл не имеет права на выполнение. Добавим:

```
chmod u+x wordpress-project/backup.sh
```

Проверим и запустим:

```
ls -l wordpress-project/backup.sh
```

```
-rwxr--r-- 1 student student 38 мар 10 16:05 backup.sh
```

```
./wordpress-project/backup.sh
```

```
Backup started
```

Задание 4. Ограничьте доступ к каталогу logs

```
chmod 700 wordpress-project/logs
```

Теперь только владелец может заходить в этот каталог и видеть его содержимое.

Специальные биты (обзор)

Помимо стандартных `rwx` в Linux существуют три специальных бита. Подробно они вам понадобятся не скоро, но полезно знать об их существовании:

- **SUID** — при запуске файла он выполняется от имени владельца, а не от имени того, кто запустил. Используется, например, командой `passwd`, чтобы обычный пользователь мог изменить свой пароль.
- **SGID** — аналогично, но для группы. На каталоге означает, что новые файлы внутри наследуют группу каталога.
- **Sticky bit** — на каталоге запрещает пользователям удалять чужие файлы. Классический пример — каталог `/tmp`.

Вы можете заметить их в выводе `ls -l` как `s` или `t` вместо `x`:

```
drwxrwxrwt  — sticky bit на /tmp (буква t вместо последнего x)
-rwsr-xr-x  — SUID на /usr/bin/passwd (буква s вместо x владельца)
```

Практическое задание

1. Создайте файл `secret.txt` в домашнем каталоге. Установите права `600`. Убедитесь с помощью `ls -l`, что только владелец может его читать и писать.
2. Создайте каталог `shared/`. Установите права `755`. Создайте внутри файл `info.txt` с правами `644`. Объясните, кто и что может делать с этим файлом и каталогом.
3. Создайте файл `run.sh`. Добавьте ему право на выполнение для владельца с помощью символьного формата (`chmod u+x`). Проверьте результат.
4. Посмотрите права на каталог `/tmp` (`ls -ld /tmp`). Какой специальный бит вы видите? Что он означает?
5. Переведите в числовой формат:
   - `rwxr-xr--` = ?
   - `rw-rw----` = ?
   - `rwx------` = ?
6. Переведите в буквенный формат:
   - `644` = ?
   - `711` = ?
   - `000` = ?

Итоги

В этой главе вы:

- узнали, зачем нужны права доступа в многопользовательской системе;
- разобрались с концепцией «владелец / группа / остальные»;
- научились читать строку прав в выводе `ls -l`;
- освоили числовой (восьмеричный) формат прав;
- научились изменять права командой `chmod` (символьный и числовой формат);
- познакомились с `chown` и `chgrp` для смены владельца и группы;
- узнали о существовании специальных битов (SUID, SGID, sticky bit).

В следующей главе мы освоим **текстовые редакторы** — nano и основы vim. Они понадобятся для редактирования конфигурационных файлов на сервере.

Этап 4. Текстовые редакторы: nano и vim

Зачем нужен текстовый редактор в терминале

В предыдущих главах мы создавали файлы командами `touch` и `echo`, а для просмотра использовали `cat` и `less`. Но что, если нужно **изменить** файл — исправить строку в конфигурации, добавить параметр, убрать лишнее?

В Windows вы бы открыли файл в Блокноте. В Linux на сервере графического интерфейса нет — есть только терминал. Поэтому нужен **текстовый редактор, работающий прямо в командной строке**.

Таких редакторов существует несколько. Мы разберём два:

- **nano** — простой и понятный, подходит для повседневной работы;
- **vim** — мощный, но со своеобразным управлением. Мы изучим только самый минимум, чтобы вы не растерялись, если vim откроется неожиданно.

  Привязка к проекту. Совсем скоро (в главе 11) мы будем настраивать nginx и PHP-FPM. Для этого потребуется редактировать конфигурационные файлы прямо на сервере. Навык работы с текстовым редактором в терминале — обязательный инструмент системного администратора.

nano — основной редактор

Установка

В Debian 13 nano обычно уже установлен. Проверить можно так:

```
nano --version
```

Если команда выведет версию — всё в порядке. Если нет — установите nano командой:

```
sudo apt install nano
```

(Команду `sudo` и установку пакетов мы подробно разберём в главах 07 и 08. Пока просто введите эту команду, если nano не найден.)

Открытие файла

Чтобы открыть файл для редактирования:

```
nano notes.txt
```

Если файл `notes.txt` существует — он откроется для редактирования. Если не существует — nano создаст новый пустой файл с этим именем (при сохранении).

Интерфейс nano

Когда вы откроете файл, экран будет выглядеть примерно так:

```
  GNU nano 8.3                    notes.txt

Здесь находится содержимое файла.
Вы можете просто печатать текст.


                          [ Новый файл ]
^G Помощь    ^O Записать  ^W Поиск     ^K Вырезать
^X Выход     ^R Вставить  ^\ Замена    ^U Вставить
```

Экран разделён на три части:

- **верхняя строка** — имя редактора и имя открытого файла;
- **основная область** — содержимое файла, здесь вы редактируете текст;
- **нижние две строки** — подсказки по основным командам.

Что означает символ ^

В подсказках внизу вы видите записи вроде `^X Выход` или `^O Записать`. Символ `^` означает клавишу **Ctrl**. То есть:

Запись	Что нажать
`^X`	`Ctrl+X`
`^O`	`Ctrl+O`
`^W`	`Ctrl+W`
`M-U`	`Alt+U`

Запись `M-` (Meta) означает клавишу **Alt**.

Клавиши	Действие

Навигация

nano — обычный текстовый редактор, похожий на Блокнот. Перемещаться по тексту можно привычными способами:

Клавиши	Действие
← → ↑ ↓	Перемещение по символам и строкам
`Home` / `End`	Начало / конец строки
`Page Up` / `Page Down`	Страница вверх / вниз
`Ctrl+_` (подчёркивание)	Перейти к строке по номеру
`Ctrl+C`	Показать текущую позицию (строка, столбец)

> **Совет.** Комбинация `Ctrl+_` полезна, когда вы видите ошибку вроде «ошибка в строке 42» — можно быстро перейти к нужной строке по номеру.

Редактирование текста

В nano вы просто печатаете текст — как в обычном редакторе. Никаких специальных режимов нет. Курсор стоит в нужном месте — вводите текст, он появляется на экране.

Клавиши	Действие
Любые символы	Ввод текста в позиции курсора
`Backspace`	Удалить символ перед курсором
`Delete`	Удалить символ под курсором
`Ctrl+K`	Вырезать (удалить) текущую строку целиком
`Ctrl+U`	Вставить последнюю вырезанную строку
`Alt+U`	Отменить последнее действие (undo)
`Alt+E`	Повторить отменённое действие (redo)

Комбинация `Ctrl+K` / `Ctrl+U` работает как «вырезать / вставить» для целых строк. Это удобно для перемещения строк: вырезаете строку в одном месте, перемещаете курсор в другое и вставляете.

Поиск и замена

Для поиска текста:

1. Нажмите `Ctrl+W` (Where is — «где находится»).
2. Введите искомый текст.
3. Нажмите `Enter` — курсор перейдёт к первому совпадению.
4. Чтобы найти следующее совпадение, снова нажмите `Ctrl+W` и `Enter`.

Для поиска и замены:

1. Нажмите `Ctrl+\`.
2. Введите текст, который хотите найти, и нажмите `Enter`.
3. Введите текст для замены и нажмите `Enter`.
4. nano спросит, заменять ли каждое совпадение: `Y` — да, `N` — нет, `A` — заменить все.

Сохранение и выход

Клавиши	Действие
`Ctrl+O`	Сохранить файл (nano спросит имя — нажмите `Enter` для подтверждения)
`Ctrl+X`	Выйти из nano

Если вы изменили файл и нажали `Ctrl+X`, nano спросит:

```
Сохранить изменённый буфер?  Y Да  N Нет  ^C Отмена
```

- `Y` — сохранить и выйти;
- `N` — выйти без сохранения;
- `Ctrl+C` — отменить выход и продолжить редактирование.

На практике самая частая последовательность — **`Ctrl+X`, затем `Y`, затем `Enter`** — сохранить файл и выйти.

Полезные приёмы

Номера строк

По умолчанию nano не показывает номера строк. Можно включить их при запуске:

```
nano -l notes.txt
```

Или прямо во время работы — нажмите `Alt+N` для переключения отображения номеров строк.

Открытие файла на определённой строке

Если вы знаете, что ошибка в строке 42:

```
nano +42 config.txt
```

nano откроет файл и сразу поставит курсор на строку 42.

Подсветка синтаксиса

nano умеет подсвечивать синтаксис для многих форматов: конфигурационных файлов, скриптов, HTML и других. В Debian подсветка обычно включена по умолчанию. Если нет — можно включить, добавив в файл `~/.nanorc` строку:

```
include "/usr/share/nano/*.nanorc"
```

(Как отредактировать этот файл? Конечно же, в nano: `nano ~/.nanorc`.)

Шпаргалка по nano

Комбинация	Действие
`Ctrl+O`	Сохранить
`Ctrl+X`	Выйти
`Ctrl+K`	Вырезать строку
`Ctrl+U`	Вставить строку
`Ctrl+W`	Найти текст
`Ctrl+\`	Найти и заменить
`Ctrl+_`	Перейти к строке
`Ctrl+C`	Показать позицию курсора
`Alt+U`	Отменить действие
`Alt+N`	Вкл/выкл номера строк

Практика: редактируем конфигурацию проекта

Применим знания nano на нашем сквозном проекте. Откроем файл конфигурации nginx и наполним его содержимым.

```
nano wordpress-project/config/nginx.conf
```

Введите следующий текст (просто печатайте — nano работает как обычный редактор):

```
server {
    listen 80;
    server_name localhost;

    root /var/www/wordpress;
    index index.php index.html;

    location / {
        try_files $uri $uri/ /index.php?$args;
    }

    location ~ \.php$ {
        fastcgi_pass unix:/run/php/php-fpm.sock;
        fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
        include fastcgi_params;
    }
}
```

Не беспокойтесь, если не понимаете значение каждой строки — мы подробно разберём этот конфигурационный файл в главе 11. Сейчас важно попрактиковаться в работе с редактором.

Сохраните файл: `Ctrl+X`, затем `Y`, затем `Enter`.

Проверим, что файл сохранился:

```
cat wordpress-project/config/nginx.conf
```

Теперь потренируемся в редактировании. Откройте файл снова:

```
nano wordpress-project/config/nginx.conf
```

Выполните действия:

1. Перейдите к строке 2 (`Ctrl+_`, введите `2`, `Enter`).
2. Замените `80` на `8080` — поставьте курсор на `8`, удалите `80` и напечатайте `8080`.
3. Найдите слово `localhost` (`Ctrl+W`, введите `localhost`, `Enter`).
4. Отмените последнее действие (`Alt+U`), убедитесь, что изменение откатилось.
5. Выйдите без сохранения (`Ctrl+X`, затем `N`).

vim — обзор

Зачем знать vim

vim (Vi Improved) — один из старейших и самых мощных текстовых редакторов в мире Unix/Linux. Он установлен практически на любой Linux-системе — даже на минимальных серверах, где нет nano.

Знать vim полезно по нескольким причинам:

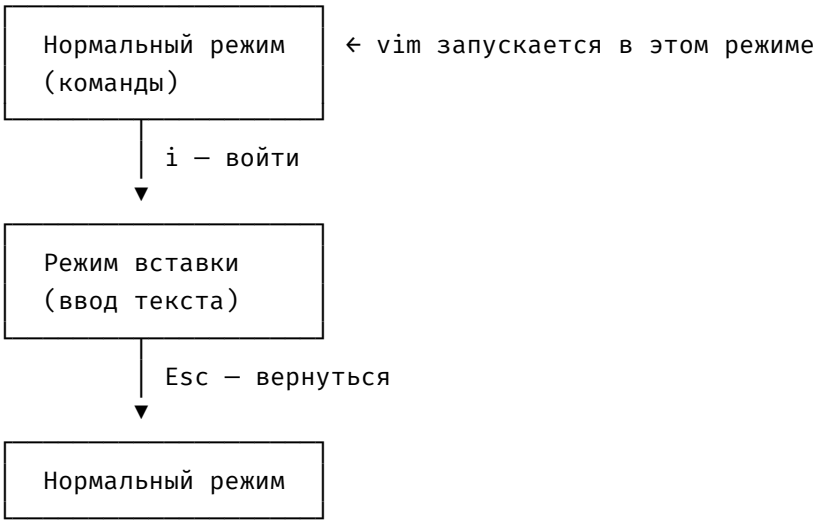
- **vim может оказаться единственным редактором** на сервере, к которому вы подключились;
- **некоторые программы открывают vim автоматически** — например, `git commit` (глава 13) или `crontab -e` по умолчанию запускают vim для редактирования;
- если vim открылся неожиданно и вы не знаете, как из него выйти, это может вызвать панику (это настолько частая ситуация, что стала мемом в IT).

Наша цель — не изучить vim в совершенстве, а научиться **открыть файл, внести правку, сохранить и выйти**. Этого достаточно для начала.

Два режима vim

Главное отличие vim от nano: **в vim нельзя просто начать печатать**. У vim есть несколько режимов работы, но для начала важны два:

- **нормальный режим** (Normal mode) — режим по умолчанию. Клавиши выполняют команды, а не вводят текст. Например, `dd` удаляет строку, `x` удаляет символ;
- **режим вставки** (Insert mode) — режим ввода текста, похожий на обычный редактор.



Правило: если что-то пошло не так — **нажмите `Esc`**. Вы всегда вернётесь в нормальный режим, откуда можно сохранить файл или выйти.

Минимальный набор действий

Открыть файл

```
vim notes.txt
```

vim откроется в **нормальном режиме**. Символ ~ (тильда) в начале строк означает, что эти строки не существуют — файл короче, чем экран.

Войти в режим вставки

Нажмите `i` (insert). Внизу экрана появится надпись `-- INSERT --` (или `-- ВСТАВКА --`). Теперь можно печатать текст как обычно.

Вернуться в нормальный режим

Нажмите `Esc`. Надпись `-- INSERT --` исчезнет.

Сохранить и выйти

Из **нормального режима** (нажмите `Esc`, если не уверены) введите команду. Команды в vim начинаются с двоеточия:

Команда	Действие
`:w`	Сохранить файл (write)
`:q`	Выйти (quit)
`:wq`	Сохранить и выйти
`:q!`	Выйти без сохранения (принудительно)

При вводе команды двоеточие и текст появляются в нижней строке экрана. После ввода команды нажмите `Enter`.

> **Не можете выйти из vim?** Нажмите `Esc`, затем введите `:q!` и нажмите `Enter`. Это закроет vim без сохранения — самый надёжный способ выбраться.

Шпаргалка по vim

Действие	Клавиша / команда
Войти в режим вставки	`i`
Вернуться в нормальный режим	`Esc`
Сохранить	`:w` + `Enter`
Выйти	`:q` + `Enter`
Сохранить и выйти	`:wq` + `Enter`
Выйти без сохранения	`:q!` + `Enter`
Удалить символ (нормальный режим)	`x`
Удалить строку (нормальный режим)	`dd`
Отмена действия	`u`

Как сменить редактор по умолчанию

Если программы (вроде `git` или `crontab`) открывают vim, а вам удобнее nano, можно изменить редактор по умолчанию:

```
export EDITOR=nano
```

Чтобы это сохранилось между сессиями, добавьте строку в файл `~/.bashrc`:

```
echo 'export EDITOR=nano' >> ~/.bashrc
```

> **Что здесь происходит?** Символ `>>` — это **перенаправление вывода** с добавлением в конец файла. Команда `echo` выводит текст, а `>>` записывает его в файл `~/.bashrc`, не затирая то, что там было. Подробно перенаправление ввода-вывода мы разберём в следующей главе (05).

О переменных окружения и файле `~/.bashrc` мы подробно поговорим в главе 06.

Практическое задание

1. Откройте в nano файл `~/wordpress-project/config/php.ini`. Введите следующие строки:

   ```
   upload_max_filesize = 64M
   post_max_size = 64M
   max_execution_time = 300
   memory_limit = 256M
   ```

   Сохраните файл и выйдите.

2. Откройте этот же файл снова. С помощью поиска (`Ctrl+W`) найдите слово `256M`. Замените его на `512M`. Сохраните и выйдите.

3. Откройте в nano новый файл `~/wordpress-project/site/index.html`. Введите:

   ```
   <!DOCTYPE html>
   <html>
   <head>
       <title>WordPress скоро будет здесь</title>
   </head>
   <body>
       <h1>Сайт в процессе настройки</h1>
   </body>
   </html>
   ```

   Сохраните файл. Проверьте содержимое командой `cat`.

4. Откройте любой файл в vim (`vim ~/wordpress-project/site/index.html`). Попрактикуйтесь:

   - войдите в режим вставки (`i`);
   - добавьте пустую строку;
   - вернитесь в нормальный режим (`Esc`);
   - выйдите без сохранения (`:q!`).

5. Снова откройте файл в vim. Войдите в режим вставки, добавьте комментарий `<!-- Настроено вручную -->` перед `</body>`. Сохраните и выйдите (`:wq`). Проверьте результат командой `cat`.
6. Переведите nano в режим отображения номеров строк. Откройте файл `nginx.conf` и перейдите к строке 5 с помощью `Ctrl+_`. На какой строке находится директива `index`?

Итоги

В этой главе вы:

- поняли, зачем нужен текстовый редактор в терминале;
- освоили **nano**: открытие, редактирование, поиск, замена, сохранение;
- познакомились с нотацией `^` (Ctrl) и `M-` (Alt) в подсказках nano;
- узнали, как показывать номера строк и открывать файл на нужной строке;
- познакомились с **vim**: два режима, вход в режим вставки, сохранение, выход;
- научились менять редактор по умолчанию через переменную `EDITOR`;
- попрактиковались на файлах сквозного проекта.

В следующей главе мы перейдём к мощным инструментам обработки текста — **coreutils и конвейеры**: grep, sort, awk, sed и другие команды, которые превращают командную строку в полноценный инструмент анализа данных.

Этап 5. Coreutils и конвейеры: обработка текста

Зачем это нужно

В предыдущих главах мы работали с файлами поштучно: создавали, копировали, редактировали в nano. Но в реальной работе системного администратора часто возникают задачи другого масштаба:

- в лог-файле nginx **тысячи строк** — как найти среди них ошибки?
- нужно быстро посчитать, **сколько раз** определённый IP-адрес обращался к серверу;
- требуется **извлечь** из конфигурационного файла только определённые параметры.

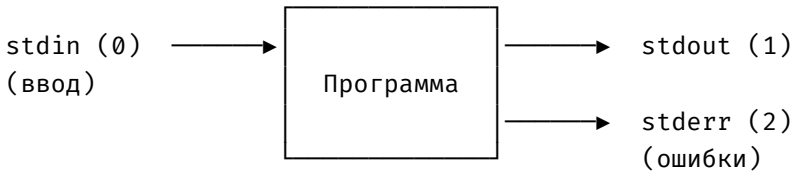
Для всего этого в Linux существует набор небольших утилит, каждая из которых делает **одну простую вещь**: подсчитать строки, отсортировать, отфильтровать, вырезать столбец. По отдельности они не впечатляют, но их сила — в возможности **соединять** друг с другом. Этот принцип называется **философией Unix**: каждая программа делает одно дело хорошо, а для сложных задач программы объединяются в цепочку.

Прежде чем перейти к самим утилитам, нам нужно разобраться, как работает этот механизм соединения — **перенаправление ввода-вывода и конвейеры**.

Врезка: перенаправление ввода-вывода

Три стандартных потока

Каждая программа в Linux при запуске получает три **потока данных**:



Поток	Номер	Описание	По умолчанию
stdin	0	Стандартный ввод — откуда программа читает данные	Клавиатура
stdout	1	Стандартный вывод — куда программа пишет результат	Экран (терминал)

Поток	Номер	Описание	По умолчанию
stderr	2	Стандартный вывод ошибок — куда программа пишет ошибки	Экран (терминал)

Когда вы вводите `ls -l`, программа `ls` читает содержимое каталога и выводит результат в **stdout** — вы видите его на экране. Если вы укажете несуществующий каталог (`ls /нет-такого`), сообщение об ошибке пойдёт в **stderr** — и тоже появится на экране, но через другой поток.

Перенаправление в файл

Символы > и >> позволяют направить вывод программы **в файл** вместо экрана.

```
# Записать список файлов в файл (создать или перезаписать)
ls -l > filelist.txt

# Добавить в конец файла (не затирая содержимое)
echo "новая строка" >> filelist.txt
```

Разница между > и >>:

Оператор	Действие
>	Создать файл или **перезаписать** его содержимое
>>	**Добавить** в конец файла

> **Осторожно!** Оператор > без предупреждения перезаписывает файл. Если вы случайно выполните `echo "тест" > important.conf`, всё содержимое файла будет заменено словом «тест». Будьте внимательны с > и >>.

Перенаправление ошибок

Чтобы перенаправить stderr (поток ошибок), используется `2>`:

```
# Ошибки — в файл, результат — на экран
ls /etc /нет-такого 2> errors.txt

# Ошибки — в никуда (подавить сообщения об ошибках)
ls /нет-такого 2> /dev/null
```

Файл `/dev/null` — это «чёрная дыра» в Linux. Всё, что в него записано, исчезает. Его используют, когда нужно подавить вывод ошибок.

Чтобы направить и stdout, и stderr **в один файл**:

```
command > output.txt 2>&1
```

Запись `2>&1` означает «перенаправить поток 2 (stderr) туда же, куда идёт поток 1 (stdout)».

Конвейер (pipe)

Конвейер — это самый важный механизм в этой главе. Символ | (вертикальная черта, «пайп») соединяет **stdout одной команды** с **stdin другой**:

```
┌──────────┐   stdout    stdin   ┌──────────┐
│ Команда 1 │──────────────|──────│ Команда 2 │
└──────────┘                     └──────────┘
```

Примеры:

```
# Посчитать количество файлов в каталоге
ls /etc | wc -l

# Показать первые 5 строк из отсортированного списка
ls /etc | sort | head -5
```

Конвейер можно наращивать — соединять три, четыре и более команд:

```
команда1 | команда2 | команда3 | команда4
```

Данные «текут» слева направо: вывод каждой команды становится вводом следующей. Далее в этой главе мы будем активно использовать конвейеры.

Подготовка: пример лог-файла nginx

Чтобы практиковаться на реальных данных, создадим файл с примером логов веб-сервера nginx. Это записи о запросах к нашему будущему сайту WordPress.

Создайте файл:

```
nano ~/wordpress-project/logs/access.log
```

Введите следующие строки (или скопируйте через терминал):

```
192.168.1.10 - - [15/Mar/2026:10:12:01 +0300] "GET / HTTP/1.1" 200 3421
192.168.1.10 - - [15/Mar/2026:10:12:02 +0300] "GET /style.css HTTP/1.1" 200
 ↪  1532
192.168.1.25 - - [15/Mar/2026:10:13:15 +0300] "GET /about HTTP/1.1" 200 2100
10.0.0.5 - - [15/Mar/2026:10:14:03 +0300] "POST /wp-login.php HTTP/1.1" 403 150
192.168.1.10 - - [15/Mar/2026:10:15:22 +0300] "GET /wp-admin/ HTTP/1.1" 302 0
192.168.1.25 - - [15/Mar/2026:10:15:30 +0300] "GET /contact HTTP/1.1" 404 274
10.0.0.5 - - [15/Mar/2026:10:16:45 +0300] "POST /wp-login.php HTTP/1.1" 403 150
10.0.0.5 - - [15/Mar/2026:10:17:01 +0300] "POST /wp-login.php HTTP/1.1" 403 150
192.168.1.30 - - [15/Mar/2026:10:18:10 +0300] "GET / HTTP/1.1" 200 3421
192.168.1.30 - - [15/Mar/2026:10:18:11 +0300] "GET /style.css HTTP/1.1" 200
 ↪  1532
192.168.1.25 - - [15/Mar/2026:10:19:05 +0300] "GET /blog HTTP/1.1" 200 4512
10.0.0.5 - - [15/Mar/2026:10:20:30 +0300] "POST /wp-login.php HTTP/1.1" 403 150
192.168.1.10 - - [15/Mar/2026:10:21:00 +0300] "GET /blog/first-post HTTP/1.1"
 ↪  200 3800
192.168.1.30 - - [15/Mar/2026:10:22:15 +0300] "GET /about HTTP/1.1" 200 2100
192.168.1.25 - - [15/Mar/2026:10:23:40 +0300] "GET
 ↪  /wp-content/uploads/photo.jpg HTTP/1.1" 200 85432
```

Сохраните файл (`Ctrl+X`, `Y`, `Enter`).

Каждая строка — это один HTTP-запрос к серверу. Разберём формат на примере первой строки:

```
192.168.1.10 - - [15/Mar/2026:10:12:01 +0300] "GET / HTTP/1.1" 200 3421
```

Где:

Элемент	Что означает
`192.168.1.10`	IP-адрес клиента
`-` (первый)	Имя удалённого пользователя (часто пусто)
`-` (второй)	Идентификатор пользователя (часто пусто)
`[15/Mar/... +0300]`	Дата и время запроса
`"GET / HTTP/1.1"`	Сам запрос (метод, URL, протокол)
`200`	Код ответа HTTP
`3421`	Размер ответа сервера (в байтах)

Код ответа HTTP говорит о результате запроса:

- **200** — OK, страница найдена;
- **302** — перенаправление на другой адрес;
- **403** — доступ запрещён;
- **404** — страница не найдена.

Теперь у нас есть данные для работы. Приступим к утилитам.

wc — подсчёт строк, слов и символов

Команда `wc` (word count) подсчитывает количество строк, слов и байт в файле:

```
wc ~/wordpress-project/logs/access.log
```

```
 15  210 1350 /home/student/wordpress-project/logs/access.log
```

Три числа: **15 строк**, **210 слов**, **1350 байт**.

Чаще всего нужно только количество строк — для этого есть флаг `-l`:

```
wc -l ~/wordpress-project/logs/access.log
```

```
15 /home/student/wordpress-project/logs/access.log
```

В нашем логе 15 записей — 15 запросов к серверу.

Другие полезные флаги:

Флаг	Что считает
`-l`	Строки
`-w`	Слова
`-c`	Байты
`-m`	Символы (для UTF-8 может отличаться от байтов)

wc отлично работает в конвейере — подсчитывает строки, приходящие на stdin:

```
# Сколько файлов в /etc?
ls /etc | wc -l
```

sort — сортировка строк

Команда `sort` сортирует строки в алфавитном порядке:

```
sort ~/wordpress-project/logs/access.log
```

На экране появятся все строки файла, отсортированные по первому символу (по IP-адресу, потому что он стоит в начале каждой строки). Строки с `10.0.0.5` окажутся вверху, строки с `192.168.1.*` — ниже. Сам файл при этом не изменится — sort, как и большинство утилит в этой главе, выводит результат на экран (в stdout), не трогая исходные данные.

Полезные флаги:

Флаг	Действие
`-r`	Обратный порядок (от Я к А, от большего к меньшему)
`-n`	Числовая сортировка (иначе `9` будет после `10`)
`-k N`	Сортировать по N-му полю (столбцу)
`-t 'X'`	Использовать символ X как разделитель полей
`-u`	Убрать дублирующиеся строки (как `sort | uniq`)

Пример — отсортировать лог по коду ответа HTTP. Код ответа — это 9-е поле (поля разделены пробелами):

```
sort -k 9 -n ~/wordpress-project/logs/access.log
```

Строки с кодом `200` окажутся в начале, с `403` и `404` — в конце.

uniq — удаление повторяющихся строк

Команда `uniq` удаляет **подряд идущие** одинаковые строки. Это важный нюанс: если одинаковые строки разделены другими строками, `uniq` их не заметит. Поэтому `uniq` почти всегда используется **после `sort`**.

```
# Извлечь IP-адреса, отсортировать и убрать дубликаты
cut -d ' ' -f 1 ~/wordpress-project/logs/access.log | sort | uniq
```

```
10.0.0.5
192.168.1.10
192.168.1.25
192.168.1.30
```

(Команду `cut` мы разберём чуть ниже — здесь она извлекает первое поле, т.е. IP-адрес.)

Самый полезный флаг `uniq` — это `-c` (count). Он показывает **количество повторений** каждой строки:

```
cut -d ' ' -f 1 ~/wordpress-project/logs/access.log | sort | uniq -c
```

```
      4 10.0.0.5
      4 192.168.1.10
      4 192.168.1.25
      3 192.168.1.30
```

Теперь видно, что IP-адрес `10.0.0.5` обращался к серверу 4 раза — и мы знаем из лога, что все 4 раза это были попытки входа в `wp-login.php` с ответом 403. Подозрительно!

Другие флаги:

Флаг	Действие
`-c`	Показать количество повторений
`-d`	Показать только повторяющиеся строки
`-u`	Показать только уникальные (неповторяющиеся) строки

cut — извлечение столбцов

Команда `cut` вырезает из каждой строки определённые поля (столбцы) или диапазоны символов.

По разделителю (самый частый вариант)

```
cut -d '<разделитель>' -f <номера_полей>
```

- `-d ' '` — разделитель — пробел;
- `-f 1` — взять первое поле;
- `-f 1,3` — первое и третье поле;
- `-f 1-3` — поля с первого по третье.

Примеры на нашем лог-файле:

```
# Извлечь только IP-адреса (первое поле, разделитель — пробел)
cut -d ' ' -f 1 ~/wordpress-project/logs/access.log
```

```
192.168.1.10
192.168.1.10
192.168.1.25
10.0.0.5
...
```

```
# Извлечь URL запроса (7-е поле)
cut -d ' ' -f 7 ~/wordpress-project/logs/access.log
```

```
/
/style.css
/about
/wp-login.php
...
```

По символам

```
# Извлечь первые 12 символов из каждой строки
cut -c 1-12 ~/wordpress-project/logs/access.log
```

```
192.168.1.10
192.168.1.10
192.168.1.25
10.0.0.5 - -
...
```

grep — поиск строк по шаблону

`grep` — одна из самых важных и часто используемых команд в Linux. Она фильтрует строки, содержащие заданный **шаблон** (образец):

```
grep '<шаблон>' <файл>
```

Простой поиск

```
# Найти все строки с кодом 404
grep '404' ~/wordpress-project/logs/access.log
```

```
192.168.1.25 - - [15/Mar/2026:10:15:30 +0300] "GET /contact HTTP/1.1" 404 274
```

Одна строка — значит, только одна страница не была найдена.

```
# Найти все обращения к wp-login.php
grep 'wp-login' ~/wordpress-project/logs/access.log
```

```
10.0.0.5 - - [15/Mar/2026:10:14:03 +0300] "POST /wp-login.php HTTP/1.1" 403 150
10.0.0.5 - - [15/Mar/2026:10:16:45 +0300] "POST /wp-login.php HTTP/1.1" 403 150
10.0.0.5 - - [15/Mar/2026:10:17:01 +0300] "POST /wp-login.php HTTP/1.1" 403 150
10.0.0.5 - - [15/Mar/2026:10:20:30 +0300] "POST /wp-login.php HTTP/1.1" 403 150
```

Четыре неудачных попытки входа с одного IP-адреса. В реальной жизни это мог бы быть сигнал о попытке подбора пароля.

Полезные флаги

Флаг	Действие
`-i`	Игнорировать регистр (`grep -i 'get'` найдёт и GET, и get)
`-c`	Показать только количество совпадений (не сами строки)
`-n`	Показать номера строк
`-v`	Инвертировать — показать строки, **не** содержащие шаблон
`-r`	Рекурсивный поиск по всем файлам в каталоге
`-l`	Показать только имена файлов с совпадениями

Примеры:

```
# Сколько запросов вернули ошибку 403?
grep -c '403' ~/wordpress-project/logs/access.log
```

```
4
```

```
# Показать все строки, КРОМЕ успешных (не содержащих 200)
grep -v '200' ~/wordpress-project/logs/access.log
```

```
# Найти слово «server» во всех файлах конфигурации
grep -r 'server' ~/wordpress-project/config/
```

grep в конвейере

grep часто используется как фильтр в середине конвейера:

```
# Сколько раз IP 10.0.0.5 обращался к серверу?
grep '10.0.0.5' ~/wordpress-project/logs/access.log | wc -l
```

```
4
```

```
# Какие URL запрашивал IP 192.168.1.25?
grep '192.168.1.25' ~/wordpress-project/logs/access.log | cut -d ' ' -f 7
```

```
/about
/contact
/blog
/wp-content/uploads/photo.jpg
```

Врезка: регулярные выражения (базовый уровень)

До сих пор мы искали в grep точные слова: `404`, `wp-login`. Но иногда нужен более гибкий поиск — например, «все строки, начинающиеся с 192.168» или «все файлы с расширением .php или .html». Для таких задач существуют **регулярные выражения** (regular expressions, regex).

Регулярное выражение — это **шаблон**, описывающий множество строк. Вместо того чтобы искать конкретное слово, вы описываете **паттерн** — правило, которому строка должна соответствовать.

Символ	Значение	Пример	Совпадёт с

Основные метасимволы

Символ	Значение	Пример	Совпадёт с
`.`	Любой один символ	`a.c`	`abc`, `a1c`, `a-c`
`*`	Предыдущий символ 0 или более раз	`ab*c`	`ac`, `abc`, `abbc`
`^`	Начало строки	`^192`	Строки, начинающиеся с `192`
`$`	Конец строки	`php$`	Строки, заканчивающиеся на `php`
`[]`	Любой символ из набора	`[0-9]`	Любая цифра
`[^]`	Любой символ, кроме указанных	`[^0-9]`	Любой символ, кроме цифры
`\`	Экранирование спецсимвола	`\.php`	Точка и `php` (буквально)

Важно: точка `.` в регулярных выражениях — это спецсимвол, означающий «любой символ». Если вам нужна именно точка (например, в имени файла `.php`), её нужно **экранировать** обратной косой чертой: `\.php`.

Что значит «экранировать»? Некоторые символы имеют специальное значение — точка означает «любой символ», `*` — «повторение» и т.д. Экранирование — это способ сказать: «воспринимай этот символ буквально, а не как спецсимвол». Для этого перед ним ставится обратная косая черта `\`. Запись `\.` означает «именно точка», а не «любой символ».

Примеры с grep

```
# Строки, начинающиеся с 192
grep '^192' ~/wordpress-project/logs/access.log


# Строки, содержащие .php (точка экранирована)
grep '\.php' ~/wordpress-project/logs/access.log


# Строки с кодом ответа, начинающимся на 4 (400, 403, 404...)
grep '" 4[0-9][0-9] ' ~/wordpress-project/logs/access.log
```

Расширенные регулярные выражения

Флаг `-E` (или команда `egrep`) включает **расширенный** синтаксис с дополнительными возможностями:

Символ	Значение	Пример	Совпадёт с
`+`	Предыдущий символ 1 или более раз	`ab+c`	`abc`, `abbc` (но не `ac`)
`?`	Предыдущий символ 0 или 1 раз	`colou?r`	`color`, `colour`
`\\|`	Или (альтернатива)	`cat\\|dog`	`cat` или `dog`
`()`	Группировка	`(ab)+`	`ab`, `abab`, `ababab`

```
# Найти строки с кодом 403 ИЛИ 404
grep -E '(403|404)' ~/wordpress-project/logs/access.log
```

Регулярные выражения — обширная тема. Здесь мы разобрали только основы, достаточные для повседневной работы с grep. Со временем вы будете осваивать более сложные шаблоны по мере необходимости.

---

sed — потоковая замена текста

Команда `sed` (stream editor) обрабатывает текст «на лету» — строку за строкой. Самая частая операция — **замена** (substitution):

```
sed 's/старое/новое/' файл
```

Буква `s` означает substitute (заменить). sed заменит **первое** вхождение шаблона в каждой строке. Чтобы заменить **все** вхождения в строке, добавьте флаг `g` (global):

```
sed 's/старое/новое/g' файл
```

Примеры

```
# Заменить HTTP/1.1 на HTTP/2 в выводе (файл не изменяется)
sed 's/HTTP\/1.1/HTTP\/2/' ~/wordpress-project/logs/access.log
```

> **Важно:** по умолчанию sed выводит результат на экран, **не изменяя** исходный файл. Чтобы изменить сам файл, используйте флаг `-i`:
>
> ```
> sed -i 's/старое/новое/g' файл
> ```
>
> Будьте осторожны с `-i` — отменить изменения будет невозможно.

```
# Удалить все строки, содержащие 403 (d — delete)
sed '/403/d' ~/wordpress-project/logs/access.log


# Показать только строки с 5-й по 10-ю (аналог head/tail, но гибче)
sed -n '5,10p' ~/wordpress-project/logs/access.log
```

Флаг `-n` подавляет обычный вывод, а команда `p` (print) выводит только указанные строки.

sed в конвейере

sed часто используется для преобразования данных между другими командами:

```
# Извлечь URL и убрать кавычки
cut -d '"' -f 2 ~/wordpress-project/logs/access.log | sed 's/ HTTP\/1.1//'


GET /
GET /style.css
GET /about
POST /wp-login.php
...
```

awk — обработка текста по столбцам

awk — это мини-язык программирования для обработки текстовых данных. Звучит сложно, но базовое использование простое. awk автоматически разбивает каждую строку на поля (столбцы) по пробелам и позволяет обращаться к ним по номерам:

- $1 — первое поле;
- $2 — второе поле;
- $NF — последнее поле;
- $0 — вся строка целиком.

Базовый синтаксис

```
awk '{print $N}' файл
```

Примеры

```
# Извлечь IP-адрес (1-е поле) и код ответа (9-е поле)
awk '{print $1, $9}' ~/wordpress-project/logs/access.log
```

```
192.168.1.10 200
192.168.1.10 200
192.168.1.25 200
10.0.0.5 403
192.168.1.10 302
192.168.1.25 404
...
```

Преимущество awk перед cut — awk **автоматически** разбивает строку по пробелам (и любому количеству пробелов), а cut требует указания разделителя и считает каждый пробел отдельно.

```
# Извлечь IP-адрес и размер ответа (последнее поле)
awk '{print $1, $NF}' ~/wordpress-project/logs/access.log
```

```
192.168.1.10 3421
192.168.1.10 1532
192.168.1.25 2100
10.0.0.5 150
...
```

Фильтрация в awk

awk может фильтровать строки по условию — аналогично grep, но с доступом к отдельным полям:

```
# Показать строки, где код ответа (поле 9) равен 404
awk '$9 == 404' ~/wordpress-project/logs/access.log
```

```
# Показать IP-адреса запросов с кодом ответа 403
awk '$9 == 403 {print $1}' ~/wordpress-project/logs/access.log
```

```
10.0.0.5
10.0.0.5
10.0.0.5
10.0.0.5
```

Указание разделителя

Если поля разделены не пробелами, а другим символом (например, двоеточием в файле `/etc/passwd`), используйте флаг `-F`:

```
# Показать имена пользователей из /etc/passwd (разделитель — двоеточие)
awk -F: '{print $1}' /etc/passwd
```

Комбинирование: решаем реальные задачи

Сила утилит — в комбинировании через конвейеры. Вот несколько типичных задач системного администратора, решённых цепочками команд.

Задача 1: найти 3 самых активных IP-адреса

```
awk '{print $1}' ~/wordpress-project/logs/access.log | sort | uniq -c | sort
↪  -rn | head -3
```

Разберём по шагам:

Шаг	Команда	Что делает
1	`awk '{print $1}'`	Извлекает IP-адреса
2	`sort`	Сортирует, чтобы одинаковые IP оказались рядом
3	`uniq -c`	Считает повторения
4	`sort -rn`	Сортирует по числу повторений (от большего к меньшему)
5	`head -3`	Берёт первые 3 строки

Результат:

```
  4 10.0.0.5
  4 192.168.1.10
  4 192.168.1.25
```

Задача 2: подсчитать количество запросов с ошибками

```
awk '$9 >= 400' ~/wordpress-project/logs/access.log | wc -l
```

```
5
```

Пять запросов с кодом ответа 400 и выше (ошибки клиента и сервера).

Задача 3: список уникальных URL, которые запрашивали

```
awk '{print $7}' ~/wordpress-project/logs/access.log | sort -u
```

```
/
/about
/blog
/blog/first-post
/contact
/style.css
/wp-admin/
/wp-content/uploads/photo.jpg
/wp-login.php
```

Флаг `sort -u` (unique) — сокращение для `sort | uniq`.

Задача 4: сохранить только ошибки в отдельный файл

```
grep -E '" (4[0-9]{2}|5[0-9]{2}) ' ~/wordpress-project/logs/access.log >
↪  ~/wordpress-project/logs/errors.log
```

Здесь используется регулярное выражение для поиска кодов 4xx и 5xx, а результат перенаправляется в новый файл.

Проверим:

```
cat ~/wordpress-project/logs/errors.log
```

Задача 5: общий объём переданных данных

```
awk '{sum += $NF} END {print sum, "байт"}' ~/wordpress-project/logs/access.log
```

Здесь `awk` суммирует значения последнего поля (размер ответа) по всем строкам. Блок `END` выполняется после обработки всех строк и выводит итог.

Практическое задание

Все задания выполняются на файле `~/wordpress-project/logs/access.log`.

1. Подсчитайте общее количество строк в лог-файле с помощью `wc`.
2. С помощью `grep` найдите все строки, содержащие `GET`. Сколько их? (Подсказка: `grep -c`.)
3. Извлеките из лога только IP-адреса (первое поле) с помощью `cut` или `awk`. Выведите список уникальных IP-адресов.
4. Найдите все запросы, которые вернули код `200`. Подсчитайте их количество.
5. С помощью `grep` и регулярного выражения найдите все строки, где URL заканчивается на `.php`.
6. Определите, какой IP-адрес сделал больше всего запросов. Используйте цепочку: `awk | sort | uniq -c | sort -rn | head -1`.

7. Используя `sed`, выведите содержимое лог-файла, заменив все вхождения `HTTP/1.1` на `HTTP/2`. (Не изменяйте файл — просто выведите на экран.)
8. С помощью `awk` выведите только те строки, где размер ответа (последнее поле) больше 3000 байт.
9. Перенаправьте все строки с кодом 403 в файл `~/wordpress-project/logs/blocked.log`. Проверьте результат командой `cat`.
10. (*) Определите самую часто запрашиваемую страницу (URL). Подсказка: извлеките URL (7-е поле), отсортируйте, подсчитайте и покажите первую строку.

Итоги

В этой главе вы:

- разобрались с тремя стандартными потоками: stdin, stdout, stderr;
- научились перенаправлять вывод в файл (`>`, `>>`) и ошибки (`2>`);
- освоили конвейер `|` — главный инструмент комбинирования команд;
- познакомились с утилитами обработки текста:
  - `wc` — подсчёт строк, слов, байт;
  - `sort` — сортировка;
  - `uniq` — удаление дубликатов и подсчёт повторений;
  - `cut` — извлечение столбцов;
  - `grep` — фильтрация строк по шаблону;
  - `sed` — потоковая замена текста;
  - `awk` — обработка текста по столбцам;
- узнали основы регулярных выражений: `.`, `*`, `^`, `$`, `[]`, `\`;
- научились решать реальные задачи цепочками команд на примере логов nginx.

В следующей главе мы перейдём к **основам bash-скриптов** — научимся объединять команды в файлы-программы, которые можно запускать повторно, и познакомимся с переменными окружения и PATH.

Этап 6. Основы bash-скриптов

Зачем нужны скрипты

В предыдущих главах вы научились работать с файлами, правами, текстовыми редакторами и утилитами обработки текста. Каждый раз вы вводили команды вручную — одну за другой. Но что, если одну и ту же последовательность команд нужно выполнять регулярно?

Представьте: каждый вечер вы должны проверить лог-файл nginx, подсчитать количество ошибок и сохранить отчёт. Это три-четыре команды, которые нужно вводить каждый день. Вместо этого можно записать их в файл и запускать одной командой. Такой файл называется **скриптом** (script).

Скрипт — это текстовый файл с последовательностью команд, которые выполняются автоматически, одна за другой.

> **Привязка к проекту.** К концу этой главы мы напишем скрипт, который создаёт резервную копию нашего проекта WordPress — проверяет, что нужный каталог существует, архивирует его и сообщает об успехе или ошибке.

Прежде чем писать скрипты, нам нужно разобраться с важной темой — как система находит программы и что такое переменные окружения.

Врезка: переменные окружения и PATH

Что такое переменные окружения

Когда вы входите в систему, оболочка bash создаёт набор **переменных окружения** (environment variables) — именованных значений, которые доступны всем запускаемым программам. Они хранят настройки системы и пользователя.

Посмотреть все переменные окружения можно командой `env`:

```
env
```

Вывод будет длинным. Вот несколько важных переменных:

Переменная	Описание	Пример значения
`HOME`	Домашний каталог пользователя	`/home/student`
`USER`	Имя текущего пользователя	`student`
`SHELL`	Путь к оболочке пользователя	`/bin/bash`
`PWD`	Текущий рабочий каталог	`/home/student`
`LANG`	Язык и локаль	`ru_RU.UTF-8`
`PATH`	Где искать программы	`/usr/local/bin:/usr/bin:/bin`

Переменная	Описание	Пример значения
`EDITOR`	Текстовый редактор по умолчанию	`nano`

Чтобы посмотреть значение конкретной переменной, используйте `echo` и знак $ перед именем:

```
echo $HOME
```

```
/home/student
```

```
echo $USER
```

```
student
```

Знак $ говорит bash: «подставь значение переменной». Без $ bash воспримет слово `HOME` как обычный текст:

```
echo HOME
```

```
HOME
```

Создание и экспорт переменных

Вы можете создать свою переменную:

```
GREETING="Привет, мир"
echo $GREETING
```

```
Привет, мир
```

> **Обратите внимание:** вокруг знака `=` **не должно быть пробелов**. Запись `GREETING = "Привет"` вызовет ошибку — bash решит, что `GREETING` — это команда, а `=` и `"Привет"` — её аргументы.

Такая переменная существует только в текущей оболочке. Если вы запустите из неё другую программу (дочерний процесс), она эту переменную не увидит. Чтобы переменная стала доступна дочерним процессам, её нужно **экспортировать**:

```
export GREETING="Привет, мир"
```

Именно поэтому в главе 04, когда мы задавали редактор по умолчанию, мы писали `export EDITOR=nano` — чтобы все программы (git, crontab и т.д.) видели эту настройку.

PATH — как система находит команды

Когда вы вводите команду, например `ls`, bash должен найти файл программы `ls` на диске. Но как он знает, где искать?

Переменная **PATH** содержит список каталогов, разделённых двоеточием. Bash просматривает их по порядку, пока не найдёт файл с нужным именем:

```
echo $PATH
```

```
/usr/local/bin:/usr/bin:/bin:/usr/local/sbin:/usr/sbin:/sbin
```

Это значит: сначала искать в `/usr/local/bin`, потом в `/usr/bin`, потом в `/bin` и так далее.

Команда `which` показывает, где именно bash нашёл программу:

```
which ls
```

```
/usr/bin/ls
```

```
which nano
```

```
/usr/bin/nano
```

Если программа не находится ни в одном каталоге из PATH, bash выдаст ошибку «command not found». Именно поэтому для запуска скрипта из текущего каталога нужно писать `./script.sh` — текущий каталог `.` обычно не входит в PATH.

Файл ~/.bashrc

Переменные окружения, заданные в терминале, исчезают при выходе из системы. Чтобы настройки сохранялись, их записывают в файл **`~/.bashrc`** — скрипт, который bash автоматически выполняет при каждом входе в систему.

В главе 04 мы уже добавляли туда строку `export EDITOR=nano`. Посмотрим, что ещё есть в этом файле:

```
cat ~/.bashrc
```

Вы увидите настройки приглашения командной строки (PS1), алиасы и другие параметры. Если вы добавили что-то в `~/.bashrc`, но изменения не применились, выполните:

```
source ~/.bashrc
```

Команда `source` выполняет файл в текущей оболочке, применяя все настройки без необходимости выходить и входить заново.

Первый скрипт

Создание

Создадим простой скрипт в nano:

```
nano ~/wordpress-project/hello.sh
```

Введите:

```
#!/bin/bash

echo "Привет! Сейчас:"
date
echo "Вы находитесь в каталоге:"
pwd
```

Сохраните и выйдите (`Ctrl+X`, `Y`, `Enter`).

Shebang

Первая строка `#!/bin/bash` называется **shebang** (или «шебанг»). Она сообщает системе, какой интерпретатор использовать для выполнения скрипта.

```
#!/bin/bash
│└
│└─ путь к интерпретатору
└── специальная последовательность символов (#!)
```

Без shebang система не будет знать, что этот файл нужно выполнять через bash. Всегда начинайте скрипты с этой строки.

Запуск

Попробуем запустить:

```
./wordpress-project/hello.sh
```

```
bash: ./wordpress-project/hello.sh: Отказано в доступе
```

Мы уже сталкивались с этим в главе 03 — у файла нет права на выполнение. Добавим:

```
chmod u+x ~/wordpress-project/hello.sh
```

Запустим ещё раз:

```
./wordpress-project/hello.sh
```

```
Привет! Сейчас:
Сб мар 15 12:00:00 MSK 2026
Вы находитесь в каталоге:
/home/student
```

Почему перед именем файла `./`? Как мы выяснили в разделе про PATH, текущий каталог не входит в список каталогов для поиска программ. Префикс `./` явно указывает: «запусти файл из текущего каталога». Без него bash будет искать программу `hello.sh` в каталогах PATH и не найдёт.

Альтернативный способ запуска

Скрипт можно запустить и без права на выполнение, явно вызвав bash:

```
bash ~/wordpress-project/hello.sh
```

В этом случае `chmod +x` не нужен — bash сам читает файл и выполняет команды. Но в повседневной работе удобнее делать скрипт исполняемым и запускать через `./`.

Переменные в скриптах

Объявление и использование

Переменные в скриптах работают так же, как в командной строке:

```
#!/bin/bash

PROJECT_DIR="$HOME/wordpress-project"
BACKUP_DIR="$HOME/backups"

echo "Проект: $PROJECT_DIR"
echo "Резервные копии: $BACKUP_DIR"
```

Имена переменных принято писать **ЗАГЛАВНЫМИ_БУКВАМИ** — это не обязательно, но помогает отличать переменные от команд.

Фигурные скобки

Иногда bash не может понять, где заканчивается имя переменной. Для этого используются фигурные скобки `${...}`:

```
FILE="access"
echo "$FILE_log"      # Ошибка! bash ищет переменную FILE_log
echo "${FILE}_log"    # Правильно: access_log
```

Кавычки

В bash есть два вида кавычек, и они работают **по-разному**:

Кавычки	Поведение	Пример	Результат
Двойные `"..."`	Раскрывают переменные	`"Привет, $USER"`	`Привет, student`
Одинарные `'...'`	Всё буквально	`'Привет, $USER'`	`Привет, $USER`

```
#!/bin/bash

NAME="мир"
```

```
echo "Привет, $NAME"    # Привет, мир
echo 'Привет, $NAME'    # Привет, $NAME
```

Правило простое: используйте **двойные кавычки**, когда нужно подставить значение переменной, и **одинарные**, когда нужен текст как есть.

Подстановка команд

Результат выполнения команды можно сохранить в переменную с помощью `$(...)`:

```
#!/bin/bash

TODAY=$(date +%Y-%m-%d)
FILE_COUNT=$(ls ~/wordpress-project | wc -l)

echo "Сегодня: $TODAY"
echo "Файлов в проекте: $FILE_COUNT"
```

```
Сегодня: 2026-03-15
Файлов в проекте: 5
```

Конструкция `$(команда)` выполняет команду и подставляет её вывод вместо себя.

Аргументы скрипта

Скрипту можно передавать аргументы при запуске — как опции обычным командам:

```
./script.sh аргумент1 аргумент2 аргумент3
```

Внутри скрипта аргументы доступны через специальные переменные:

Переменная	Значение
`$0`	Имя самого скрипта
`$1`	Первый аргумент
`$2`	Второй аргумент
`$#`	Количество аргументов
`$@`	Все аргументы

Пример — скрипт, который приветствует пользователя по имени:

```
#!/bin/bash

echo "Скрипт: $0"
echo "Привет, $1!"
echo "Передано аргументов: $#"
```

```
chmod u+x greet.sh
./greet.sh Евгений
```

```
Скрипт: ./greet.sh
Привет, Евгений!
Передано аргументов: 1
```

Условия (if)

Базовый синтаксис

```
if [ условие ]; then
    # команды, если условие истинно
fi
```

> **Важно:** пробелы внутри квадратных скобок **обязательны**. `[ -f file ]` — правильно, `[-f file]` — ошибка.

Примеры условий

Условие	Что проверяет
`[ -f файл ]`	Файл существует и является обычным файлом
`[ -d каталог ]`	Каталог существует
`[ -e путь ]`	Путь существует (файл или каталог)
`[ -z "$строка" ]`	Строка пуста
`[ -n "$строка" ]`	Строка не пуста
`[ "$a" = "$b" ]`	Строки равны
`[ "$a" != "$b" ]`	Строки не равны
`[ "$a" -eq "$b" ]`	Числа равны (equal)
`[ "$a" -gt "$b" ]`	a больше b (greater than)
`[ "$a" -lt "$b" ]`	a меньше b (less than)

if / else / elif

```
#!/bin/bash

FILE="$HOME/wordpress-project/config/nginx.conf"

if [ -f "$FILE" ]; then
    echo "Файл $FILE найден"
else
    echo "Файл $FILE не существует!"
fi
```

Для нескольких условий используется `elif` (else if):

```
#!/bin/bash

if [ $# -eq 0 ]; then
    echo "Ошибка: укажите имя файла"
elif [ -f "$1" ]; then
    echo "Файл $1 существует, размер:"
    wc -c "$1"
else
    echo "Файл $1 не найден"
fi
```

Проверка результата команды

`if` может проверять результат выполнения команды. Если команда завершилась успешно (код возврата 0), условие считается истинным:

```
#!/bin/bash

if grep -q '404' ~/wordpress-project/logs/access.log; then
    echo "Внимание: в логе есть ошибки 404!"
else
    echo "Ошибок 404 не найдено"
fi
```

Флаг `-q` (quiet) заставляет grep работать молча — не выводить совпадения, а только вернуть код успеха или неуспеха.

Циклы

for — перебор списка

Цикл `for` выполняет команды для каждого элемента списка:

```
for переменная in список; do
    # команды
done
```

Примеры:

```
#!/bin/bash

# Перебор явного списка
for CITY in Москва Петербург Новосибирск; do
    echo "Город: $CITY"
done
```

```
Город: Москва
Город: Петербург
Город: Новосибирск
```

```
#!/bin/bash

# Перебор файлов в каталоге
for FILE in ~/wordpress-project/config/*; do
    echo "Конфигурационный файл: $FILE"
done
```

```
#!/bin/bash

# Перебор аргументов скрипта
for ARG in "$@"; do
    echo "Аргумент: $ARG"
done
```

while — цикл с условием

Цикл `while` выполняется, пока условие истинно:

```
while [ условие ]; do
    # команды
done
```

Пример — чтение файла строка за строкой:

```
#!/bin/bash

while read LINE; do
    echo "Строка: $LINE"
done < ~/wordpress-project/logs/access.log
```

Конструкция `< файл` в конце цикла — это перенаправление ввода (из главы 05): содержимое файла подаётся на stdin команды `read`, которая считывает по одной строке.

Пример — обратный отсчёт:

```
#!/bin/bash

COUNT=5
while [ $COUNT -gt 0 ]; do
    echo "$COUNT..."
    COUNT=$((COUNT - 1))
done
echo "Поехали!"
```

```
5...
4...
3...
2...
1...
Поехали!
```

Запись `$(( выражение ))` — это арифметическое вычисление в bash.

Коды возврата

Что это такое

Каждая команда в Linux при завершении возвращает числовой **код возврата** (exit code, exit status):

- **0** — команда завершилась **успешно**;
- **не 0** (1, 2, 127 и т.д.) — произошла **ошибка**.

Код возврата последней команды хранится в специальной переменной `$?`:

```
ls /etc/hostname
echo $?
```

```
/etc/hostname
0
```

```
ls /несуществующий-файл
echo $?
```

```
ls: невозможно получить доступ к '/несуществующий-файл': Нет такого файла или
 ↪  каталога
2
```

exit в скриптах

Команда `exit` завершает скрипт с указанным кодом возврата:

```
#!/bin/bash

if [ $# -eq 0 ]; then
    echo "Ошибка: не указано имя файла" >&2
    exit 1
fi

echo "Обработка файла $1..."
exit 0
```

Запись `>&2` перенаправляет сообщение об ошибке в stderr (поток ошибок), как мы узнали в главе 05. Это хорошая практика — сообщения об ошибках отправлять в stderr, а обычный вывод — в stdout.

> **Соглашение:** `exit 0` — успех, `exit 1` — общая ошибка. Можно использовать и другие числа для разных типов ошибок, но для простых скриптов достаточно 0 и 1.

Практика: скрипт резервного копирования

Соберём все знания воедино и напишем полезный скрипт для нашего проекта. Он будет создавать резервную копию каталога wordpress-project.

```
nano ~/wordpress-project/backup.sh
```

Введите:

```
#!/bin/bash

# Скрипт резервного копирования проекта WordPress
# Использование: ./backup.sh [каталог_для_бэкапов]

# Настройки
PROJECT_DIR="$HOME/wordpress-project"
BACKUP_DIR="${1:-$HOME/backups}"
DATE=$(date +%Y-%m-%d_%H-%M-%S)
ARCHIVE_NAME="wordpress-backup-${DATE}.tar.gz"

# Проверяем, что каталог проекта существует
```

```
if [ ! -d "$PROJECT_DIR" ]; then
    echo "Ошибка: каталог проекта $PROJECT_DIR не найден" >&2
    exit 1
fi

# Создаём каталог для бэкапов, если его нет
if [ ! -d "$BACKUP_DIR" ]; then
    echo "Создаю каталог для бэкапов: $BACKUP_DIR"
    mkdir -p "$BACKUP_DIR"
fi

# Создаём архив
echo "Создаю резервную копию..."
tar czf "$BACKUP_DIR/$ARCHIVE_NAME" -C "$HOME" wordpress-project

# Проверяем результат
if [ $? -eq 0 ]; then
    echo "Готово! Резервная копия сохранена:"
    echo "  $BACKUP_DIR/$ARCHIVE_NAME"
    echo "  Размер: $(du -h "$BACKUP_DIR/$ARCHIVE_NAME" | cut -f1)"
else
    echo "Ошибка при создании резервной копии!" >&2
    exit 1
fi
```

Сохраните файл и сделайте его исполняемым:

```
chmod u+x ~/wordpress-project/backup.sh
```

Запустите:

```
~/wordpress-project/backup.sh
```

```
Создаю каталог для бэкапов: /home/student/backups
Создаю резервную копию...
Готово! Резервная копия сохранена:
  /home/student/backups/wordpress-backup-2026-03-15_12-00-00.tar.gz
  Размер: 4.0K
```

Разберём несколько моментов в этом скрипте:

- **${1:-$HOME/backups}** — если первый аргумент не передан, используется значение по умолчанию `$HOME/backups`. Конструкция `${переменная:-значение}` означает «подставь значение переменной, а если она пуста — подставь значение после `:-`».
- **tar czf** — создаёт сжатый архив (tar + gzip). Подробнее о tar вы узнаете в дальнейшей работе, пока достаточно знать, что эта команда упаковывает каталог в один файл.
- **[! -d ...]** — восклицательный знак означает отрицание: «если каталог НЕ существует».
- **du -h** — показывает размер файла в читаемом формате (K, M, G).

Можно указать другой каталог для бэкапов:

```
~/wordpress-project/backup.sh /tmp/my-backups
```

Практическое задание

1. Напишите скрипт `sysinfo.sh`, который выводит:
   - имя пользователя (`$USER`);
   - имя хоста (`hostname`);
   - текущую дату и время (`date`);
   - текущий каталог (`pwd`). Сделайте его исполняемым и запустите.
2. Напишите скрипт `check-file.sh`, который принимает имя файла как аргумент и сообщает, существует он или нет. Если аргумент не передан — выведите подсказку по использованию и завершите скрипт с кодом 1.
3. Напишите скрипт `count-errors.sh`, который подсчитывает количество строк с кодом 403 и с кодом 404 в файле `~/wordpress-project/logs/access.log` и выводит результат:

   ```
   Ошибок 403: 4
   Ошибок 404: 1
   ```

   Подсказка: используйте `grep -c` и подстановку команд `$(...)`.
4. Напишите скрипт `list-configs.sh`, который с помощью цикла `for` перебирает все файлы в `~/wordpress-project/config/` и для каждого выводит его имя и размер.
5. Напишите скрипт `greet.sh`, который принимает имя как аргумент. Если имя не передано, скрипт должен спросить: «Как вас зовут?» и прочитать ответ командой `read`. Подсказка:

   ```
   read -p "Как вас зовут? " NAME
   ```

6. Измените скрипт `backup.sh` так, чтобы он сохранял не более трёх последних бэкапов, удаляя самые старые. Подсказка: используйте `ls -t` (сортировка по времени) и `tail -n +4` (строки начиная с четвёртой).

Итоги

В этой главе вы:

- узнали, зачем нужны скрипты и как они упрощают работу;
- разобрались с переменными окружения и их ролью в системе;
- поняли, как PATH определяет, где bash ищет команды;
- познакомились с файлом `~/.bashrc` и командой `source`;
- написали первый скрипт: shebang, `chmod +x`, запуск через `./`;
- научились работать с переменными, кавычками и подстановкой команд;
- освоили аргументы скрипта (`$1`, `$2`, `$#`, `$@`);
- научились использовать условия (`if`, `else`, `elif`) и циклы (`for`, `while`);
- разобрались с кодами возврата и командой `exit`;
- написали реальный скрипт резервного копирования для проекта.

Поздравляем! Вы завершили первую часть курса — **«Пользователь Linux»**. Теперь вы умеете ориентироваться в системе, работать с файлами и правами, редактировать конфигурации, обрабатывать текст и автоматизировать рутинные задачи скриптами.

Во второй части — **«Системный администратор»** — мы перейдём к управлению системой: установке программ, управлению пользователями, работе с сетью и в итоге развернём наш сайт на nginx + PHP-FPM + WordPress.

Следующая глава — **управление пакетами (apt)**: как устанавливать, обновлять и удалять программы в Debian.

Этап 7. Управление пакетами (apt) и репозитории

Что такое пакеты

В Windows, чтобы установить программу, вы обычно заходите на сайт разработчика, скачиваете `.exe`-файл и запускаете установщик. В Linux всё устроено иначе.

Пакет — это архив, в котором лежат файлы программы, её зависимости и инструкции по установке. Пакеты хранятся в **репозиториях** — централизованных хранилищах, похожих на магазин приложений. Чтобы установить программу, вы не ищете её по интернету, а запрашиваете из репозитория одной командой.

Для управления пакетами в Debian используется система **APT** (Advanced Package Tool). Это набор утилит, которые:

- скачивают пакеты из репозиториев;
- автоматически устанавливают **зависимости** — другие пакеты, необходимые для работы программы;
- обновляют установленные программы;
- удаляют ненужное.

> **Привязка к проекту.** В следующих главах мы будем устанавливать nginx, PHP-FPM и MariaDB (базу данных для WordPress) — всё через apt. Умение работать с пакетами — ключевой навык администратора.

Что такое зависимости

Программы редко работают сами по себе. Например, для PHP-FPM нужны библиотеки обработки XML, работы с базами данных, шифрования и другие. Все они оформлены как отдельные пакеты.

Когда вы устанавливаете программу через apt, менеджер пакетов автоматически определяет, какие зависимости нужны, скачивает и устанавливает их. Вам не нужно разбираться в этом вручную.

Репозитории

Что это такое

Репозиторий — это сервер (или набор серверов), где хранятся пакеты. Debian поддерживает десятки тысяч пакетов, и все они доступны из официальных репозиториев.

Список репозиториев, которые использует ваша система, хранится в файле `/etc/apt/sources.list` и в каталоге `/etc/apt/sources.list.d/`.

Посмотрим содержимое:

```
cat /etc/apt/sources.list
```

Вы увидите строки примерно такого вида:

```
deb http://deb.debian.org/debian trixie main contrib non-free-firmware
deb http://deb.debian.org/debian trixie-updates main contrib non-free-firmware
deb http://security.debian.org/debian-security trixie-security main contrib
 ↪  non-free-firmware
```

Разберём структуру строки:

```
deb http://deb.debian.org/debian trixie main contrib non-free-firmware
```

Где:

Элемент	Что означает
`deb`	Тип (бинарные пакеты, не исходники)
`http://deb.../debian`	URL-адрес сервера репозитория
`trixie`	Кодовое имя версии ОС (Debian 13)
`main`	Секция: свободное ПО (основная)
`contrib`	Секция: зависит от несвободного ПО
`non-free-firmware`	Секция: проприетарные драйверы и прошивки

Секции репозитория

Секция	Описание
main	Полностью свободное ПО, поддерживается командой Debian
contrib	Свободное ПО, но зависит от несвободных компонентов
non-free	Несвободное ПО (проприетарные драйверы и т.п.)
non-free-firmware	Прошивки устройств (Wi-Fi, видеокарты и др.)

Для большинства задач достаточно секции **main**. Всё, что мы будем устанавливать в этом курсе (nginx, PHP, MariaDB), находится в main.

Обновление списка пакетов

После добавления или изменения репозиториев (и при первом входе на свежую систему) нужно обновить локальный список доступных пакетов:

```
sudo apt update
```

Эта команда не устанавливает и не обновляет программы — она лишь загружает свежий каталог пакетов с серверов репозиториев. Считайте это обновлением оглавления в магазине приложений.

> **Что такое `sudo`?** Команда `sudo` позволяет выполнить действие от имени суперпользователя (root). Управление пакетами — системная операция, и для неё нужны права администратора. Подробно `sudo` и управление пользователями мы разберём в следующей главе (08). Пока просто добавляйте `sudo` перед командами `apt`.

Поиск и информация о пакетах

apt search — поиск по имени и описанию

```
apt search nginx
```

Команда выведет список пакетов, в имени или описании которых встречается слово «nginx». Список может быть длинным — используйте конвейер с `less` для постраничного просмотра:

```
apt search nginx | less
```

Можно уточнить поиск, комбинируя с grep:

```
apt search nginx | grep ^nginx
```

Символ ^ (начало строки) из регулярных выражений (глава 05) отфильтрует только строки, **начинающиеся** с «nginx» — то есть имена пакетов, а не описания.

apt show — подробная информация

```
apt show nginx
```

```
Package: nginx
Version: 1.26.0-2
Priority: optional
Section: httpd
Maintainer: Debian Nginx Maintainers
 ↪  <pkg-nginx-maintainers@alioth-lists.debian.net>
Installed-Size: 236 kB
Depends: nginx-core (<< 1.26.0-2.1~) | nginx-full (<< 1.26.0-2.1~) | ...
...
Description: small, powerful, scalable web/proxy server
```

Здесь видно версию, размер, зависимости и описание пакета.

dpkg -l — список установленных пакетов

```
dpkg -l
```

Эта команда покажет все установленные пакеты. Их обычно сотни, поэтому используйте grep для фильтрации:

```
dpkg -l | grep nano
```

```
ii  nano  8.3-1  amd64  small, friendly text editor inspired by Pstretch
```

Статус `ii` в начале означает, что пакет **установлен** и **настроен**.

Проверка, установлен ли пакет

Быстрый способ проверить, установлен ли конкретный пакет:

```
dpkg -l nginx 2>/dev/null | grep ^ii
```

Если вывод пустой — пакет не установлен. Если строка с `ii` появилась — установлен.

Установка пакетов

apt install

```
sudo apt install <имя_пакета>
```

apt покажет, какие пакеты будут установлены (включая зависимости), сколько места потребуется, и попросит подтверждение:

```
sudo apt install tree
```

```
Чтение списков пакетов… Готово
Построение дерева зависимостей… Готово
Чтение сведений о состоянии… Готово
Будут установлены следующие НОВЫЕ пакеты:
  tree
0 обновлено, 1 установлено новых, 0 для удаления и 12 не обновлено.
Необходимо скачать 49,2 kB архивов.
После данной операции объём занятого дискового пространства возрастёт на 116
 ↪  kB.
Хотите продолжить? [Д/н]
```

Нажмите `Y` (или `Д`) и `Enter` для подтверждения. Если хотите пропустить вопрос и сразу согласиться, используйте флаг `-y`:

```
sudo apt install -y tree
```

Установка нескольких пакетов

```
sudo apt install nginx php-fpm mariadb-server
```

Все три пакета (и их зависимости) будут установлены за одну операцию.

Обновление пакетов

Обновление списка + обновление пакетов

Типичная последовательность для обновления системы:

```
# Шаг 1: обновить каталог пакетов
sudo apt update

# Шаг 2: обновить все установленные пакеты
sudo apt upgrade
```

`apt upgrade` обновляет все пакеты, для которых доступна новая версия. Перед обновлением apt покажет список изменений и попросит подтверждение.

apt upgrade vs apt full-upgrade

Команда	Поведение
`apt upgrade`	Обновляет пакеты, **не удаляя** и не добавляя новых
`apt full-upgrade`	Обновляет пакеты, **может удалить** ненужные или добавить новые зависимости

Для обычного обновления используйте `apt upgrade`. Команда `apt full-upgrade` нужна при переходе между версиями дистрибутива (например, с Debian 12 на Debian 13) — в повседневной работе она требуется редко.

Удаление пакетов

apt remove — удаление программы

```
sudo apt remove tree
```

Пакет будет удалён, но его **конфигурационные файлы** останутся на диске. Это удобно, если вы планируете установить программу заново — настройки сохранятся.

apt purge — полное удаление

```
sudo apt purge tree
```

Удаляет пакет **вместе с конфигурационными файлами**. Используйте, если уверены, что программа больше не нужна.

apt autoremove — удаление ненужных зависимостей

Когда вы устанавливаете программу, apt подтягивает зависимости. При удалении программы зависимости могут остаться. Команда `autoremove` удаляет «осиротевшие» пакеты, которые больше никому не нужны:

```
sudo apt autoremove
```

> **Совет.** Периодически запускайте `sudo apt autoremove`, чтобы освободить место на диске от ненужных зависимостей.

Сводная таблица команд apt

Команда	Действие
`apt update`	Обновить список пакетов из репозиториев
`apt upgrade`	Обновить все установленные пакеты
`apt search <запрос>`	Найти пакет по имени или описанию
`apt show <пакет>`	Показать информацию о пакете
`apt install <пакет>`	Установить пакет
`apt remove <пакет>`	Удалить пакет (конфиги остаются)
`apt purge <пакет>`	Удалить пакет вместе с конфигами
`apt autoremove`	Удалить ненужные зависимости
`dpkg -l`	Список всех установленных пакетов

Практика: утилита tree

Установим полезную утилиту `tree`, которая показывает структуру каталогов в виде дерева.

```
sudo apt install -y tree
```

Теперь посмотрим структуру нашего проекта:

```
tree ~/wordpress-project
```

```
/home/student/wordpress-project
├── backup.sh
├── config
│   ├── nginx.conf
│   └── php.ini
├── hello.sh
├── logs
│   └── access.log
```

```
└── site
    └── index.html

3 directories, 6 files
```

Сравните с тем, как эту же информацию показывает `ls -R` — вывод `tree` гораздо нагляднее.

Полезные флаги:

```
# Показать только каталоги (без файлов)
tree -d ~/wordpress-project

# Показать права доступа
tree -p ~/wordpress-project

# Ограничить глубину вложенности
tree -L 1 ~/wordpress-project
```

Практическое задание

1. Обновите список пакетов командой `apt update`. Обратите внимание на вывод — сколько репозиториев было опрошено?
2. Найдите пакет `curl` с помощью `apt search`. Посмотрите информацию о нём (`apt show curl`). Что делает эта программа?
3. Установите `curl`. Проверьте, что он установлен:

   ```
   curl --version
   ```

4. Выполните `dpkg -l | grep curl`. Что означает статус `ii` в начале строки?
5. Удалите `curl` командой `apt remove`. Проверьте, что программа удалена (`curl --version` должна вернуть ошибку). Затем установите его заново.
6. Посмотрите содержимое файла `/etc/apt/sources.list`. Какие репозитории подключены? Какие секции (main, contrib и т.д.) используются?
7. Выполните `apt list --upgradable`, чтобы увидеть, какие пакеты можно обновить. Если такие есть — обновите их командой `apt upgrade`.

Итоги

В этой главе вы:

- узнали, что такое пакеты, зависимости и репозитории;
- разобрались со структурой файла `/etc/apt/sources.list` и секциями репозитория Debian (main, contrib, non-free);
- научились обновлять список пакетов (`apt update`);
- освоили поиск (`apt search`), просмотр информации (`apt show`) и проверку установленных пакетов (`dpkg -l`);
- научились устанавливать (`apt install`), обновлять (`apt upgrade`) и удалять пакеты (`apt remove`, `apt purge`, `apt autoremove`);
- установили и попробовали утилиту `tree`.

В следующей главе мы разберём **пользователей, группы и sudo** — узнаем, как устроена система пользователей в Linux и как безопасно выполнять административные задачи.

Этап 8. Пользователи, группы и sudo

Зачем в Linux несколько пользователей

В Windows вы привыкли к одной учётной записи, под которой делаете всё: работаете с документами, устанавливаете программы, меняете настройки. Иногда система просит «права администратора» — и вы нажимаете «Да».

В Linux подход строже. Каждый процесс и каждый файл принадлежат конкретному пользователю. Веб-сервер работает от одного пользователя, база данных — от другого, а вы подключаетесь по SSH от третьего. Это сделано намеренно:

- **Изоляция.** Если взломают веб-сервер, злоумышленник получит права только этого пользователя, а не всей системы.
- **Разграничение ответственности.** Каждый сервис имеет доступ только к своим файлам.
- **Аудит.** Всегда видно, кто и что сделал.

  Привязка к проекту. В этой главе мы создадим пользователя `webmaster` и группу `webdev` для управления нашим wordpress-project. В главе 11, когда будем настраивать nginx и PHP-FPM, каждый из них будет работать от отдельного системного пользователя — и вы уже будете понимать, почему это важно.

Пользователь root

В каждой Linux-системе есть суперпользователь **root** (UID 0). Он может всё: читать любые файлы, завершать любые процессы, менять любые настройки. Работать постоянно под root — всё равно что ходить по стройке без каски: можно, но одна ошибка будет стоить дорого.

```
rm -rf /home/student/old-project   # удалит каталог студента
rm -rf /                           # уничтожит ВСЮ систему
```

Под обычным пользователем вторая команда просто не выполнится — система скажет «Permission denied». Под root она сотрёт всё без вопросов. Поэтому правило простое: **работайте под обычным пользователем, а root-права запрашивайте только когда нужно** (через `sudo`, о котором поговорим ниже).

Как устроены пользователи

UID и файл /etc/passwd

Каждый пользователь в системе имеет числовой идентификатор — **UID** (User ID). Система различает пользователей именно по UID, а не по имени.

Информация о пользователях хранится в файле `/etc/passwd`. Посмотрим его:

```
cat /etc/passwd
```

Каждая строка описывает одного пользователя. Формат:

```
имя:x:UID:GID:комментарий:домашний_каталог:оболочка
```

Пример:

```
root:0:0:root:/root:/bin/bash
student:x:1000:1000::/home/student:/bin/bash
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
```

Поле	Значение
`имя`	Логин пользователя
`x`	Раньше здесь хранился пароль; теперь он в `/etc/shadow`
`UID`	Числовой идентификатор пользователя
`GID`	Числовой идентификатор основной группы
`комментарий`	Полное имя или описание (может быть пустым)
`домашний_каталог`	Каталог, в который попадает пользователь при входе
`оболочка`	Программа, запускаемая при входе (`/bin/bash`, `/usr/sbin/nologin` и т.д.)

Системные и обычные пользователи

Пользователи делятся на две категории:

- **Системные** (UID < 1000) — создаются автоматически для сервисов. Например, `www-data` для веб-сервера, `mysql` для базы данных. У них обычно нет пароля, а оболочка установлена в `/usr/sbin/nologin` или `/bin/false` — войти под ними нельзя.
- **Обычные** (UID ⩾ 1000) — реальные люди, которые подключаются к системе. У них есть домашний каталог `/home/имя` и рабочая оболочка (`/bin/bash`).

```
UID 0          root (суперпользователь)
UID 1–999      системные пользователи (www-data, sshd, mysql...)
UID 1000+      обычные пользователи (student, webmaster...)
```

Пароли и /etc/shadow

Пароли хранятся не в `/etc/passwd`, а в отдельном файле `/etc/shadow`, доступном только root. Это сделано для безопасности: `/etc/passwd` читают многие программы, и хранить в нём пароли (даже в зашифрованном виде) было бы опасно.

Заглянуть в `/etc/shadow` можно только с правами root:

```
sudo cat /etc/shadow
```

Детали формата `/etc/shadow` для нашего курса не важны — достаточно знать, что пароли хранятся именно там.

Группы

Зачем нужны группы

В главе 03 мы разбирали права доступа в формате `rwxrwxrwx`: владелец — группа — остальные. Тогда мы фокусировались на владельце. Теперь разберёмся с группами.

Группа — это способ объединить нескольких пользователей и дать им общие права на файлы. Например:

- Три разработчика работают над одним проектом. Вместо того чтобы давать каждому права по отдельности, создаём группу `webdev` и делаем каталог проекта доступным для этой группы.
- Веб-сервер и PHP-FPM должны читать одни и те же файлы. Оба сервиса можно добавить в общую группу.

GID и файл /etc/group

Каждая группа имеет числовой идентификатор — **GID** (Group ID). Информация о группах хранится в файле `/etc/group`:

```
cat /etc/group
```

Формат строки:

```
имя_группы:x:GID:список_участников
```

Пример:

```
sudo:x:27:student
www-data:x:33:
users:x:100:student,webmaster
```

Здесь группа `sudo` содержит пользователя `student`, а группа `users` — `student` и `webmaster`.

Основная и дополнительные группы

У каждого пользователя есть:

- **Основная группа** (primary group) — указана в `/etc/passwd` (поле GID). Файлы, которые создаёт пользователь, по умолчанию принадлежат этой группе.
- **Дополнительные группы** (supplementary groups) — перечислены в `/etc/group`. Дают дополнительные права.

Посмотреть группы текущего пользователя:

```
id
```

Вывод:

```
uid=1000(student) gid=1000(student) groups=1000(student),27(sudo)
```

Здесь основная группа — `student`, дополнительная — `sudo`.

Управление пользователями

Создание пользователя: adduser

В Debian для создания пользователей используют команду `adduser`. Она задаёт интерактивные вопросы и делает всё сама: создаёт домашний каталог, копирует туда файлы из `/etc/skel`, предлагает задать пароль.

```
sudo adduser webmaster
```

Система попросит ввести пароль и необязательную информацию (полное имя, телефон и т.д.) — их можно пропустить, нажимая Enter.

> **adduser vs useradd.** В Debian есть две команды для создания пользователей. `useradd` — низкоуровневая утилита, которая создаёт запись в `/etc/passwd`, но не делает ничего другого (не создаёт домашний каталог, не просит пароль). `adduser` — дружелюбная обёртка, которая вызывает `useradd` внутри и делает всю рутину за вас. **В повседневной работе используйте `adduser`.**

Смена пароля: passwd

Сменить пароль пользователя:

```
sudo passwd webmaster
```

Сменить свой собственный пароль (без sudo):

```
passwd
```

Изменение пользователя: usermod

Команда `usermod` позволяет изменить параметры существующего пользователя. Самая частая задача — добавить пользователя в дополнительную группу:

```
sudo usermod -aG sudo webmaster
```

Флаг	Значение
`-aG`	**a**ppend (добавить) к списку **G**roups (групп)
`-d`	Изменить домашний каталог
`-s`	Изменить оболочку (shell)
`-l`	Переименовать пользователя

> **Важно:** флаг `-a` обязателен при использовании `-G`. Без него команда **заменит** все дополнительные группы пользователя на указанную, а не добавит к существующим. Это частая ошибка, которая может лишить пользователя доступа к sudo.

Удаление пользователя: deluser

```
sudo deluser webmaster          # удалить пользователя
sudo deluser --remove-home webmaster  # удалить вместе с домашним каталогом
```

Аналогично `adduser`, команда `deluser` — дружелюбная обёртка над `userdel`.

Отключение пользователя: usermod -L

Иногда удалять пользователя не нужно — достаточно **заблокировать** ему вход. Например, сотрудник ушёл в отпуск или уволился, но его файлы и домашний каталог должны остаться:

```
sudo usermod -L webmaster      # заблокировать (Lock)
```

Команда добавляет символ `!` перед хэшем пароля в `/etc/shadow`, и войти под этим пользователем больше нельзя. При этом все файлы, права и членство в группах сохраняются.

Чтобы разблокировать:

```
sudo usermod -U webmaster      # разблокировать (Unlock)
```

> **Когда блокировка лучше удаления?** Если пользователь владеет файлами или процессами, удаление может оставить в системе «бесхозные» объекты с числовым UID вместо имени. Блокировка безопаснее: доступ закрыт, а порядок в системе сохранён.

Управление группами

Создание и удаление групп

```
sudo addgroup webdev      # создать группу
sudo delgroup webdev      # удалить группу
```

Добавление пользователя в группу

Добавить существующего пользователя в существующую группу:

```
sudo adduser webmaster webdev
```

Или через `usermod`:

```
sudo usermod -aG webdev webmaster
```

Оба варианта делают одно и то же. `adduser` в данном случае проще для запоминания.

Проверка принадлежности

```
id webmaster
```

Вывод:

```
uid=1001(webmaster) gid=1001(webmaster)
↪  groups=1001(webmaster),27(sudo),1002(webdev)
```

Команда `groups` показывает то же самое, но короче:

```
groups webmaster
```

```
webmaster : webmaster sudo webdev
```

Переключение пользователей: su

Команда `su` (switch user) позволяет переключиться на другого пользователя:

```
su webmaster       # переключиться, но остаться в текущем каталоге
su - webmaster     # переключиться с полной загрузкой окружения
```

Разница важна: `su -` (с дефисом) загружает окружение целевого пользователя — его переменные, PATH, домашний каталог. Без дефиса вы окажетесь «внутри» другого пользователя, но с частью старого окружения, что может привести к путанице.

Чтобы вернуться обратно, нажмите `Ctrl+D` или введите `exit`.

Переключиться на root:

```
su -
```

Система попросит пароль root. Но в Debian (и многих других дистрибутивах) пароль root может быть не задан — тогда войти через `su` не получится. Для этого и существует `sudo`.

sudo: выполнение команд от имени root

Зачем нужен sudo

Вместо того чтобы переключаться на root, можно выполнить одну команду с правами суперпользователя:

```
sudo apt update
```

Преимущества sudo перед работой под root:

- **Точечные права.** Вы получаете root-доступ только на одну команду, а не на всю сессию.
- **Аудит.** Каждый вызов sudo записывается в системный журнал — видно, кто, когда и что выполнил.
- **Без пароля root.** Вы вводите свой пароль, а не пароль root. Пароль root можно вообще не задавать.

Группа sudo

В Debian пользователь получает право на sudo, если состоит в группе `sudo`. Мы уже добавили туда `webmaster`:

```
sudo usermod -aG sudo webmaster
```

После этого `webmaster` может выполнять любые команды через `sudo`.

> **Когда изменения вступают в силу?** Изменение групп пользователя вступает в силу при следующем входе в систему. Если вы добавили пользователя в группу, но `id` показывает старый список — нужно выйти и войти заново (или выполнить `su - webmaster` в текущем терминале).

Файл /etc/sudoers и visudo

Настройки sudo хранятся в файле `/etc/sudoers`. Редактировать его напрямую нельзя — синтаксическая ошибка может заблокировать sudo для всех пользователей. Для редактирования используется специальная команда:

```
sudo visudo
```

Она открывает `/etc/sudoers` в редакторе и **проверяет синтаксис** перед сохранением. Если вы допустили ошибку, `visudo` предупредит и не сохранит файл.

Формат правил в `/etc/sudoers`:

```
кто   на_каком_хосте=(от_чьего_имени)   какие_команды
```

Пример записи, которая даёт группе `sudo` полный доступ:

```
%sudo   ALL=(ALL:ALL) ALL
```

Разберём по частям:

Часть	Значение
`%sudo`	Группа `sudo` (знак `%` означает группу)
`ALL=`	На любом хосте
`(ALL:ALL)`	От имени любого пользователя и группы
`ALL`	Любые команды

Именно благодаря этой строке все участники группы `sudo` могут выполнять любые команды через `sudo`.

> **Каталог /etc/sudoers.d/.** Вместо редактирования основного файла можно создавать отдельные файлы в каталоге `/etc/sudoers.d/`. Это удобнее: каждое правило в своём файле, проще управлять. Файлы в этом каталоге подключаются автоматически директивой `@includedir /etc/sudoers.d` в конце `/etc/sudoers`.

Практика: настраиваем пользователя для проекта

Пришло время применить всё на практике. Мы создадим пользователя и группу для управления нашим wordpress-project.

Шаг 1. Создаём группу и пользователя

```
sudo addgroup webdev
sudo adduser webmaster
```

При создании `webmaster` задайте пароль (запомните его — он понадобится для входа).

Шаг 2. Добавляем webmaster в группы

```
sudo adduser webmaster webdev
sudo adduser webmaster sudo
```

Проверяем:

```
id webmaster
```

Вы должны увидеть группы `webmaster`, `webdev` и `sudo`.

Шаг 3. Меняем владельца каталога проекта

Вспомним, что каталог `wordpress-project` принадлежит вашему текущему пользователю. Передадим его `webmaster` и группе `webdev`:

```
sudo chown -R webmaster:webdev ~/wordpress-project
```

Но этого недостаточно. Каталог проекта находится внутри вашего домашнего каталога, а в Debian домашние каталоги создаются с правами `drwxr-x---` — другие пользователи не могут даже пройти сквозь них. Откроем проход:

```
chmod o+x ~
```

Эта команда добавляет только право на **проход** (`x`) для остальных пользователей. Читать содержимое домашнего каталога (`ls /home/student`) они по-прежнему не смогут — для этого нужен бит `r`, который мы не добавляли.

Проверяем:

```
ls -la ~/wordpress-project/
```

Все файлы и каталоги должны принадлежать `webmaster:webdev`.

Шаг 4. Настраиваем права для группы

Убедимся, что участники группы `webdev` могут читать и писать в каталог проекта:

```
sudo chmod -R g+rwX ~/wordpress-project
```

> **Почему заглавная X?** Строчная `x` добавила бы право на выполнение всем файлам, включая текстовые и конфигурационные. Заглавная `X` добавляет право на выполнение только каталогам (чтобы в них можно было входить) и файлам, которые уже имеют хотя бы один бит выполнения. Мы использовали этот приём в главе 03.

Шаг 5. Проверяем от имени webmaster

```
su - webmaster
cd ~student/wordpress-project
ls -la
cat config/nginx.conf
echo "test" >> site/index.html
cat site/index.html
exit
```

Если всё настроено правильно, `webmaster` сможет читать и изменять файлы проекта.

> **Почему `chmod o+x ~`, а не `chmod o+rx ~`?** Бит `x` на каталоге разрешает проход через него (то есть обращение к файлам внутри по полному пути). Бит `r` разрешает чтение списка файлов. Мы добавили только `x`: webmaster сможет попасть в `wordpress-project`, но не сможет просмотреть, что ещё лежит в вашем домашнем каталоге. Это стандартная практика, когда нужно предоставить доступ к вложенному каталогу, не раскрывая всё содержимое.

Справочник команд

Команда	Описание
`adduser имя`	Создать пользователя (интерактивно)
`deluser имя`	Удалить пользователя
`deluser --remove-home имя`	Удалить пользователя с домашним каталогом
`usermod -L имя`	Заблокировать пользователя (без удаления)
`usermod -U имя`	Разблокировать пользователя
`passwd имя`	Задать/сменить пароль
`usermod -aG группа имя`	Добавить пользователя в группу
`addgroup имя`	Создать группу
`delgroup имя`	Удалить группу
`adduser пользователь группа`	Добавить пользователя в группу
`id имя`	Показать UID, GID и группы пользователя
`groups имя`	Показать группы пользователя
`su - имя`	Переключиться на пользователя (с загрузкой окружения)
`sudo команда`	Выполнить команду от имени root
`sudo visudo`	Безопасно редактировать `/etc/sudoers`
`cat /etc/passwd`	Посмотреть список пользователей
`cat /etc/group`	Посмотреть список групп

Практические задания

1. Посмотрите содержимое файла `/etc/passwd`. Найдите строку своего пользователя. Определите его UID, GID, домашний каталог и оболочку.
2. Выполните команду `id` без аргументов. Сравните вывод с тем, что нашли в `/etc/passwd`. В каких дополнительных группах вы состоите?
3. Создайте пользователя `testuser` командой `sudo adduser testuser`. Проверьте, что он появился в `/etc/passwd` (используйте `grep`). Какой UID ему присвоен?
4. Создайте группу `developers`. Добавьте в неё `testuser`. Убедитесь командой `id testuser`, что группа появилась в списке.
5. Переключитесь на `testuser` командой `su - testuser`. Попробуйте выполнить `sudo apt update`. Что произойдёт и почему?
6. Вернитесь в свою учётную запись (`exit`). Добавьте `testuser` в группу `sudo`. Снова переключитесь на `testuser` и повторите `sudo apt update`. Теперь команда должна работать.
7. Удалите пользователя `testuser` вместе с его домашним каталогом. Убедитесь, что он исчез из `/etc/passwd`.
8. Посмотрите файл `/etc/group`. Найдите группу `webdev`. Какие пользователи в ней состоят?

Итоги

В этой главе вы:

- узнали, зачем в Linux несколько пользователей и почему опасно работать под root;
- разобрались с файлами `/etc/passwd`, `/etc/shadow` и `/etc/group`;
- научились создавать и удалять пользователей (`adduser`, `deluser`) и группы (`addgroup`, `delgroup`);
- освоили изменение пользователей (`usermod`, `passwd`) и проверку принадлежности к группам (`id`, `groups`);
- научились переключаться между пользователями (`su`) и выполнять команды от имени root (`sudo`);
- познакомились с `visudo` и форматом `/etc/sudoers`;
- создали пользователя `webmaster` и группу `webdev` для сквозного проекта, настроили права на каталог `wordpress-project`.

В следующей главе мы разберём **процессы и сервисы (systemd)** — узнаем, как Linux запускает и управляет программами, и научимся работать с `systemctl` и `journalctl`.

Этап 9. Процессы и сервисы (systemd)

Что такое процесс

В Windows вы наверняка открывали «Диспетчер задач» (Ctrl+Shift+Esc) и видели список запущенных программ: браузер, антивирус, проводник. Каждая запущенная программа — это **процесс**.

В Linux всё точно так же. Каждая работающая программа — это процесс с уникальным номером **PID** (Process ID). Когда вы подключаетесь по SSH, запускается процесс оболочки bash. Когда набираете `ls`, создаётся ещё один процесс, который выполняет эту команду и завершается.

Процессы образуют **дерево**: каждый процесс (кроме самого первого) создан другим процессом — **родительским**. Самый первый процесс в системе имеет PID 1 — это **systemd**, о котором мы подробно поговорим ниже.

```
systemd (PID 1)
├── sshd
│   └── sshd (ваше подключение)
│       └── bash
│           └── ls
├── cron
└── nginx
    ├── nginx (worker)
    └── nginx (worker)
```

> **Привязка к проекту.** В этой главе мы научимся управлять процессами и сервисами. В практической части обернём наш скрипт `backup.sh` из главы 06 в systemd-сервис с таймером — получится автоматический бэкап wordpress-project по расписанию.

Просмотр процессов

ps — снимок процессов

Команда `ps` показывает список процессов **в данный момент** (как фотография — снимок, а не видео):

```
ps aux
```

Флаг	Значение
`a`	Показать процессы всех пользователей
`u`	Показать подробный формат (пользователь, CPU, память)
`x`	Показать процессы без терминала (демоны, сервисы)

Вывод содержит много строк. Чтобы найти конкретный процесс, используйте `grep` (глава 05):

```
ps aux | grep sshd
```

```
root       654  0.0  0.3  15432  6892 ?        Ss   10:00   0:00 sshd:
 ↪  /usr/sbin/sshd -D
root       987  0.0  0.4  16784  8256 ?        Ss   10:05   0:00 sshd: student
 ↪  [priv]
student    991  0.0  0.2  16784  5432 ?        S    10:05   0:00 sshd:
 ↪  student@pts/0
student   1023  0.0  0.0   6420   720 pts/0    S+   10:10   0:00 grep sshd
```

Обратите внимание: последняя строка — это сам процесс `grep`, который тоже содержит слово «sshd». Это нормально.

Основные столбцы вывода `ps aux`:

Столбец	Значение
`USER`	Владелец процесса
`PID`	Идентификатор процесса
`%CPU`	Использование процессора
`%MEM`	Использование памяти
`COMMAND`	Команда, которой запущен процесс

pstree — дерево процессов

Чтобы увидеть иерархию процессов (кто кого породил):

```
pstree
```

С отображением PID:

```
pstree -p
```

top — интерактивный мониторинг

Команда `top` показывает процессы в реальном времени — как «Диспетчер задач» в Windows. Информация обновляется каждые несколько секунд:

```
top
```

Полезные клавиши внутри `top`:

Клавиша	Действие
`q`	Выйти
`k`	Завершить процесс (спросит PID)
`M`	Сортировать по памяти
`P`	Сортировать по CPU
`1`	Показать все ядра процессора

> **htop.** Существует улучшенная версия — `htop`, с цветным интерфейсом и более удобным управлением. Если хотите попробовать, установите её: `sudo apt install htop`. Для нашего курса достаточно стандартного `top`.

Управление процессами

Сигналы

Чтобы повлиять на работающий процесс (остановить, перезапустить), ему отправляют **сигнал**. Самые важные:

Сигнал	Номер	Описание
`SIGTERM`	15	Вежливая просьба завершиться. Процесс может сохранить данные и выйти корректно
`SIGKILL`	9	Принудительное завершение. Процесс не может его проигнорировать
`SIGHUP`	1	«Перечитай конфигурацию». Многие сервисы перезагружают настройки без перезапуска

kill — отправка сигнала

```
kill 1234          # отправить SIGTERM процессу с PID 1234
kill -9 1234       # отправить SIGKILL (принудительное завершение)
kill -HUP 1234     # отправить SIGHUP (перечитать конфигурацию)
```

Всегда начинайте с `kill` (SIGTERM) — дайте процессу завершиться корректно. Используйте `kill -9` только если процесс не реагирует на обычный `kill`.

Фоновые процессы

Когда вы запускаете команду в терминале, она занимает его до завершения. Но можно запустить команду **в фоне**:

```
sleep 60 &
```

Символ `&` в конце запускает команду в фоновом режиме. Терминал остаётся свободным.

Полезные клавиши и команды:

Действие	Команда
Прервать текущий процесс	`Ctrl+C` (отправляет SIGINT)
Приостановить текущий процесс	`Ctrl+Z` (отправляет SIGTSTP)
Посмотреть фоновые задачи	`jobs`
Вернуть задачу на передний план	`fg`
Продолжить задачу в фоне	`bg`

Пример работы:

```
sleep 120          # запустили, терминал занят
                   # нажимаем Ctrl+Z — процесс приостановлен
jobs               # видим: [1]+  Stopped  sleep 120
bg                 # продолжить в фоне
jobs               # видим: [1]+  Running  sleep 120 &
fg                 # вернуть на передний план
                   # нажимаем Ctrl+C — процесс завершён
```

Что такое systemd

До сих пор мы работали с обычными процессами — запустили команду, она поработала и завершилась. Но в системе есть программы, которые должны работать постоянно: SSH-сервер, веб-сервер, планировщик задач. Такие постоянно работающие программы называют **сервисами** (или **демонами**).

Кто-то должен запустить все сервисы при загрузке системы, следить за ними и перезапускать, если они упали. Эту роль выполняет **systemd** — система инициализации и управления сервисами в Debian (и большинстве современных дистрибутивов Linux).

systemd — это тот самый процесс с PID 1, корень дерева процессов. Он запускается ядром первым и затем поднимает всю остальную систему: монтирует диски, настраивает сеть, запускает сервисы.

Каждый объект, которым управляет systemd, называется **юнитом** (unit). Юниты бывают разных типов:

Тип	Расширение	Описание
Сервис	`.service`	Программа-демон (nginx, sshd, cron)
Таймер	`.timer`	Запуск задач по расписанию (замена cron)
Сокет	`.socket`	Активация сервиса при подключении к порту
Точка монтирования	`.mount`	Монтирование файловой системы
Цель	`.target`	Группа юнитов (аналог «уровня запуска»)

Чаще всего вы будете работать с типами `.service` и `.timer`.

systemctl: управление сервисами

`systemctl` — главная команда для работы с systemd. Рассмотрим её на примере сервиса SSH, который уже работает в вашей системе (без него вы бы не смогли подключиться).

Статус сервиса

```
systemctl status ssh
```

```
● ssh.service - OpenBSD Secure Shell server
     Loaded: loaded (/lib/systemd/system/ssh.service; enabled; preset: enabled)
     Active: active (running) since Mon 2026-03-16 10:00:00 MSK; 2h ago
       Docs: man:sshd(8)
   Main PID: 654 (sshd)
      Tasks: 1 (limit: 4647)
     Memory: 5.2M
        CPU: 132ms
     CGroup: /system.slice/ssh.service
             └─654 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"
```

Ключевые строки:

Строка	Значение
`Loaded: loaded (...; enabled)`	Юнит загружен; **enabled** — запускается автоматически при загрузке
`Active: active (running)`	Сервис работает прямо сейчас
`Main PID: 654`	PID основного процесса

Запуск, остановка, перезапуск

```
sudo systemctl start ssh       # запустить сервис
sudo systemctl stop ssh        # остановить сервис
sudo systemctl restart ssh     # перезапустить (stop + start)
sudo systemctl reload ssh      # перечитать конфигурацию без перезапуска
```

restart vs reload. `restart` полностью останавливает и запускает сервис заново — клиенты на секунду теряют связь. `reload` просит сервис перечитать конфигурацию «на лету», без остановки. Не все сервисы поддерживают `reload` — если не поддерживает, команда вернёт ошибку.

Автозапуск при загрузке

```
sudo systemctl enable ssh      # включить автозапуск
sudo systemctl disable ssh     # отключить автозапуск
```

`enable` не запускает сервис прямо сейчас — только добавляет его в автозагрузку. Чтобы сделать и то, и другое одной командой:

```
sudo systemctl enable --now ssh
```

Список сервисов

Посмотреть все загруженные сервисы:

```
systemctl list-units --type=service
```

Только активные (работающие):

```
systemctl list-units --type=service --state=running
```

journalctl: чтение логов

systemd ведёт централизованный журнал — все сообщения от всех сервисов собираются в одном месте. Читать этот журнал можно командой `journalctl`.

Логи конкретного сервиса

```
journalctl -u ssh
```

Флаг `-u` (unit) фильтрует записи по имени юнита.

Последние записи

```
journalctl -u ssh -n 20        # последние 20 строк
```

Следить в реальном времени

```
journalctl -u ssh -f
```

Флаг `-f` (follow) работает как `tail -f` — показывает новые записи по мере их появления. Выход — `Ctrl+C`.

Фильтрация по времени

```
journalctl -u ssh --since "2026-03-16 10:00"
journalctl -u ssh --since "1 hour ago"
journalctl -u ssh --since today
```

> **Подробнее о логировании.** В этой главе мы рассматриваем `journalctl` как инструмент диагностики сервисов. Полноценный разбор логирования и мониторинга — в главе 12.

Устройство unit-файла

Мы научились управлять сервисами снаружи. Теперь заглянем внутрь — посмотрим, как описан сервис.

Где хранятся unit-файлы

Unit-файлы лежат в нескольких каталогах:

Каталог	Назначение
`/lib/systemd/system/`	Файлы, установленные пакетами (nginx, ssh, cron)
`/etc/systemd/system/`	Файлы, созданные администратором (имеют приоритет)

Посмотреть содержимое unit-файла можно командой:

```
systemctl cat ssh
```

Разбор ssh.service

Рассмотрим упрощённую структуру unit-файла SSH-сервера:

```
[Unit]
Description=OpenBSD Secure Shell server
Documentation=man:sshd(8)
After=network.target

[Service]
Type=notify
ExecStartPre=/usr/sbin/sshd -t
ExecStart=/usr/sbin/sshd -D
ExecReload=/bin/kill -HUP $MAINPID
Restart=on-failure

[Install]
WantedBy=multi-user.target
```

Разберём по секциям:

Секция [Unit] — общее описание:

Директива	Значение
`Description`	Человекочитаемое описание сервиса
`Documentation`	Ссылка на документацию
`After`	Запускать после указанных юнитов (здесь — после настройки сети)

Секция [Service] — как запускать и контролировать:

Директива	Значение
`Type`	Тип сервиса (`notify`, `simple`, `oneshot`, `forking`)
`ExecStartPre`	Команда перед запуском (здесь — проверка конфигурации)
`ExecStart`	Основная команда запуска
`ExecReload`	Команда для `systemctl reload` (перечитать конфигурацию)
`Restart`	Когда перезапускать (`on-failure` — только при сбое)

Секция [Install] — автозапуск:

Директива	Значение
`WantedBy`	К какой цели (target) привязать при `systemctl enable`

Что такое multi-user.target? Это цель, которая соответствует обычному режиму работы сервера (многопользовательский режим с сетью, но без графического интерфейса). Большинство серверных сервисов привязываются именно к ней.

Практика: оборачиваем backup.sh в systemd-сервис

В главе 06 мы написали скрипт `backup.sh`, который создаёт архив каталога `wordpress-project`. Сейчас мы превратим его в systemd-сервис с таймером — чтобы бэкап запускался автоматически по расписанию. Это современная альтернатива планировщику `cron`.

Шаг 1. Подготавливаем скрипт

Убедимся, что скрипт находится в правильном месте и имеет права на выполнение:

```
ls -l ~/wordpress-project/backup.sh
```

Если прав на выполнение нет, добавим:

```
chmod +x ~/wordpress-project/backup.sh
```

Шаг 2. Создаём service-файл

Создадим файл `/etc/systemd/system/wordpress-backup.service`:

```
sudo nano /etc/systemd/system/wordpress-backup.service
```

Содержимое:

```
[Unit]
Description=Backup wordpress-project
After=local-fs.target

[Service]
Type=oneshot
User=webmaster
ExecStart=/home/student/wordpress-project/backup.sh
```

Разберём, что здесь написано:

- `Type=oneshot` — сервис выполняет задачу один раз и завершается (в отличие от `simple`, который работает постоянно).
- `User=webmaster` — скрипт запускается от имени пользователя `webmaster`, которого мы создали в главе 08.
- `ExecStart` — путь к нашему скрипту бэкапа.

Шаг 3. Создаём timer-файл

Создадим файл `/etc/systemd/system/wordpress-backup.timer`:

```
sudo nano /etc/systemd/system/wordpress-backup.timer
```

Содержимое:

```
[Unit]
Description=Run wordpress-backup daily

[Timer]
OnCalendar=daily
Persistent=true

[Install]
WantedBy=timers.target
```

Разберём секцию `[Timer]`:

Директива	Значение
`OnCalendar=daily`	Запускать раз в сутки (в полночь)
`Persistent=true`	Если система была выключена в момент запуска — выполнить при следующем включении

Другие варианты расписания `OnCalendar`:

Значение	Когда
`hourly`	Каждый час
`daily`	Раз в сутки
`weekly`	Раз в неделю
`*-*-* 03:00:00`	Каждый день в 3:00
`Mon *-*-* 09:00:00`	Каждый понедельник в 9:00

Шаг 4. Запускаем таймер

Сообщим systemd, что появились новые unit-файлы:

```
sudo systemctl daemon-reload
```

Включим и запустим таймер:

```
sudo systemctl enable --now wordpress-backup.timer
```

Шаг 5. Проверяем

Посмотрим, что таймер активен:

```
systemctl list-timers
```

В списке должен появиться `wordpress-backup.timer` с указанием следующего времени срабатывания.

Запустим сервис вручную, чтобы убедиться, что он работает:

```
sudo systemctl start wordpress-backup.service
```

Проверим статус:

```
systemctl status wordpress-backup.service
```

Посмотрим лог:

```
journalctl -u wordpress-backup.service
```

Вы должны увидеть сообщения о выполнении скрипта и успешном завершении.

Справочник команд

Процессы

Команда	Описание
`ps aux`	Показать все процессы
`ps aux | grep имя`	Найти процесс по имени
`pstree`	Показать дерево процессов
`pstree -p`	Дерево процессов с PID
`top`	Интерактивный мониторинг процессов
`kill PID`	Послать SIGTERM (завершить корректно)
`kill -9 PID`	Послать SIGKILL (завершить принудительно)
`kill -HUP PID`	Послать SIGHUP (перечитать конфигурацию)

Фоновые процессы

Команда	Описание
`команда &`	Запустить в фоне
`Ctrl+C`	Прервать текущий процесс
`Ctrl+Z`	Приостановить текущий процесс
`jobs`	Список фоновых задач
`fg`	Вернуть задачу на передний план
`bg`	Продолжить задачу в фоне

systemctl

Команда	Описание
`systemctl status имя`	Статус сервиса
`systemctl start имя`	Запустить сервис
`systemctl stop имя`	Остановить сервис
`systemctl restart имя`	Перезапустить сервис
`systemctl reload имя`	Перечитать конфигурацию
`systemctl enable имя`	Включить автозапуск
`systemctl disable имя`	Отключить автозапуск
`systemctl enable --now имя`	Включить автозапуск и запустить
`systemctl list-units --type=service`	Список сервисов
`systemctl list-timers`	Список таймеров
`systemctl cat имя`	Показать unit-файл
`systemctl daemon-reload`	Перечитать unit-файлы после изменений

journalctl

Команда	Описание
`journalctl -u имя`	Логи сервиса
`journalctl -u имя -n 20`	Последние 20 строк
`journalctl -u имя -f`	Следить в реальном времени
`journalctl -u имя --since today`	Логи с начала дня

Практические задания

1. Выполните `ps aux` и найдите процесс вашей bash-сессии. Запишите его PID. Совпадает ли он с выводом переменной `echo $$`?
2. Выполните `pstree -p` и найдите свою сессию. Проследите цепочку от `systemd (1)` до вашего `bash`. Какие промежуточные процессы в этой цепочке?
3. Запустите `sleep 300 &` в фоне. Найдите этот процесс через `ps aux | grep sleep`. Завершите его командой `kill`. Убедитесь, что он исчез (`jobs`).
4. Выполните `systemctl status ssh`. Какой PID у основного процесса? Когда сервис был запущен? Включён ли автозапуск (enabled/disabled)?
5. Посмотрите unit-файл сервиса cron: `systemctl cat cron`. Найдите директиву `ExecStart`. Какая команда запускает планировщик?
6. Выполните `systemctl list-units --type=service --state=running`. Сколько сервисов сейчас работает в системе? Найдите знакомые (ssh, cron).
7. Посмотрите логи SSH за сегодня: `journalctl -u ssh --since today`. Видны ли в них ваши подключения?
8. Если вы создали таймер `wordpress-backup.timer`, проверьте его статус через `systemctl list-timers`. Когда запланирован следующий запуск? Запустите сервис вручную (`sudo systemctl start wordpress-backup.service`) и убедитесь в успешном выполнении через `journalctl`.

Итоги

В этой главе вы:

- узнали, что такое процессы, PID и дерево процессов;
- научились просматривать процессы (`ps aux`, `pstree`, `top`) и управлять ими (сигналы `kill`, фоновые задачи);
- познакомились с systemd — системой управления сервисами;
- освоили `systemctl` для запуска, остановки, перезапуска сервисов и управления автозагрузкой;
- научились читать логи сервисов через `journalctl`;
- разобрали структуру unit-файла (секции Unit, Service, Install);
- создали собственный сервис и таймер для автоматического бэкапа wordpress-project.

В следующей главе мы разберём **сетевые настройки** — узнаем, как Linux работает с сетью, познакомимся с командами `ip` и `ss`, настроим файрвол и разберём основы DNS.

Этап 10. Сеть: ip, ss, firewall (ufw), основы DNS

Зачем разбираться в сети

Веб-сервер без сети — как телефон без связи. В главе 11 мы установим nginx и откроем сайт миру, но сначала нужно понять, как Linux работает с сетью: какой у машины IP-адрес, какие порты открыты, кто к ним подключается и как защитить сервер файрволом.

В Windows вы, возможно, пользовались командой `ipconfig` или заходили в «Центр управления сетями». В Linux те же задачи решаются из командной строки — и здесь это даже удобнее.

> **Привязка к проекту.** В этой главе мы узнаем IP-адрес нашего сервера, научимся проверять сетевые соединения и настроим файрвол, открыв порты для SSH (22) и будущего веб-сервера (80, 443). Всё это понадобится в главе 11, когда будем запускать nginx.

Врезка: основы IP-сетей

Прежде чем переходить к командам, разберёмся с базовыми понятиями. Если вы уже знакомы с IP-адресами и портами — можете пропустить этот раздел.

IP-адреса

Каждое устройство в сети имеет уникальный **IP-адрес** — как почтовый адрес для компьютера. В формате IPv4 он выглядит так:

```
192.168.1.100
```

Четыре числа от 0 до 255, разделённые точками.

IP-адреса делятся на **частные** (используются внутри локальной сети) и **публичные** (видны в интернете):

Диапазон	Тип	Пример использования
`10.0.0.0` – `10.255.255.255`	Частный	Корпоративные сети
`172.16.0.0` – `172.31.255.255`	Частный	Облачные провайдеры
`192.168.0.0` – `192.168.255.255`	Частный	Домашние сети

Диапазон	Тип	Пример использования
Всё остальное	Публичный	Серверы в интернете

Ваша виртуальная машина, скорее всего, имеет частный адрес (например, `192.168.x.x` или `10.x.x.x`).

Особый адрес **127.0.0.1** (localhost) — это «сам компьютер». Он всегда указывает на текущую машину.

Маска подсети и CIDR

Маска подсети определяет, какая часть адреса обозначает сеть, а какая — конкретный компьютер в ней. Записывается двумя способами:

```
255.255.255.0      — классическая запись
/24                — CIDR-нотация (то же самое)
```

Запись `192.168.1.0/24` означает: сеть `192.168.1.x`, где последнее число (от 0 до 255) — адрес компьютера. В такой сети может быть до 254 устройств.

Для нашего курса достаточно знать, что `/24` — самая распространённая маска для небольших сетей.

Шлюз (gateway)

Шлюз — это «дверь наружу» из локальной сети. Когда компьютер хочет связаться с адресом за пределами своей подсети, он отправляет пакет через шлюз. Обычно это маршрутизатор (роутер) с адресом вроде `192.168.1.1`.

Порты

IP-адрес определяет компьютер, а **порт** — конкретную программу на нём. Порт — это число от 1 до 65535.

Когда вы подключаетесь по SSH, ваш клиент соединяется с адресом сервера на **порт 22**. Когда откроете сайт в браузере — браузер обратится на **порт 80** (HTTP) или **443** (HTTPS).

Известные порты:

Порт	Сервис
22	SSH
80	HTTP (веб-сервер)
443	HTTPS (веб-сервер, шифрованное соединение)
3306	MySQL / MariaDB
5432	PostgreSQL

Аналогия: IP-адрес — это адрес здания, а порт — номер квартиры. Почтальон (сетевой пакет) приходит по адресу и стучится в нужную квартиру.

TCP и UDP

Два основных протокола передачи данных:

- **TCP** — надёжный: гарантирует доставку и порядок пакетов. Используется для SSH, HTTP, баз данных — всего, где важна целостность данных.
- **UDP** — быстрый, но без гарантий: пакеты могут потеряться. Используется для DNS-запросов, видео, игр — где скорость важнее надёжности.

В большинстве серверных задач вы будете работать с TCP.

---

ip: сетевые интерфейсы и адреса

Команда `ip` — главный инструмент для просмотра и настройки сети в Linux. Она заменила устаревшую `ifconfig`.

Посмотреть IP-адреса

```
ip addr show
```

Или короче:

```
ip a
```

Пример вывода:

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536
    inet 127.0.0.1/8 scope host lo
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500
    inet 192.168.1.100/24 brd 192.168.1.255 scope global eth0
```

Что здесь важно:

Элемент	Значение
`lo`	Loopback — виртуальный интерфейс «на себя» (127.0.0.1)
`eth0`	Сетевой интерфейс (может называться `ens3`, `enp0s3` и т.д.)
`UP`	Интерфейс включён и работает
`inet 192.168.1.100/24`	IP-адрес и маска подсети

Названия интерфейсов. В разных системах сетевой интерфейс может называться по-разному: `eth0`, `ens3`, `enp0s3`. Это зависит от системы именования. В примерах мы используем `eth0`, но у вас имя может отличаться — это нормально.

Посмотреть маршруты

```
ip route show
```

Или короче:

```
ip r
```

```
default via 192.168.1.1 dev eth0
192.168.1.0/24 dev eth0 proto kernel scope link src 192.168.1.100
```

Первая строка — **шлюз по умолчанию** (`192.168.1.1`). Все пакеты «наружу» отправляются через него.

Посмотреть интерфейсы

```
ip link show
```

Показывает список сетевых интерфейсов и их состояние (UP/DOWN), без IP-адресов.

ss: сетевые соединения

Команда `ss` (socket statistics) показывает, какие программы слушают сетевые порты и какие соединения установлены. Она заменила устаревшую `netstat`.

Какие порты слушают

Самая полезная комбинация флагов:

```
ss -tlnp
```

Флаг	Значение
`-t`	Только TCP-соединения
`-l`	Только слушающие (listening) порты
`-n`	Показывать номера портов, а не имена сервисов
`-p`	Показывать имя процесса (нужен sudo для чужих процессов)

Пример вывода:

```
State   Recv-Q  Send-Q  Local Address:Port  Peer Address:Port  Process
LISTEN  0       128     0.0.0.0:22           0.0.0.0:*
 ↪  users:(("sshd",pid=654,fd=3))
```

Здесь видно: процесс `sshd` слушает порт 22 на всех интерфейсах (`0.0.0.0`). Когда мы установим nginx, здесь появится ещё и порт 80.

Чтобы увидеть процессы всех пользователей:

```
sudo ss -tlnp
```

Установленные соединения

```
ss -tn
```

Покажет активные TCP-соединения (без `-l` — только установленные, не слушающие). Вы увидите своё SSH-подключение.

Проверка соединений

ping — доступность хоста

```
ping 8.8.8.8
```

Отправляет пакеты и показывает время ответа. `8.8.8.8` — публичный DNS-сервер Google, удобен для проверки доступа в интернет.

```
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=118 time=5.42 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=118 time=5.38 ms
```

Остановить — `Ctrl+C`. Или указать количество пакетов:

```
ping -c 3 8.8.8.8      # отправить 3 пакета и остановиться
```

Если ответа нет — хост недоступен, или между вами файрвол блокирует ICMP-пакеты.

curl — HTTP-запрос

`curl` отправляет HTTP-запрос к серверу. Полезно для проверки веб-сервисов:

```
curl http://example.com
```

Показывает HTML-код страницы. Чтобы увидеть только заголовки ответа:

```
curl -I http://example.com
```

```
HTTP/1.1 200 OK
Content-Type: text/html; charset=UTF-8
```

> **Привязка к проекту.** В главе 11, когда установим nginx, мы будем проверять его работу именно через `curl http://localhost`.

DNS: разрешение имён

Что такое DNS

Компьютеры общаются по IP-адресам, но людям удобнее запоминать имена: `google.com` вместо `142.250.74.46`. Превращением имён в адреса занимается **DNS** (Domain Name System) — своего рода телефонная книга интернета.

Когда вы набираете `ping google.com`, система сначала спрашивает DNS-сервер: «Какой IP у google.com?» — получает ответ и только потом отправляет пакет.

/etc/resolv.conf — настройки DNS

Файл `/etc/resolv.conf` содержит адрес DNS-сервера, к которому обращается система:

```
cat /etc/resolv.conf
```

```
nameserver 192.168.1.1
```

Здесь `nameserver` — адрес DNS-сервера (часто совпадает со шлюзом).

/etc/hosts — локальные записи

Файл `/etc/hosts` позволяет задать соответствие «имя → IP» вручную, минуя DNS. Система проверяет его **раньше**, чем обращается к DNS-серверу:

```
cat /etc/hosts
```

```
127.0.0.1       localhost
127.0.1.1       debian
```

Это удобно для разработки: можно добавить запись вроде `127.0.0.1 mysite.local` и обращаться к локальному серверу по имени.

Проверка DNS

Команда `host` показывает, в какой IP-адрес разрешается доменное имя:

```
host google.com
```

```
google.com has address 142.250.74.46
google.com has IPv6 address 2a00:1450:4010:c05::71
google.com mail is handled by 10 smtp.google.com.
```

Более подробная информация — через `dig` (если установлен):

```
dig google.com
```

ufw: файрвол

Зачем нужен файрвол

Файрвол (межсетевой экран) контролирует, какой сетевой трафик разрешён, а какой заблокирован. По умолчанию сервер принимает подключения на любой открытый порт. Файрвол позволяет ограничить доступ: например, разрешить SSH (порт 22) и HTTP (порт 80), но заблокировать всё остальное.

> **ufw и nftables.** В ядре Linux за фильтрацию трафика отвечает подсистема **nftables** — мощный, но сложный инструмент с собственным языком правил. **ufw** (Uncomplicated Firewall) — удобная обёртка над nftables, аналогично тому, как `adduser` — обёртка над `useradd`. Для повседневных задач ufw более чем достаточен.

Установка и включение

В Debian ufw может быть не установлен по умолчанию:

```
sudo apt install ufw
```

Перед включением файрвола **обязательно** разрешите SSH, иначе вы потеряете доступ к серверу:

```
sudo ufw allow 22
sudo ufw enable
```

> **Внимание!** Если вы включите ufw, не разрешив порт 22, ваше SSH-соединение будет разорвано, и вы не сможете подключиться к серверу. Всегда разрешайте SSH перед включением файрвола.

Статус и правила

```
sudo ufw status
```

```
Status: active

To                         Action      From
--                         ------      ----
22                         ALLOW       Anywhere
```

Подробный вывод с номерами правил:

```
sudo ufw status numbered
```

Разрешение и запрет

```
sudo ufw allow 80          # разрешить HTTP
sudo ufw allow 443         # разрешить HTTPS
sudo ufw allow 80/tcp      # разрешить только TCP на порт 80
```

Можно использовать имена сервисов:

```
sudo ufw allow ssh         # то же, что allow 22
sudo ufw allow http        # то же, что allow 80
```

Запретить порт:

```
sudo ufw deny 3306         # запретить доступ к MySQL снаружи
```

Удаление правил

```
sudo ufw status numbered   # посмотреть номера правил
sudo ufw delete 3          # удалить правило номер 3
```

Или по описанию:

```
sudo ufw delete allow 80
```

Отключение файрвола

```
sudo ufw disable
```

Правила сохраняются — при следующем `ufw enable` они вернутся.

Практика: настраиваем сеть для проекта

Шаг 1. Узнаём IP-адрес сервера

```
ip addr show
```

Найдите адрес вашего сетевого интерфейса (не `127.0.0.1`). Запишите его — он понадобится в главе 11 для доступа к nginx.

Шаг 2. Проверяем шлюз и доступ в интернет

```
ip route show
ping -c 3 8.8.8.8
```

Если `ping` работает — у вас есть доступ в интернет (нужен для установки пакетов).

Шаг 3. Смотрим, что слушает сеть

```
sudo ss -tlnp
```

Вы должны увидеть как минимум `sshd` на порту 22. Запомните эту команду — в главе 11 вы будете проверять, что nginx слушает порт 80.

Шаг 4. Настраиваем файрвол

```
sudo apt install ufw
sudo ufw allow 22          # SSH — обязательно первым!
sudo ufw allow 80          # HTTP — для будущего nginx
sudo ufw allow 443         # HTTPS — для будущего nginx
sudo ufw enable
sudo ufw status
```

Проверяем, что SSH по-прежнему работает (откройте второй терминал и попробуйте подключиться).

Шаг 5. Проверяем DNS

```
host google.com
cat /etc/resolv.conf
```

Убедитесь, что DNS работает — имена разрешаются в IP-адреса.

Справочник команд

Сетевые интерфейсы и адреса

Команда	Описание
`ip addr show` (или `ip a`)	Показать IP-адреса интерфейсов
`ip route show` (или `ip r`)	Показать таблицу маршрутов и шлюз
`ip link show`	Показать сетевые интерфейсы

Соединения и диагностика

Команда	Описание
`ss -tlnp`	Показать слушающие TCP-порты с процессами
`ss -tn`	Показать установленные TCP-соединения
`ping -c 3 адрес`	Проверить доступность хоста (3 пакета)
`curl URL`	Отправить HTTP-запрос
`curl -I URL`	Показать только HTTP-заголовки

DNS

Команда	Описание
`host имя`	Узнать IP-адрес по доменному имени
`dig имя`	Подробный DNS-запрос
`cat /etc/resolv.conf`	Посмотреть настроенный DNS-сервер
`cat /etc/hosts`	Посмотреть локальные DNS-записи

Файрвол (ufw)

Команда	Описание
`sudo ufw status`	Статус файрвола и список правил
`sudo ufw status numbered`	Правила с номерами
`sudo ufw enable`	Включить файрвол
`sudo ufw disable`	Отключить файрвол
`sudo ufw allow порт`	Разрешить порт
`sudo ufw deny порт`	Запретить порт
`sudo ufw delete номер`	Удалить правило по номеру

Практические задания

1. Выполните `ip addr show`. Найдите IP-адрес вашего сервера (не 127.0.0.1). В какой подсети он находится (посмотрите маску)?
2. Выполните `ip route show`. Какой шлюз по умолчанию (default via)? Выполните `ping -c 3` на этот шлюз — отвечает ли он?
3. Выполните `sudo ss -tlnp`. Какие сервисы слушают порты? На каких портах? Соответствует ли это тому, что вы ожидаете?
4. Выполните `ping -c 3 8.8.8.8`. Какое среднее время ответа? Теперь выполните `ping -c 3 google.com`. Если работает — значит, DNS настроен правильно.
5. Посмотрите файл `/etc/resolv.conf`. Какой DNS-сервер используется? Проверьте его работу командой `host ya.ru`.
6. Установите ufw (`sudo apt install ufw`). Разрешите порт 22, затем включите файрвол. Проверьте, что SSH по-прежнему работает (откройте новое подключение, не закрывая текущее!).
7. Добавьте правила для портов 80 и 443. Посмотрите список правил с номерами (`sudo ufw status numbered`). Удалите правило для порта 443, затем добавьте его заново.
8. Добавьте в файл `/etc/hosts` строку `127.0.0.1 mysite.local` (потребуется sudo). Проверьте командой `ping -c 1 mysite.local`, что имя разрешается в 127.0.0.1.

Итоги

В этой главе вы:

- разобрались с основами IP-сетей: адреса, маски, подсети, порты, протоколы TCP и UDP;
- научились смотреть IP-адреса и маршруты (`ip addr`, `ip route`);
- освоили проверку сетевых соединений и открытых портов (`ss -tlnp`);
- познакомились с инструментами диагностики (`ping`, `curl`);
- узнали, как работает DNS, и изучили файлы `/etc/resolv.conf` и `/etc/hosts`;
- установили и настроили файрвол ufw, открыв порты для SSH и будущего веб-сервера.

В следующей главе мы наконец **установим и настроим nginx, PHP-FPM и WordPress** — соберём все знания вместе и запустим полноценный веб-сайт на нашем сервере.

Этап 11. Установка и настройка nginx + PHP-FPM + WordPress

Что мы будем делать

Это кульминация второй части курса. Мы соберём все знания вместе и развернём полноценный веб-сайт на WordPress. Для этого нам нужны четыре компонента:

```
Браузер (клиент)
    │
    ▼
  nginx (веб-сервер, порт 80)
    │
    ├── статика (.html, .css, .js, картинки) → отдаёт сам
    │
    └── динамика (.php) → передаёт в PHP-FPM
                              │
                              ▼
                          WordPress (PHP-код)
                              │
                              ▼
                          MariaDB (база данных)
```

- **nginx** — веб-сервер. Принимает HTTP-запросы от браузера. Статические файлы (картинки, CSS) отдаёт сам, а PHP-скрипты передаёт в PHP-FPM.
- **PHP-FPM** — обработчик PHP-кода. Выполняет скрипты WordPress и возвращает результат nginx.
- **MariaDB** — база данных. WordPress хранит в ней статьи, пользователей, настройки.
- **WordPress** — CMS (система управления контентом), написанная на PHP. Генерирует HTML-страницы на основе данных из базы.

Привязка к проекту. Все предыдущие главы готовили нас к этому моменту: мы умеем работать с файлами, управлять правами, устанавливать пакеты, настраивать пользователей и сервисы, работать с сетью и файрволом. Теперь применим всё на практике.

Врезка: клиент-серверная модель и HTTP

Клиент и сервер

Когда вы открываете сайт в браузере, происходит следующее:

1. **Браузер** (клиент) отправляет **запрос** на сервер.
2. **Сервер** (nginx) обрабатывает запрос и отправляет **ответ**.

Это называется **клиент-серверной моделью**. Клиент всегда инициирует общение, сервер всегда отвечает.

HTTP-запрос

Запрос состоит из:

- **Метод** — что хочет клиент:
  - `GET` — получить страницу (самый частый)
  - `POST` — отправить данные (форма, логин)
- **URL** — какую страницу запрашивают (`/`, `/about`, `/wp-login.php`)
- **Заголовки** — дополнительная информация (тип браузера, язык и т.д.)

Пример:

```
GET /index.html HTTP/1.1
Host: mysite.local
```

HTTP-ответ

Ответ содержит:

- **Код состояния** — результат обработки
- **Тело** — содержимое страницы (HTML, JSON, картинка)

Основные коды:

Код	Значение
`200 OK`	Всё хорошо, страница найдена
`301 Moved Permanently`	Страница переехала на другой адрес
`403 Forbidden`	Доступ запрещён
`404 Not Found`	Страница не найдена
`500 Internal Server Error`	Ошибка на сервере (например, в PHP-коде)

Статика и динамика

- **Статический контент** — файлы, которые отдаются как есть: HTML, CSS, JavaScript, картинки. Nginx отлично справляется с этим сам.
- **Динамический контент** — страницы, которые генерируются программой (PHP, Python и т.д.) на основе данных. Для WordPress каждая страница — результат выполнения PHP-кода, который читает данные из базы и собирает HTML.

Установка nginx

Устанавливаем и проверяем

```
sudo apt update
sudo apt install nginx
```

nginx запускается автоматически после установки. Проверяем:

```
systemctl status nginx
```

Вы должны увидеть `active (running)`. Теперь проверим, что веб-сервер отвечает:

```
curl http://localhost
```

Вы увидите HTML-код стартовой страницы nginx: «Welcome to nginx!».

Проверим, что nginx слушает порт 80:

```
sudo ss -tlnp | grep :80
```

Структура каталогов nginx

Каталог / файл	Назначение
`/etc/nginx/nginx.conf`	Главный конфигурационный файл
`/etc/nginx/sites-available/`	Доступные конфигурации сайтов
`/etc/nginx/sites-enabled/`	Включённые конфигурации (симлинки)
`/var/www/html/`	Каталог по умолчанию для файлов сайта
`/var/log/nginx/`	Логи (access.log, error.log)

Механизм работы: в `sites-available/` лежат файлы конфигурации сайтов, а в `sites-enabled/` — символические ссылки на те из них, которые должны быть активны. Это позволяет быстро включать и отключать сайты без удаления конфигурации.

Настройка серверного блока для WordPress

Создаём каталог для сайта

```
sudo mkdir -p /var/www/wordpress
sudo chown -R webmaster:webdev /var/www/wordpress
```

Мы используем пользователя `webmaster` и группу `webdev` из главы 08.

Создаём конфигурацию сайта

Создадим файл конфигурации:

```
sudo nano /etc/nginx/sites-available/wordpress
```

Содержимое:

```
server {
    listen 80;
    server_name _;

    root /var/www/wordpress;
    index index.php index.html;

    location / {
        try_files $uri $uri/ /index.php?$args;
    }

    location ~ \.php$ {
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:/run/php/php-fpm.sock;
    }

    location ~ /\.ht {
        deny all;
    }
}
```

Разберём по частям:

Директива	Значение
`listen 80`	Слушать порт 80 (HTTP)
`server_name _`	Отвечать на любое имя хоста
`root /var/www/wordpress`	Корневой каталог сайта
`index index.php index.html`	Файлы по умолчанию (в порядке приоритета)

Блоки `location` определяют, как обрабатывать разные запросы:

- **`location /`** — для всех запросов: сначала ищем файл (`$uri`), потом каталог (`$uri/`), если ничего не найдено — передаём в `index.php` (так работают «красивые» URL WordPress).
- **`location ~ \.php$`** — для PHP-файлов: передаём запрос в PHP-FPM через Unix-сокет.
- **`location ~ /\.ht`** — запрещаем доступ к файлам `.htaccess` и `.htpasswd` (они не нужны в nginx, но WordPress может их создать).

Включаем сайт

Создаём символическую ссылку в `sites-enabled` и убираем дефолтный сайт:

```
sudo ln -s /etc/nginx/sites-available/wordpress /etc/nginx/sites-enabled/
sudo rm /etc/nginx/sites-enabled/default
```

Проверяем конфигурацию на ошибки:

```
sudo nginx -t
```

```
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

Перезагружаем nginx:

```
sudo systemctl reload nginx
```

Установка PHP-FPM

Устанавливаем PHP и необходимые модули

```
sudo apt install php-fpm php-mysql php-xml php-mbstring php-curl php-gd
↪  php-intl php-zip
```

Пакет	Зачем нужен
`php-fpm`	Обработчик PHP (FastCGI Process Manager)
`php-mysql`	Подключение к MariaDB
`php-xml`	Работа с XML (нужен WordPress)
`php-mbstring`	Поддержка многобайтовых строк (кириллица)
`php-curl`	HTTP-запросы из PHP
`php-gd`	Обработка изображений
`php-intl`	Интернационализация
`php-zip`	Работа с ZIP-архивами

Проверяем, что PHP-FPM запущен:

```
systemctl status php*-fpm
```

Как nginx взаимодействует с PHP-FPM

nginx и PHP-FPM общаются через **Unix-сокет** — специальный файл в файловой системе, который работает как канал связи между программами:

```
/run/php/php-fpm.sock
```

Когда nginx получает запрос к `.php`-файлу, он передаёт его через этот сокет в PHP-FPM. PHP-FPM выполняет скрипт и возвращает результат обратно в nginx, который отправляет его клиенту.

Почему сокет, а не сетевой порт? Unix-сокет быстрее TCP-порта, потому что данные не проходят через сетевой стек. Это имеет смысл, когда nginx и PHP-FPM работают на одной машине.

Проверяем связку nginx + PHP-FPM

Создадим тестовый PHP-файл:

```
echo '<?php phpinfo(); ?>' | sudo tee /var/www/wordpress/info.php
```

Проверяем:

```
curl http://localhost/info.php
```

Если вы видите длинный HTML-вывод с информацией о PHP — всё работает. Если видите текст `<?php phpinfo(); ?>` — значит, nginx не передаёт запрос в PHP-FPM (проверьте конфигурацию).

После проверки **обязательно удалите** этот файл — он раскрывает информацию о сервере:

```
sudo rm /var/www/wordpress/info.php
```

Установка MariaDB

Устанавливаем и защищаем

```
sudo apt install mariadb-server
```

MariaDB запустится автоматически. Теперь запустим скрипт начальной настройки безопасности:

```
sudo mysql_secure_installation
```

Скрипт задаст несколько вопросов:

Вопрос	Рекомендуемый ответ
Enter current password for root	Нажмите Enter (пароля нет)
Switch to unix_socket authentication	n (уже настроено)
Change the root password	n (для MariaDB root используется unix-сокет)
Remove anonymous users	Y
Disallow root login remotely	Y
Remove test database	Y
Reload privilege tables	Y

Создаём базу данных и пользователя

Подключаемся к MariaDB от root:

```
sudo mariadb
```

Создаём базу данных, пользователя и даём права:

```
CREATE DATABASE wordpress CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'wpuser'@'localhost' IDENTIFIED BY 'SecurePassword123';
GRANT ALL PRIVILEGES ON wordpress.* TO 'wpuser'@'localhost';
FLUSH PRIVILEGES;
EXIT;
```

> **Важно:** замените `SecurePassword123` на надёжный пароль. Запишите его — он понадобится при настройке WordPress.

Проверяем, что подключение работает:

```
mariadb -u wpuser -p wordpress
```

Введите пароль. Если вы увидели приглашение `MariaDB [wordpress]>` — всё работает. Наберите `EXIT;` для выхода.

Установка WordPress

Скачиваем и распаковываем

```
cd /tmp
curl -LO https://wordpress.org/latest.tar.gz
sudo tar -xzf latest.tar.gz -C /var/www/
```

Команда `tar -xzf` распакует архив, и в `/var/www/` появится каталог `wordpress` с файлами CMS. Мы указали именно этот путь, потому что наш серверный блок nginx настроен на `root /var/www/wordpress`.

> **Если каталог /var/www/wordpress уже существует** (мы создали его ранее для тестового `info.php`), `tar` распакует файлы прямо в него. Если вы хотите начать с чистого каталога, предварительно удалите его: `sudo rm -rf /var/www/wordpress`.

Настраиваем права

```
sudo chown -R webmaster:www-data /var/www/wordpress
sudo find /var/www/wordpress -type d -exec chmod 755 {} \;
sudo find /var/www/wordpress -type f -exec chmod 644 {} \;
```

Здесь:

- **Владелец** — `webmaster` (может редактировать файлы).
- **Группа** — `www-data` (под этим пользователем работает PHP-FPM, ему нужен доступ на чтение).
- **Каталоги** — `755` (rwxr-xr-x): владелец может всё, остальные только читают и входят.
- **Файлы** — `644` (rw-r–r–): владелец читает и пишет, остальные только читают.

WordPress нужно разрешить запись в некоторые каталоги (для загрузки медиафайлов и обновлений):

```
sudo chmod -R g+w /var/www/wordpress/wp-content
```

Настраиваем wp-config.php

WordPress хранит настройки подключения к базе данных в файле `wp-config.php`. Создадим его из шаблона:

```
cd /var/www/wordpress
sudo cp wp-config-sample.php wp-config.php
sudo nano wp-config.php
```

Найдите и измените строки:

```
define( 'DB_NAME', 'wordpress' );
define( 'DB_USER', 'wpuser' );
define( 'DB_PASSWORD', 'SecurePassword123' );
define( 'DB_HOST', 'localhost' );
```

Укажите те имя базы, пользователя и пароль, которые вы создали в MariaDB.

Также найдите блок с «солями» (Authentication Unique Keys and Salts) и замените его. Сгенерировать уникальные значения можно командой:

```
curl -s https://api.wordpress.org/secret-key/1.1/salt/
```

Скопируйте вывод и вставьте в `wp-config.php` вместо строк-заглушек.

Завершаем установку через браузер

Если вы работаете на виртуальной машине, откройте в браузере на своём компьютере:

```
http://IP-адрес-сервера
```

Где `IP-адрес-сервера` — адрес, который вы узнали в главе 10 через `ip addr show`.

Если всё настроено правильно, вы увидите мастер установки WordPress. Выберите язык, задайте название сайта, имя администратора и пароль.

> **Нет доступа к браузеру?** Если вы работаете только через терминал, можно проверить, что WordPress отвечает, с помощью curl:
>
> ```
> curl -I http://localhost
> ```
>
> Код `200` или `302` (перенаправление на установку) означает, что WordPress работает.

Проверка всего стека

После установки убедимся, что все компоненты работают:

```
# Статус сервисов
systemctl status nginx
systemctl status php*-fpm
systemctl status mariadb

# Открытые порты
sudo ss -tlnp

# Ответ от сайта
curl -I http://localhost

# Логи (если что-то не работает)
journalctl -u nginx --since "10 minutes ago"
journalctl -u php*-fpm --since "10 minutes ago"
```

Вы должны увидеть:

- nginx, PHP-FPM и MariaDB — `active (running)`
- Порты 80 (nginx), 3306 (MariaDB) — в списке `ss`
- `curl` возвращает `HTTP/1.1 200 OK` или `HTTP/1.1 302 Found`

Справочник команд

nginx

Команда	Описание
`sudo apt install nginx`	Установить nginx
`sudo nginx -t`	Проверить конфигурацию
`sudo systemctl reload nginx`	Перечитать конфигурацию
`sudo ln -s /etc/nginx/sites-available/сайт /etc/nginx/sites-enabled/`	Включить сайт

PHP-FPM

Команда	Описание
`sudo apt install php-fpm php-mysql ...`	Установить PHP-FPM и модули
`systemctl status php*-fpm`	Статус PHP-FPM
`php -v`	Версия PHP

MariaDB

Команда	Описание
`sudo apt install mariadb-server`	Установить MariaDB
`sudo mysql_secure_installation`	Начальная настройка безопасности
`sudo mariadb`	Подключиться к MariaDB от root
`mariadb -u пользователь -p база`	Подключиться под указанным пользователем

WordPress

Команда	Описание
`curl -LO https://wordpress.org/latest.tar.gz`	Скачать WordPress
`sudo tar -xzf latest.tar.gz -C /var/www/`	Распаковать в /var/www/
`curl -s https://api.wordpress.org/secret-key/1.1/salt/`	Сгенерировать соли

Практические задания

1. Выполните `curl -I http://localhost`. Какой код ответа вы получили? Какой сервер указан в заголовке `Server`?
2. Посмотрите лог доступа nginx: `sudo tail -5 /var/log/nginx/access.log`. Найдите свой запрос от `curl`. Какой метод, URL и код ответа записаны в логе?
3. Выполните `sudo ss -tlnp`. Найдите все процессы, связанные с нашим стеком: nginx (порт 80), MariaDB (порт 3306). На каких адресах они слушают?
4. Посмотрите unit-файл nginx: `systemctl cat nginx`. Найдите директиву `ExecStart`. Сравните с тем, что мы разбирали в главе 09 для ssh.service — какие общие элементы?
5. Подключитесь к MariaDB (`sudo mariadb`) и выполните `SHOW DATABASES;`. Видна ли база `wordpress`? Выполните `USE wordpress; SHOW TABLES;` — какие таблицы создал WordPress?
6. Остановите PHP-FPM: `sudo systemctl stop php*-fpm`. Попробуйте открыть сайт через `curl http://localhost`. Какую ошибку вы получите? Запустите PHP-FPM обратно и убедитесь, что сайт снова работает.
7. Посмотрите файл `/etc/nginx/sites-available/wordpress`. Измените директиву `index`, убрав `index.php` из списка. Выполните `sudo nginx -t` и `sudo systemctl reload nginx`. Что произойдёт при обращении к `http://localhost`? Верните настройку обратно.

Итоги

В этой главе вы:

- разобрались с клиент-серверной моделью, HTTP-запросами и ответами;
- установили и настроили nginx — создали серверный блок для WordPress;
- установили PHP-FPM и связали его с nginx через Unix-сокет;
- установили MariaDB, создали базу данных и пользователя;
- скачали и настроили WordPress, прошли мастер установки;
- проверили работу всего стека: nginx + PHP-FPM + WordPress + MariaDB;
- **развернули полноценный веб-сайт** — главную цель сквозного проекта.

В следующей главе мы разберём **логирование и мониторинг** — научимся анализировать логи nginx и системные журналы, чтобы находить проблемы и следить за работой сервера.

Этап 12. Логирование и мониторинг

Зачем следить за логами

Сервер работает круглосуточно, и вы не можете наблюдать за ним каждую секунду. Логи — это глаза и уши администратора. Они фиксируют всё, что происходит: кто обратился к сайту, какие ошибки возникли, почему сервис не запустился.

Без логов невозможно:

- понять, почему пользователь видит ошибку 500;
- найти, кто сканирует сервер в поисках уязвимостей;
- выяснить, когда и почему «упал» сайт;
- оценить нагрузку — сколько запросов в час получает сервер.

> **Привязка к проекту.** В предыдущих главах мы развернули WordPress на nginx + PHP-FPM + MariaDB. Теперь научимся анализировать логи работающего сайта — инструментами из главы 05 (grep, awk, sort, uniq, wc), но уже на реальных данных.

Системный журнал: journalctl (расширение)

В главе 09 мы познакомились с `journalctl` и научились смотреть логи конкретного сервиса (`-u`), следить в реальном времени (`-f`) и фильтровать по времени (`--since`). Теперь расширим эти знания.

Приоритеты сообщений

Каждое сообщение в журнале имеет **приоритет** — уровень важности. Можно фильтровать по нему, чтобы видеть только ошибки и не тонуть в информационных сообщениях:

```
journalctl -p err                    # только ошибки и критичнее
journalctl -p warning                # предупреждения и критичнее
journalctl -p err --since today      # ошибки за сегодня
```

Таблица приоритетов (от самого серьёзного к информационному):

Приоритет	Номер	Описание
`emerg`	0	Система неработоспособна
`alert`	1	Требуется немедленное вмешательство
`crit`	2	Критическая ошибка

Приоритет	Номер	Описание
err	3	Ошибка
warning	4	Предупреждение
notice	5	Важное информационное сообщение
info	6	Информационное сообщение
debug	7	Отладочная информация

Флаг `-p err` покажет сообщения с приоритетами 0–3 (от `emerg` до `err`). Это самый полезный фильтр для поиска проблем.

Логи текущей загрузки

```
journalctl -b                        # логи с момента загрузки
journalctl -b -p err                 # ошибки с момента загрузки
```

Если сервер недавно перезагружался и что-то сломалось, `-b` покажет только сообщения текущей сессии — без «хвоста» от прошлых загрузок.

Размер журнала и ротация

Журнал systemd растёт со временем и может занять значительную часть диска. Проверим, сколько места он занимает:

```
journalctl --disk-usage
```

```
Archived and active journals take up 120.5M in the file system.
```

Если журнал стал слишком большим, можно ограничить его размер или возраст:

```
sudo journalctl --vacuum-size=500M   # оставить не более 500 МБ
sudo journalctl --vacuum-time=7d     # удалить записи старше 7 дней
```

Для постоянного ограничения размера отредактируйте файл `/etc/systemd/journald.conf` и задайте параметр:

```
SystemMaxUse=500M
```

Логи nginx

nginx ведёт свои собственные логи, отдельно от системного журнала. Это текстовые файлы, которые можно анализировать утилитами из главы 05.

Где лежат логи

Файл	Содержание
`/var/log/nginx/access.log`	Каждый HTTP-запрос к серверу
`/var/log/nginx/error.log`	Ошибки nginx и PHP-FPM

Формат access.log

По умолчанию nginx использует формат **combined**. Каждая строка — один запрос:

```
192.168.1.50 - - [17/Mar/2026:10:15:32 +0300] "GET /wp-login.php HTTP/1.1" 200
↪  4523 "-" "Mozilla/5.0 (Windows NT 10.0; Win64; x64)"
```

Разберём по полям:

#	Поле	Пример	Значение
1	IP-адрес клиента	`192.168.1.50`	Кто обратился
2	Идентификатор	`-`	Обычно пустой
3	Пользователь	`-`	Имя (при HTTP-авторизации)
4	Дата и время	`[17/Mar/2026:10:15:32 +0300]`	Когда
5	Запрос	`"GET /wp-login.php HTTP/1.1"`	Метод, URL, протокол
6	Код ответа	`200`	Результат (200, 404, 500…)
7	Размер ответа	`4523`	В байтах
8	Referrer	`"-"`	Откуда пришёл пользователь
9	User-Agent	`"Mozilla/5.0 ..."`	Браузер или бот

Знание формата — ключ к анализу. Поля разделены пробелами, а дата, запрос, referrer и user-agent заключены в кавычки или квадратные скобки.

Формат error.log

Ошибки выглядят иначе:

```
2026/03/17 10:20:45 [error] 1234#1234: *5 open()
↪  "/var/www/wordpress/favicon.ico" failed (2: No such file or directory),
↪  client: 192.168.1.50, server: _, request: "GET /favicon.ico HTTP/1.1"
```

Здесь видно: дату, уровень ошибки (`[error]`), описание проблемы (файл не найден) и детали запроса.

Анализ access.log с помощью coreutils

Вот где пригодятся навыки из главы 05. Мы будем анализировать реальные логи работающего WordPress, используя конвейеры из `awk`, `sort`, `uniq`, `grep` и `wc`.

Все примеры работают с файлом `/var/log/nginx/access.log`. Для чтения этого файла может потребоваться `sudo`.

Сколько запросов получил сервер

```
sudo wc -l /var/log/nginx/access.log
```

Топ IP-адресов (кто чаще всего обращается)

```
sudo awk '{print $1}' /var/log/nginx/access.log | sort | uniq -c | sort -rn |
↪  head
```

Разбор конвейера:

1. `awk '{print $1}'` — извлечь первое поле (IP-адрес)
2. `sort` — отсортировать (нужно для `uniq`)
3. `uniq -c` — подсчитать повторения
4. `sort -rn` — отсортировать по убыванию числа
5. `head` — показать первые 10

Результат:

```
  342 192.168.1.50
  128 10.0.0.5
   45 192.168.1.1
```

Если один IP делает тысячи запросов — возможно, это бот или атака.

Распределение по кодам ответа

```
sudo awk '{print $9}' /var/log/nginx/access.log | sort | uniq -c | sort -rn
```

Девятое поле ($9) — код ответа. Результат:

```
  412 200
   23 301
   15 404
    3 500
```

Много **404** — возможно, сломаны ссылки или кто-то сканирует сервер. **500** — ошибки в PHP-коде или проблемы с базой данных.

Найти все ошибки 404 (страница не найдена)

```
sudo awk '$9 == 404 {print $7}' /var/log/nginx/access.log | sort | uniq -c |
↪  sort -rn
```

Здесь `$9 == 404` фильтрует строки по коду, а `$7` — URL запроса. Вы увидите, какие несуществующие страницы запрашивают чаще всего.

Найти серверные ошибки (500)

```
sudo grep '" 500 ' /var/log/nginx/access.log
```

Каждая строка с кодом 500 — повод разобраться. Посмотрите также error.log — там будет описание причины.

Самые популярные страницы

```
sudo awk '{print $7}' /var/log/nginx/access.log | sort | uniq -c | sort -rn |
↪  head
```

Запросы за последний час

```
sudo awk -v d="$(date -d '1 hour ago' '+%d/%b/%Y:%H')" '$4 ~ d'
↪  /var/log/nginx/access.log | wc -l
```

Этот конвейер формирует строку с датой часовой давности и фильтрует лог по ней. Так можно отслеживать активность по часам.

Найти ботов

```
sudo grep -i 'bot\|crawl\|spider' /var/log/nginx/access.log | awk '{print $1}'
↪  | sort -u
```

Покажет IP-адреса поисковых ботов (Googlebot, Bingbot и т.д.).

Анализ error.log

Error.log содержит ошибки nginx, PHP-FPM и связанных компонентов. Типичные проблемы:

Ошибка	Причина
`No such file or directory`	Запрошенный файл не существует (404)
`Permission denied`	Нет прав на чтение файла
`upstream timed out`	PHP-FPM не ответил вовремя
`connect() failed`	PHP-FPM не запущен или сокет недоступен

Посмотреть последние ошибки:

```
sudo tail -20 /var/log/nginx/error.log
```

Найти ошибки определённого типа:

```
sudo grep 'Permission denied' /var/log/nginx/error.log
sudo grep 'upstream' /var/log/nginx/error.log
```

> **Совет.** Если сайт показывает ошибку 502 Bad Gateway, первое, что нужно проверить — работает ли PHP-FPM (`systemctl status php*-fpm`) и нет ли ошибок про `upstream` в error.log.

Ротация логов (logrotate)

Зачем нужна ротация

Логи растут непрерывно. Если access.log получает 10 000 записей в день, через год он будет весить несколько гигабайт. **Ротация логов** решает эту проблему: старый лог архивируется (сжимается), создаётся новый пустой файл, а самые старые архивы удаляются.

В Debian за это отвечает утилита **logrotate**, которая запускается автоматически (обычно раз в день через systemd-таймер или cron).

Конфигурация для nginx

```
cat /etc/logrotate.d/nginx
```

```
/var/log/nginx/*.log {
    daily
    missingok
    rotate 14
    compress
    delaycompress
    notifempty
    create 0640 www-data adm
    sharedscripts
    postrotate
        if [ -d /etc/nginx ]; then
            invoke-rc.d nginx rotate >/dev/null 2>&1
        fi
    endscript
}
```

Основные директивы:

Директива	Значение
`daily`	Ротация раз в день
`rotate 14`	Хранить 14 архивов (2 недели)
`compress`	Сжимать старые логи (gzip)
`delaycompress`	Не сжимать самый свежий архив
`notifempty`	Не ротировать пустые файлы
`create 0640 www-data adm`	Создать новый файл с указанными правами
`postrotate`	Команда после ротации (сигнал nginx перечитать файлы)

После ротации в `/var/log/nginx/` вы увидите файлы вроде:

```
access.log            — текущий лог
access.log.1          — вчерашний (ещё не сжат)
access.log.2.gz       — позавчерашний (сжат)
access.log.3.gz       — три дня назад
...
```

Практика: анализируем логи WordPress

Шаг 1. Смотрим системный журнал

Проверим, нет ли ошибок в сервисах нашего стека:

```
journalctl -u nginx -p err --since today
journalctl -u php*-fpm -p err --since today
journalctl -u mariadb -p err --since today
```

Если вывод пуст — ошибок нет, всё работает.

Шаг 2. Генерируем трафик для анализа

Чтобы в логах было что анализировать, сделаем несколько запросов:

```
curl -s http://localhost > /dev/null
curl -s http://localhost/wp-login.php > /dev/null
curl -s http://localhost/not-found-page > /dev/null
curl -s http://localhost/readme.html > /dev/null
```

Шаг 3. Анализируем access.log

```
# Сколько запросов
sudo wc -l /var/log/nginx/access.log

# Топ запрашиваемых URL
sudo awk '{print $7}' /var/log/nginx/access.log | sort | uniq -c | sort -rn |
 ↪  head

# Коды ответов
sudo awk '{print $9}' /var/log/nginx/access.log | sort | uniq -c | sort -rn

# Есть ли 404?
sudo awk '$9 == 404 {print $7}' /var/log/nginx/access.log
```

Шаг 4. Проверяем error.log

```
sudo tail -10 /var/log/nginx/error.log
```

Вы должны увидеть ошибку для `/not-found-page` — nginx не нашёл файл и вернул 404.

Шаг 5. Проверяем ротацию

```
ls -la /var/log/nginx/
cat /etc/logrotate.d/nginx
```

Справочник команд

journalctl (расширение)

Команда	Описание
`journalctl -p err`	Только ошибки и критичнее
`journalctl -p warning`	Предупреждения и критичнее
`journalctl -b`	Логи с момента загрузки
`journalctl --disk-usage`	Размер журнала на диске
`sudo journalctl --vacuum-size=500M`	Ограничить размер 500 МБ
`sudo journalctl --vacuum-time=7d`	Удалить записи старше 7 дней

Анализ логов nginx

Команда	Описание
`sudo wc -l /var/log/nginx/access.log`	Количество запросов
`sudo awk '{print $1}' access.log | sort | uniq -c | sort -rn | head`	Топ IP-адресов
`sudo awk '{print $9}' access.log | sort | uniq -c | sort -rn`	Распределение по кодам
`sudo awk '$9 == 404 {print $7}' access.log`	Запросы с ошибкой 404
`sudo awk '{print $7}' access.log | sort | uniq -c | sort -rn | head`	Популярные страницы
`sudo tail -20 /var/log/nginx/error.log`	Последние ошибки

Практические задания

1. Выполните `journalctl -p err --since today`. Есть ли ошибки в системе? Если есть — от каких сервисов?
2. Проверьте размер системного журнала (`journalctl --disk-usage`). Если он больше 500 МБ, выполните `sudo journalctl --vacuum-size=500M`.
3. Посмотрите последние 20 строк `/var/log/nginx/access.log`. Определите для каждой строки: IP клиента, запрошенный URL и код ответа.
4. Подсчитайте количество запросов в access.log. Выведите топ-5 IP-адресов по количеству обращений. Есть ли подозрительно активные?
5. Выведите распределение кодов ответа. Какой процент запросов завершился кодом 200? Есть ли коды 404 или 500?
6. Найдите все запросы к `/wp-login.php` в access.log. С каких IP-адресов они приходят? (Подсказка: `grep 'wp-login' access.log | awk '{print $1}' | sort | uniq -c`.)
7. Посмотрите error.log. Найдите ошибки типа `No such file or directory`. Какие файлы запрашивают, но не находят?

8. Посмотрите конфигурацию ротации логов nginx (`cat /etc/logrotate.d/nginx`). Сколько архивов хранится? Как часто происходит ротация? Есть ли уже ротированные файлы в `/var/log/nginx/`?

Итоги

В этой главе вы:

- расширили знания о `journalctl`: приоритеты сообщений, фильтрация ошибок, управление размером журнала;
- разобрали формат логов nginx (access.log и error.log);
- научились анализировать access.log с помощью coreutils: находить самые активные IP, популярные страницы, коды ошибок;
- освоили диагностику по error.log: поиск ошибок прав доступа, отсутствующих файлов и проблем с PHP-FPM;
- познакомились с ротацией логов (logrotate).

Часть II завершена. Вы прошли путь от установки пакетов до полноценного веб-сервера с мониторингом. В следующей части мы перейдём к **основам DevOps** — начнём с **Git** (контроль версий), а затем освоим Docker, Docker Compose и базовый CI/CD.

Этап 13. Git: основы контроля версий

Зачем нужен контроль версий

Представьте ситуацию: вы отредактировали `nginx.conf`, перезапустили nginx — и сайт перестал работать. Что было в файле до правки? Если вы не сделали копию — остаётся только вспоминать. Или другой знакомый сценарий из мира Windows:

```
backup.sh
backup_old.sh
backup_v2.sh
backup_v2_final.sh
backup_v2_final_FINAL.sh
```

Какой из этих файлов рабочий? Когда и что именно менялось? Кто менял? На эти вопросы невозможно ответить, глядя на имена файлов.

Система контроля версий решает все эти проблемы. Она хранит полную историю изменений каждого файла: кто, когда и зачем внёс правку. В любой момент можно посмотреть, что изменилось, сравнить версии и откатиться к любому состоянию проекта.

Git — самая популярная система контроля версий в мире. Её используют и разработчики, и системные администраторы, и DevOps-инженеры. Git создал Линус Торвальдс в 2005 году — тот самый человек, который создал ядро Linux.

> **Привязка к проекту.** В предыдущих главах мы собрали wordpress-project: конфиги nginx и PHP, скрипт резервного копирования, файлы сайта. Сейчас всё это лежит «просто на диске» без какой-либо истории. В этой главе мы поставим проект под контроль версий Git — и каждое изменение будет задокументировано.

Установка и настройка

Установка

Git входит в стандартные репозитории Debian:

```
sudo apt update
sudo apt install git
```

Проверим версию:

```
git --version
```

```
git version 2.47.2
```

Первоначальная настройка

Перед началом работы Git нужно указать ваше имя и email. Эти данные записываются в каждый коммит (фиксацию изменений), чтобы было понятно, кто автор:

```
git config --global user.name "Ivan Petrov"
git config --global user.email "ivan@example.com"
```

Флаг `--global` означает, что настройки действуют для всех репозиториев текущего пользователя. Они сохраняются в файле `~/.gitconfig`.

Проверим, что настройки применились:

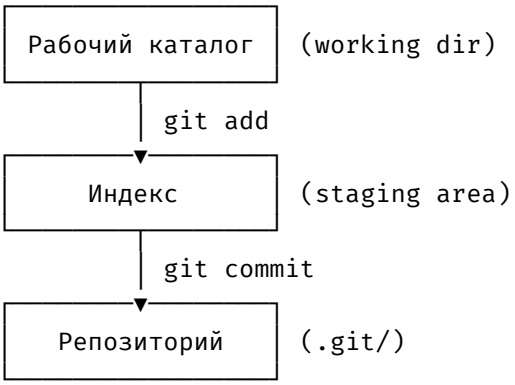
```
git config --list
```

```
user.name=Ivan Petrov
user.email=ivan@example.com
```

> **Совет.** Укажите тот же email, который будете использовать при регистрации на GitLab — тогда платформа автоматически свяжет коммиты с вашим аккаунтом.

Три зоны Git

Прежде чем перейти к командам, разберёмся, как Git устроен изнутри. Любой проект под управлением Git имеет три зоны:



- **Рабочий каталог** — файлы на диске, которые вы редактируете.
- **Индекс** (staging area, область подготовки) — «черновик» будущего коммита. Сюда вы добавляете только те изменения, которые хотите зафиксировать.
- **Репозиторий** (.git/) — база данных всех коммитов. Хранится в скрытом каталоге `.git` внутри проекта.

Типичный рабочий цикл:

1. Редактируете файлы в рабочем каталоге.
2. `git add` — переносите нужные изменения в индекс.
3. `git commit` — фиксируете содержимое индекса как новый коммит.

Зачем нужен промежуточный индекс? Он позволяет зафиксировать не все изменения сразу, а только те, которые относятся к одной задаче. Например, вы исправили баг в `nginx.conf` и одновременно добавили комментарии в `backup.sh`. Это два разных изменения — их лучше оформить двумя отдельными коммитами.

Первый репозиторий

Перейдём в каталог нашего проекта и создадим репозиторий:

```
cd ~/wordpress-project
git init
```

```
Initialized empty Git repository in /home/user/wordpress-project/.git/
```

Git создал скрытый каталог `.git/` — именно в нём хранится вся история. Сам проект при этом не изменился.

git status — текущее состояние

Первая команда, которую стоит запомнить:

```
git status
```

```
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        backup.sh
        config/
        hello.sh
        logs/
        site/

nothing added to commit but untracked files present (use "git add" to track)
```

Git видит все файлы, но пока не отслеживает ни один из них («Untracked files»). Прежде чем добавлять файлы, разберёмся, что **не нужно** хранить в репозитории.

Что не хранить в репозитории: .gitignore

Не все файлы проекта должны попадать в Git:

- **Логи** (`logs/`) — постоянно растут, засоряют историю и не несут ценности для контроля версий.
- **Временные файлы** (`*.swp`, `*.tmp`) — создаются редакторами vim и nano, не имеют отношения к проекту.
- **Файлы с секретами** — пароли, токены, приватные ключи. Если они попадут в удалённый репозиторий, их увидят другие люди.

Для этого в корне проекта создаётся файл `.gitignore`:

```
nano .gitignore
```

```
# Логи
logs/

# Временные файлы редакторов
*.swp
*.tmp
*~

# Архивы резервных копий (создаются backup.sh)
*.tar.gz
```

Проверим, что `.gitignore` работает:

```
git status
```

```
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore
        backup.sh
        config/
        hello.sh
        site/

nothing added to commit but untracked files present (use "git add" to track)
```

Каталог `logs/` исчез из списка — Git теперь его игнорирует.

Синтаксис .gitignore

Шаблон	Что игнорирует
`logs/`	Каталог `logs` и всё его содержимое
`*.swp`	Все файлы с расширением `.swp`
`*.log`	Все файлы с расширением `.log`
`build/`	Каталог `build`
`!important.log`	Исключение: **не** игнорировать `important.log`
`#`	Комментарий
`**/temp`	Каталог `temp` на любом уровне вложенности

Важно. Если файл уже был добавлен в Git до того, как вы вписали его в `.gitignore`, Git продолжит его отслеживать. Чтобы убрать такой файл из-под контроля версий (не удаляя с диска), используйте:

```
git rm --cached <файл>
```

Первый коммит

Теперь добавим все нужные файлы и сделаем первый коммит:

```
git add .
```

Точка (`.`) означает «добавить всё из текущего каталога» (с учётом `.gitignore`). Можно добавлять файлы по одному:

```
git add backup.sh
git add config/nginx.conf
```

Посмотрим, что попало в индекс:

```
git status
```

```
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   .gitignore
        new file:   backup.sh
        new file:   config/nginx.conf
        new file:   config/php.ini
        new file:   hello.sh
        new file:   site/index.html
```

Всё верно — логи не попали. Фиксируем:

```
git commit -m "Начальный коммит: структура проекта wordpress"
```

```
[main (root-commit) a1b2c3d] Начальный коммит: структура проекта wordpress
 6 files changed, 42 insertions(+)
```

```
 create mode 100644 .gitignore
 create mode 100755 backup.sh
 create mode 100644 config/nginx.conf
 create mode 100644 config/php.ini
 create mode 100755 hello.sh
 create mode 100644 site/index.html
```

Флаг `-m` задаёт **сообщение коммита** — короткое описание того, что и зачем вы сделали. Хорошее сообщение отвечает на вопрос «что изменилось и почему», а не просто «что сделано»:

Плохо	Хорошо
`fix`	`Исправлен путь к сокету PHP-FPM в конфиге nginx`
`update`	`Добавлена ротация логов в backup.sh`
`asdfg`	`Начальный коммит: структура проекта`

git log — история коммитов

```
git log
```

```
commit a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0 (HEAD -> main)
Author: Ivan Petrov <ivan@example.com>
Date:   Mon Mar 17 10:00:00 2026 +0300

    Начальный коммит: структура проекта wordpress
```

Каждый коммит имеет:

- **Хеш** (a1b2c3d…) — уникальный идентификатор, вычисленный по содержимому. Не нужно запоминать весь хеш — достаточно первых 7–8 символов.
- **Автор** — имя и email из `git config`.
- **Дата** — когда был сделан коммит.
- **Сообщение** — описание изменений.

Компактный формат:

```
git log --oneline
```

```
a1b2c3d Начальный коммит: структура проекта wordpress
```

git diff — что изменилось

Отредактируем конфигурацию nginx — добавим комментарий:

```
nano config/nginx.conf
```

Добавьте в начало файла строку:

```
# Конфигурация nginx для WordPress
```

Теперь посмотрим, что изменилось:

```
git diff
```

```
diff --git a/config/nginx.conf b/config/nginx.conf
index 1234567..abcdefg 100644
--- a/config/nginx.conf
+++ b/config/nginx.conf
@@ -1,3 +1,4 @@
+# Конфигурация nginx для WordPress
 server {
     listen 80;
     server_name example.com;
```

Строки с + — добавленные, с - — удалённые. `git diff` показывает разницу между рабочим каталогом и индексом (то есть ещё не подготовленные изменения).

Зафиксируем это изменение:

```
git add config/nginx.conf
git commit -m "Добавлен комментарий в конфигурацию nginx"
```

Посмотрим историю:

```
git log --oneline
```

```
b2c3d4e Добавлен комментарий в конфигурацию nginx
a1b2c3d Начальный коммит: структура проекта wordpress
```

Теперь в истории два коммита. В любой момент можно вернуться к первому.

Справочник основных команд

Команда	Описание
`git init`	Создать новый репозиторий в текущем каталоге
`git status`	Показать текущее состояние (что изменено, что в индексе)
`git add <файл>`	Добавить файл в индекс
`git add .`	Добавить все изменения в индексе
`git commit -m "..."`	Зафиксировать изменения из индекса
`git log`	Показать историю коммитов
`git log --oneline`	Компактная история (одна строка на коммит)
`git diff`	Показать незафиксированные изменения
`git diff --staged`	Показать изменения, добавленные в индекс
`git show <хеш>`	Показать содержимое конкретного коммита

Ветвление и слияние

До сих пор мы работали линейно: один коммит за другим в одной ветке `main`. Но в реальных проектах часто нужно работать над несколькими задачами параллельно, не мешая друг другу.

Ветка (branch) — это отдельная линия разработки. Вы можете создать ветку, вносить в ней любые изменения, а когда работа готова — слить результат обратно в основную ветку.

```
main:     A ── B ── C ──────────── F (merge)
                \               /
feature:         D ── E ──────
```

- В точке B создаётся ветка `feature`.
- Коммиты D и E делаются в ветке `feature` — основная ветка `main` при этом не затрагивается.
- В точке F ветка `feature` сливается обратно в `main`.

Создание ветки

Допустим, мы хотим улучшить скрипт `backup.sh` — добавить вывод даты и размера архива. Создадим для этого отдельную ветку:

```
git branch feature-backup
```

Посмотрим список веток:

```
git branch
```

```
  feature-backup
* main
```

Звёздочка (`*`) показывает текущую ветку. Переключимся на новую:

```
git switch feature-backup
```

```
Switched to branch 'feature-backup'
```

> **Примечание.** Команда `git switch` появилась в Git 2.23. В старых руководствах вы встретите `git checkout <ветка>` — она делает то же самое, но `switch` понятнее по смыслу. Можно создать ветку и сразу переключиться на неё одной командой: `git switch -c feature-backup`.

Работа в ветке

Отредактируем `backup.sh` — добавим вывод информации об архиве:

```
nano backup.sh
```

Добавьте в конец скрипта (перед завершением):

```
# Выводим информацию об архиве
echo "Дата: $(date '+%Y-%m-%d %H:%M')"
echo "Размер: $(du -h "$ARCHIVE" | cut -f1)"
```

Зафиксируем изменение:

```
git add backup.sh
git commit -m "Добавлен вывод даты и размера архива в backup.sh"
```

Слияние

Работа в ветке завершена. Переключимся обратно на `main` и сольём изменения:

```
git switch main
git merge feature-backup
```

```
Updating b2c3d4e..c3d4e5f
Fast-forward
 backup.sh | 3 +++
 1 file changed, 3 insertions(+)
```

Git выполнил **fast-forward** (перемотку вперёд) — просто передвинул указатель `main` на последний коммит ветки `feature-backup`, потому что в `main` не было новых коммитов после создания ветки.

Ветка больше не нужна — удалим её:

```
git branch -d feature-backup
```

Посмотрим историю:

```
git log --oneline --graph
```

```
* c3d4e5f Добавлен вывод даты и размера архива в backup.sh
* b2c3d4e Добавлен комментарий в конфигурацию nginx
* a1b2c3d Начальный коммит: структура проекта wordpress
```

Справочник команд ветвления

Команда	Описание
`git branch`	Показать список веток
`git branch <имя>`	Создать новую ветку
`git switch <имя>`	Переключиться на ветку
`git switch -c <имя>`	Создать ветку и сразу переключиться
`git merge <имя>`	Слить указанную ветку в текущую
`git branch -d <имя>`	Удалить ветку (только если она слита)
`git log --oneline --graph`	История с визуализацией веток

Конфликты слияния

Иногда Git не может автоматически слить изменения — это происходит, когда в двух ветках изменена одна и та же строка одного и того же файла. Такая ситуация называется **конфликтом**.

Конфликты — нормальная часть работы с Git. Не нужно их бояться. Давайте создадим конфликт намеренно, чтобы научиться его разрешать.

Создаём конфликт

Шаг 1. Создадим ветку — это будет точка расхождения:

```
git switch -c feature-config
```

Сразу вернёмся в `main`:

```
git switch main
```

Шаг 2. В ветке `main` отредактируем `config/nginx.conf`. Изменим директиву `listen`:

```
nano config/nginx.conf
```

Замените строку `listen 80;` на:

```
    listen 80 default_server;
```

Зафиксируем:

```
git add config/nginx.conf
git commit -m "Добавлен default_server в директиву listen"
```

Шаг 3. Переключимся на ветку и изменим ту же строку по-другому:

```
git switch feature-config
nano config/nginx.conf
```

Здесь файл ещё содержит старую строку `listen 80;` — ветка ничего не знает о коммите в `main`. Замените `listen 80;` на:

```
    listen 443 ssl;
```

Зафиксируем:

```
git add config/nginx.conf
git commit -m "Переключение nginx на HTTPS (порт 443)"
```

Шаг 4. Вернёмся в `main` и попробуем слить:

```
git switch main
git merge feature-config
```

```
Auto-merging config/nginx.conf
CONFLICT (content): Merge conflict in config/nginx.conf
Automatic merge failed; fix conflicts and then commit the result.
```

Разрешаем конфликт

Откроем файл с конфликтом:

```
nano config/nginx.conf
```

Git расставил маркеры конфликта:

```
<<<<<<< HEAD
    listen 80 default_server;
=======
    listen 443 ssl;
>>>>>>> feature-config
```

- Между `<<<<<<< HEAD` и `=======` — версия из текущей ветки (`main`).
- Между `=======` и `>>>>>>> feature-config` — версия из сливаемой ветки.

Ваша задача — выбрать правильный вариант (или объединить оба) и **удалить все маркеры** (строки с `<<<<<<<`, `=======` и `>>>>>>>`). Допустим, мы хотим оставить порт 80 с `default_server`:

```
    listen 80 default_server;
```

Сохраните файл и завершите слияние:

```
git add config/nginx.conf
git commit -m "Разрешён конфликт: оставлен listen 80 default_server"
```

Удалим ветку:

```
git branch -d feature-config
```

Посмотрим историю — теперь видно слияние:

```
git log --oneline --graph
```

```
*   e5f6a7b Разрешён конфликт: оставлен listen 80 default_server
|\
| * d4e5f6a Переключение nginx на HTTPS (порт 443)
* | c3d4e5f Добавлен default_server в директиву listen
|/
* b2c3d4e Добавлен комментарий в конфигурацию nginx
* a1b2c3d Начальный коммит: структура проекта wordpress
```

> **Совет.** Если во время слияния вы запутались и хотите отменить всё, выполните `git merge --abort` — Git вернёт файлы в состояние до начала слияния.

Откат изменений

Иногда вы редактируете файлы, что-то идёт не так — и проще вернуться к последнему зафиксированному состоянию, чем разбираться, что именно сломалось.

Откатить один файл к версии из последнего коммита:

```
git restore config/nginx.conf
```

Все незафиксированные изменения в этом файле будут потеряны — он вернётся к состоянию из последнего коммита.

Откатить все файлы в рабочем каталоге:

```
git restore .
```

> **Внимание.** Команда `git restore` безвозвратно удаляет незафиксированные изменения. Если вы не делали `git add` и `git commit` — восстановить правки будет невозможно. Поэтому коммитьте чаще: каждый коммит — это точка, к которой можно вернуться.

Если изменения уже добавлены в индекс (`git add`), но ещё не зафиксированы — сначала уберите их из индекса:

```
git restore --staged config/nginx.conf
```

Файл останется изменённым в рабочем каталоге, но выйдет из индекса. После этого можно откатить и сам файл командой `git restore`.

Удалённый репозиторий: GitLab

До сих пор вся работа шла локально — репозиторий существует только на вашем компьютере. Если диск сломается — история будет потеряна.

Удалённый репозиторий (remote) — это копия вашего репозитория на сервере. Он решает сразу несколько задач:

- **Резервная копия** — проект хранится не только у вас.
- **Совместная работа** — другие люди могут клонировать проект, вносить изменения и отправлять их обратно.
- **CI/CD** — автоматическая сборка и тестирование (об этом в главе 16).

GitLab — платформа для хранения Git-репозиториев с веб-интерфейсом, системой задач и встроенным CI/CD. Мы будем использовать его до конца курса.

> **Примечание.** Примеры ниже используют публичный сервис **gitlab.com**. Если ваш преподаватель предоставил собственный сервер GitLab (self-hosted), замените `gitlab.com` на его адрес во всех командах.

Регистрация на GitLab

1. Откройте в браузере **https://gitlab.com** и создайте аккаунт.
2. Подтвердите email.
3. Запомните своё имя пользователя (username) — оно будет частью URL вашего репозитория.

Настройка SSH-ключа

В главе 01 мы подключались к серверу по SSH. Тот же принцип используется для безопасной связи с GitLab — без ввода пароля при каждом `push` и `pull`.

Шаг 1. Сгенерируйте пару ключей (если ещё не сделали):

```
ssh-keygen -t ed25519 -C "ivan@example.com"
```

Нажмите Enter на все вопросы (имя файла по умолчанию, пароль по желанию).

Шаг 2. Скопируйте публичный ключ:

```
cat ~/.ssh/id_ed25519.pub
```

```
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAI... ivan@example.com
```

Шаг 3. Добавьте ключ в GitLab:

1. Войдите на gitlab.com.
2. Перейдите в **Settings** (иконка профиля → Edit profile) → **SSH Keys**.
3. Вставьте содержимое публичного ключа в поле «Key».
4. Нажмите **Add key**.

Шаг 4. Проверьте подключение:

```
ssh -T git@gitlab.com
```

```
Welcome to GitLab, @username!
```

Если вы видите приветствие — всё настроено верно.

Создание проекта на GitLab

1. На gitlab.com нажмите **New project → Create blank project**.
2. Имя проекта: `wordpress-project`.
3. Видимость: **Private** (только для вас) или **Public** (виден всем).
4. Снимите галочку «Initialize repository with a README» — у нас уже есть локальный репозиторий.
5. Нажмите **Create project**.

GitLab покажет инструкции для существующего репозитория. Нам нужна секция «Push an existing Git repository».

Подключение и отправка

Свяжем локальный репозиторий с удалённым:

```
cd ~/wordpress-project
git remote add origin git@gitlab.com:<username>/wordpress-project.git
```

Замените `<username>` на ваше имя пользователя GitLab.

Проверим, что remote добавлен:

```
git remote -v
```

```
origin  git@gitlab.com:<username>/wordpress-project.git (fetch)
origin  git@gitlab.com:<username>/wordpress-project.git (push)
```

`origin` — стандартное имя для основного удалённого репозитория.

Отправим все коммиты на сервер:

```
git push -u origin main
```

```
Enumerating objects: 18, done.
Counting objects: 100% (18/18), done.
Delta compression using up to 4 threads
Compressing objects: 100% (14/14), done.
Writing objects: 100% (18/18), 1.84 KiB | 1.84 MiB/s, done.
Total 18 (delta 3), reused 0 (delta 0)
To gitlab.com:<username>/wordpress-project.git
 * [new branch]      main -> main
Branch 'main' set up to track remote branch 'main' from 'origin'.
```

Флаг `-u` (upstream) запоминает связь между локальной веткой `main` и удалённой `origin/main`. После этого можно просто писать `git push` без дополнительных аргументов.

Откройте проект на gitlab.com — вы увидите все свои файлы и историю коммитов в веб-интерфейсе.

Получение изменений и клонирование

Если кто-то (или вы сами с другого компьютера) внёс изменения в удалённый репозиторий, получить их можно командой:

```
git pull
```

`git pull` скачивает новые коммиты с сервера и автоматически сливает их с вашей локальной веткой.

Чтобы скопировать удалённый репозиторий на новую машину:

```
git clone git@gitlab.com:<username>/wordpress-project.git
```

Эта команда создаст каталог `wordpress-project/` с полной историей коммитов и настроенным `origin`.

Справочник команд для работы с удалёнными репозиториями

Команда	Описание
`git remote add origin <url>`	Добавить удалённый репозиторий
`git remote -v`	Показать список удалённых репозиториев
`git push`	Отправить коммиты на сервер
`git push -u origin main`	Первая отправка с привязкой ветки
`git pull`	Получить и слить изменения с сервера
`git clone <url>`	Клонировать удалённый репозиторий

Практические задания

1. Установите Git и настройте имя и email. Проверьте настройки командой `git config --list`.
2. Инициализируйте репозиторий в каталоге `~/wordpress-project`. Выполните `git status` и объясните, что выводит команда.
3. Создайте файл `.gitignore`, который исключает каталог `logs/`, файлы `*.swp` и архивы `*.tar.gz`. Убедитесь, что `git status` больше не показывает `logs/`.
4. Добавьте все файлы в индекс и сделайте первый коммит. Посмотрите историю командой `git log`.
5. Отредактируйте `config/nginx.conf` (добавьте комментарий). Посмотрите изменения через `git diff`, затем добавьте в индекс и зафиксируйте вторым коммитом.
6. Создайте ветку `feature-backup`. Переключитесь на неё, внесите изменение в `backup.sh`, сделайте коммит. Вернитесь в `main` и выполните слияние. Удалите ветку. Посмотрите историю с `git log --oneline --graph`.
7. Создайте конфликт слияния: в `main` измените строку в `config/nginx.conf`, создайте ветку и измените ту же строку по-другому. Попробуйте выполнить `git merge`. Разрешите конфликт вручную и завершите слияние.
8. Сгенерируйте SSH-ключ (если ещё не сделали), добавьте его в GitLab. Создайте проект на GitLab и отправьте репозиторий командой `git push -u origin main`.

9. Клонируйте свой репозиторий в другой каталог (`git clone ... ~/wordpress-project-clone`). Убедитесь, что в клоне есть вся история коммитов (`git log`).

Итоги

В этой главе вы:

- узнали, зачем нужен контроль версий и как устроен Git (три зоны: рабочий каталог, индекс, репозиторий);
- научились создавать репозиторий (`git init`), отслеживать изменения (`status`, `diff`) и фиксировать их (`add`, `commit`);
- настроили `.gitignore`, чтобы логи, временные файлы и архивы не попадали в репозиторий;
- освоили ветвление (`branch`, `switch`, `merge`) и научились разрешать конфликты слияния;
- подключили удалённый репозиторий на GitLab через SSH и научились отправлять (`push`), получать (`pull`) и клонировать (`clone`) проект.

Теперь ваш проект wordpress-project хранится не только на диске, но и на сервере GitLab — с полной историей всех изменений.

В следующей главе мы познакомимся с **Docker** — инструментом контейнеризации. Он позволит упаковать весь наш стек (nginx + PHP-FPM + WordPress) в изолированные контейнеры, которые можно запускать на любом сервере одной командой.

Этап 14. Docker: контейнеризация проекта

Зачем нужны контейнеры

В главе 11 мы установили nginx, PHP-FPM и MariaDB вручную: добавляли пакеты, редактировали конфиги, создавали пользователей базы данных. Всё заработало — но представьте, что нужно повторить это на втором сервере. Или на десятом. Или что после обновления PHP сломался WordPress, а откатить изменения непросто.

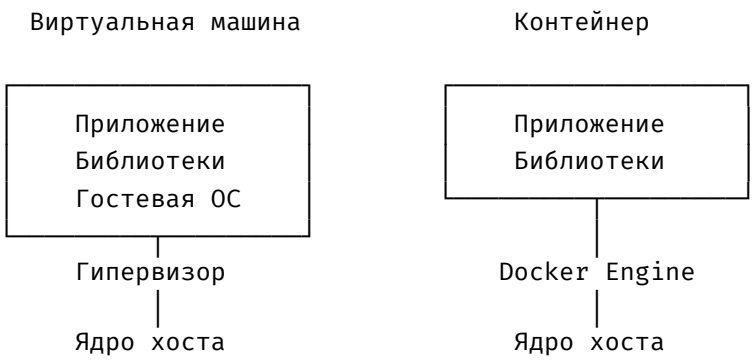
Знакомая ситуация в разработке: «У меня на машине всё работает, а на сервере — нет». Причина обычно одна — **различия в окружении**: другая версия PHP, другие пути к файлам, не хватает библиотеки.

Контейнер решает эту проблему. Это изолированная среда, в которой упакованы приложение и все его зависимости. Контейнер работает одинаково на любой машине — на вашем ноутбуке, на тестовом сервере, в продакшене.

Docker — самый популярный инструмент для работы с контейнерами.

Контейнер vs. виртуальная машина

В главе 01 мы упоминали виртуальные машины — это полноценный компьютер внутри компьютера со своей операционной системой. Контейнер устроен проще:



Контейнер не содержит отдельной ОС — он использует ядро хост-системы. Поэтому контейнеры:

- **запускаются за секунды** (а не за минуты, как виртуальная машина);
- **потребляют меньше ресурсов** — нет накладных расходов на гостевую ОС;
- **легко переносимы** — образ контейнера можно запустить на любом сервере с Docker.

Один контейнер — один процесс. Иногда новички пытаются использовать контейнер как виртуальную машину: запускают в одном контейнере и nginx, и PHP-FPM, и базу данных, ставят супервизор процессов. Технически это возможно, но это примерно как забивать шурупы молотком — работает, но неправильно. Вся суть контейнеров в том, что каждый из них отвечает за **один сервис**. Это упрощает обновление, масштабирование и отладку. Как запустить несколько контейнеров вместе — разберём в следующей главе.

Привязка к проекту. В этой главе мы научимся запускать nginx из нашего wordpress-project в контейнере Docker. А в следующей главе соберём весь стек (nginx + PHP-FPM + WordPress + MariaDB) из нескольких связанных контейнеров.

Врезка: форматы конфигурационных файлов

В предыдущих главах мы уже работали с конфигурационными файлами: `nginx.conf`, `php.ini`, `/etc/apt/sources.list`. Прежде чем перейти к Docker, разберём три основных формата, которые встречаются повсеместно в Linux и DevOps.

INI

Формат INI знаком вам по `php.ini` из главы 11. Простая структура: секции в квадратных скобках, параметры в виде `ключ = значение`:

```
[server]
host = 127.0.0.1
port = 8080
debug = true

[database]
host = localhost
name = wordpress
```

Где встречается: `php.ini`, `my.cnf` (MariaDB), `~/.gitconfig`, файлы systemd (unit-файлы из главы 09 похожи на INI).

JSON

JSON (JavaScript Object Notation) — строгий формат с фигурными скобками, кавычками и запятыми:

```
{
  "server": {
    "host": "127.0.0.1",
    "port": 8080,
    "debug": true
  },
  "database": {
    "host": "localhost",
    "name": "wordpress"
  }
}
```

Где встречается: API веб-сервисов, конфигурации VS Code и npm, файл `wp-config.json` (в новых версиях WordPress).

Особенности: не поддерживает комментарии, строгий синтаксис (забыли запятую — ошибка).

YAML

YAML (YAML Ain't Markup Language) — формат, который читается почти как обычный текст. Структура задаётся **отступами**:

```yaml
server:
  host: 127.0.0.1
  port: 8080
  debug: true

database:
  host: localhost
  name: wordpress
```

Где встречается: Docker Compose (`docker-compose.yml`), CI/CD конфиги (`.gitlab-ci.yml`, GitHub Actions), Ansible, Kubernetes.

Правила YAML, которые нужно запомнить:

- Отступы — **только пробелы**, никогда табуляция. Обычно 2 пробела.
- Вложенность определяется уровнем отступа.
- Списки обозначаются дефисом:

```yaml
fruits:
  - apple
  - banana
  - cherry
```

- Строки можно писать без кавычек (кроме специальных случаев).
- Комментарии начинаются с #.

  Важно. YAML очень чувствителен к отступам. Лишний или недостающий пробел приведёт к ошибке. Используйте редактор с подсветкой синтаксиса и будьте аккуратны.

Сравнение форматов

Формат	Синтаксис	Комментарии	Где чаще всего
INI	`ключ = значение`, секции `[...]`	`;` или #	Конфиги приложений (php.ini, my.cnf)
JSON	Фигурные скобки, кавычки	Нет	API, веб-приложения
YAML	Отступы, двоеточия	#	DevOps (Docker Compose, CI/CD, Ansible)

В следующих главах мы будем активно работать с YAML — для описания Docker Compose и CI/CD конфигураций.

Установка Docker

Docker не входит в стандартные репозитории Debian — мы установим его из официального репозитория Docker. В главе 07 мы уже добавляли сторонние репозитории — здесь принцип тот же.

Подготовка

Установим пакеты, необходимые для работы с HTTPS-репозиториями:

```
sudo apt update
sudo apt install ca-certificates curl gnupg
```

Добавление репозитория Docker

Скачаем и сохраним GPG-ключ Docker (он нужен для проверки подлинности пакетов):

```
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/debian/gpg \
  | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg
```

Добавим репозиторий Docker в список источников:

```
echo \
  "deb [arch=$(dpkg --print-architecture) \
  signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/debian \
  $(. /etc/os-release && echo "$VERSION_CODENAME") stable" \
  | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

Установка

```
sudo apt update
sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin
```

Настройка доступа без sudo

По умолчанию Docker требует прав root. Чтобы не писать `sudo` перед каждой командой, добавьте своего пользователя в группу `docker`:

```
sudo usermod -aG docker $USER
```

Выйдите из сессии SSH и подключитесь заново, чтобы изменения вступили в силу. Можно также выполнить `newgrp docker` в текущей сессии.

Проверка

```
docker run hello-world
```

```
Hello from Docker!
This message shows that your installation appears to be working correctly.
...
```

Если вы видите это сообщение — Docker установлен и работает. Что произошло:

1. Docker не нашёл образ `hello-world` локально.
2. Скачал его из **Docker Hub** (реестр образов).
3. Создал контейнер из этого образа.
4. Контейнер вывел сообщение и завершился.

Образы и контейнеры

Два ключевых понятия Docker:

- **Образ** (image) — шаблон для создания контейнера. Содержит файловую систему с приложением и всеми зависимостями. Образ неизменяем — его нельзя отредактировать, можно только создать новый.
- **Контейнер** (container) — запущенный экземпляр образа. Контейнер можно запускать, останавливать, удалять. Изменения внутри контейнера не влияют на образ.

Аналогия: образ — это рецепт, контейнер — приготовленное по рецепту блюдо. Из одного образа можно создать сколько угодно контейнеров.

Docker Hub

Docker Hub (hub.docker.com) — публичный реестр образов. Здесь хранятся готовые образы для большинства популярных приложений: nginx, PHP, MariaDB, WordPress, Ubuntu, Debian и тысячи других.

Образы бывают:

- **Официальные** — поддерживаются разработчиками приложений или Docker. Имя без префикса: `nginx`, `php`, `mariadb`.
- **Пользовательские** — созданы сообществом. Имя с префиксом: `username/myapp`.

Каждый образ имеет **теги** — версии:

Образ	Описание
`nginx:latest`	Последняя стабильная версия (по умолчанию)
`nginx:1.27`	Конкретная версия 1.27
`nginx:1.27-alpine`	Версия 1.27 на базе Alpine Linux (минимальный размер)

> **Совет.** В продакшене всегда указывайте конкретную версию (`nginx:1.27`), а не `latest`. Иначе при обновлении образа может прийти версия с несовместимыми изменениями.

Основные команды для работы с образами и контейнерами

Скачаем образ nginx:

```
docker pull nginx
```

```
Using default tag: latest
latest: Pulling from library/nginx
...
Status: Downloaded newer image for nginx:latest
```

Посмотрим список скачанных образов:

```
docker images
```

```
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
nginx         latest    a8758716bb6a   2 weeks ago    187MB
hello-world   latest    d2c94e258dcb   10 months ago  13.3kB
```

Запустим контейнер:

```
docker run -d --name test-nginx nginx
```

Флаг `-d` (detach) запускает контейнер в фоновом режиме, `--name test-nginx` задаёт имя.

Посмотрим запущенные контейнеры:

```
docker ps
```

```
CONTAINER ID   IMAGE   COMMAND                  CREATED         STATUS
 ↪  PORTS     NAMES
f3a1b2c3d4e5   nginx   "/docker-entrypoint.…"   5 seconds ago   Up 4 seconds
 ↪  80/tcp    test-nginx
```

Посмотрим **все** контейнеры (включая остановленные):

```
docker ps -a
```

Остановим и удалим контейнер:

```
docker stop test-nginx
docker rm test-nginx
```

Удалим ненужный образ:

```
docker rmi hello-world
```

Справочник команд

Команда	Описание
`docker pull <образ>`	Скачать образ из Docker Hub
`docker images`	Список скачанных образов
`docker run <образ>`	Создать и запустить контейнер
`docker ps`	Список запущенных контейнеров
`docker ps -a`	Список всех контейнеров
`docker stop <имя>`	Остановить контейнер
`docker rm <имя>`	Удалить контейнер
`docker rmi <образ>`	Удалить образ

Запуск nginx в контейнере

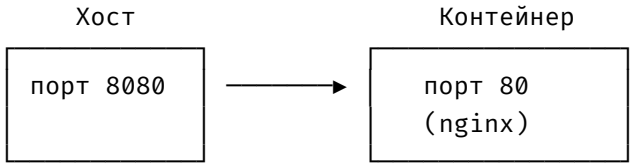
Теперь запустим nginx так, чтобы он был доступен извне и отдавал файлы нашего проекта.

Проброс порта

Контейнер по умолчанию изолирован от хост-системы. Чтобы обратиться к nginx внутри контейнера, нужно **пробросить порт** — связать порт хоста с портом контейнера:

```
docker run -d --name my-nginx -p 8080:80 nginx
```

Флаг `-p 8080:80` означает: запросы на порт **8080 хоста** перенаправлять на порт **80 контейнера** (стандартный порт nginx).



Проверим:

```
curl http://localhost:8080
```

Вы увидите стандартную страницу «Welcome to nginx!». Nginx работает внутри контейнера, но доступен через порт 8080 хоста.

Остановим контейнер перед следующим шагом:

```
docker stop my-nginx
docker rm my-nginx
```

Монтирование файлов (volumes)

Стандартная страница nginx — не то, что нам нужно. Мы хотим отдавать файлы нашего проекта. Для этого используем **тома** (volumes) — механизм монтирования каталогов хоста внутрь контейнера.

Флаг `-v` связывает путь на хосте с путём в контейнере:

```
docker run -d --name my-nginx \
  -p 8080:80 \
  -v ~/wordpress-project/site:/usr/share/nginx/html:ro \
  nginx
```

Разберём `-v ~/wordpress-project/site:/usr/share/nginx/html:ro`:

Часть	Значение
`~/wordpress-project/site`	Путь на хосте (наши файлы)
`:`	Разделитель
`/usr/share/nginx/html`	Путь внутри контейнера (куда nginx ищет файлы)
`:ro`	Read-only — контейнер может только читать эти файлы

Проверим:

```
curl http://localhost:8080
```

Теперь nginx отдаёт наш `index.html` из `wordpress-project/site/`.

Можно смонтировать и конфигурацию nginx:

```
docker stop my-nginx
docker rm my-nginx

docker run -d --name my-nginx \
  -p 8080:80 \
  -v ~/wordpress-project/site:/usr/share/nginx/html:ro \
  -v ~/wordpress-project/config/nginx.conf:/etc/nginx/conf.d/default.conf:ro \
  nginx
```

Теперь контейнер использует наш конфиг и наши файлы — но сам nginx установлен не на хосте, а внутри контейнера.

Dockerfile: собственный образ

Команда `docker run` с множеством флагов `-v` неудобна — легко забыть какой-нибудь параметр. **Dockerfile** позволяет описать образ в файле: какой базовый образ использовать, какие файлы скопировать внутрь, какие порты открыть.

Создание Dockerfile

Остановим и удалим предыдущий контейнер:

```
docker stop my-nginx
docker rm my-nginx
```

Создадим файл `Dockerfile` в корне проекта:

```
cd ~/wordpress-project
nano Dockerfile
```

```
FROM nginx:1.27

COPY config/nginx.conf /etc/nginx/conf.d/default.conf
COPY site/ /usr/share/nginx/html/

EXPOSE 80
```

Разберём каждую инструкцию:

Инструкция	Описание
`FROM nginx:1.27`	Базовый образ — nginx версии 1.27
`COPY config/nginx.conf ...`	Копирует наш конфиг внутрь образа
`COPY site/ ...`	Копирует файлы сайта внутрь образа
`EXPOSE 80`	Документирует, что контейнер слушает порт 80

> **Примечание.** `EXPOSE` не пробрасывает порт автоматически — это лишь документация для того, кто будет использовать образ. Проброс по-прежнему делается флагом `-p` при запуске.

Сборка образа

```
docker build -t wordpress-nginx .
```

Флаг `-t wordpress-nginx` задаёт имя (тег) образа. Точка (`.`) означает, что контекст сборки — текущий каталог (Docker отправит все файлы из него в процесс сборки).

```
[+] Building 2.1s (8/8) FINISHED
 => [1/3] FROM nginx:1.27
 => [2/3] COPY config/nginx.conf /etc/nginx/conf.d/default.conf
 => [3/3] COPY site/ /usr/share/nginx/html/
 => exporting to image
 => => naming to docker.io/library/wordpress-nginx
```

Проверим, что образ создан:

```
docker images
```

```
REPOSITORY        TAG       IMAGE ID       CREATED          SIZE
wordpress-nginx   latest    b1c2d3e4f5a6   10 seconds ago   187MB
nginx             1.27      a8758716bb6a   2 weeks ago      187MB
```

Запуск из собственного образа

```
docker run -d --name wp-nginx -p 8080:80 wordpress-nginx
```

Теперь не нужны флаги `-v` — конфиг и файлы сайта уже внутри образа. Проверим:

```
curl http://localhost:8080
```

.dockerignore

При сборке Docker отправляет всё содержимое текущего каталога в процесс сборки. Чтобы исключить ненужные файлы (логи, каталог `.git`, архивы), создайте файл `.dockerignore`:

```
nano .dockerignore
```

```
logs/
.git/
*.tar.gz
*.swp
```

Синтаксис такой же, как у `.gitignore`. После создания `.dockerignore` пересоберите образ — он может стать меньше, а сборка быстрее.

Справочник инструкций Dockerfile

Инструкция	Описание
`FROM <образ>`	Базовый образ
`COPY <src> <dst>`	Копировать файлы из контекста в образ
`RUN <команда>`	Выполнить команду при сборке (установить пакеты и т.п.)
`EXPOSE <порт>`	Документировать порт
`CMD ["..."]`	Команда по умолчанию при запуске контейнера
`WORKDIR <путь>`	Рабочий каталог внутри контейнера
`ENV <KEY>=<value>`	Установить переменную окружения

Управление контейнерами

Жизненный цикл контейнера

```
docker run ──► Работает ──► docker stop ──► Остановлен
                                                │
               docker start ◄───────────────────┘
                    │
                    ▼
               Работает ──► docker rm ──► Удалён
```

Контейнер можно остановить и запустить снова — данные внутри сохраняются, пока контейнер не удалён.

Просмотр логов

```
docker logs wp-nginx
```

Выводит stdout и stderr контейнера. Для nginx это access.log и error.log.

Следить за логами в реальном времени (как `journalctl -f`):

```
docker logs -f wp-nginx
```

Нажмите `Ctrl+C` для выхода.

Выполнение команд внутри контейнера

Иногда нужно заглянуть «внутрь» контейнера — посмотреть файлы, проверить конфигурацию. Для этого используется `docker exec`:

```
docker exec -it wp-nginx bash
```

Флаги `-it` обеспечивают интерактивный режим с терминалом. Вы окажетесь внутри контейнера:

```
root@f3a1b2c3d4e5:/# ls /usr/share/nginx/html/
index.html
root@f3a1b2c3d4e5:/# cat /etc/nginx/conf.d/default.conf
...
root@f3a1b2c3d4e5:/# exit
```

Обратите внимание: внутри контейнера минимальная среда — многих привычных утилит (vim, less, ps) может не быть. Это нормально: контейнеры намеренно делают компактными.

Можно выполнить одну команду без входа в интерактивный режим:

```
docker exec wp-nginx nginx -t


nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

Перезапуск контейнера

```
docker restart wp-nginx
```

Полезно после изменения конфигурации (если файлы смонтированы через volume).

Информация о контейнере

```
docker inspect wp-nginx
```

Выводит подробную информацию в формате JSON: сетевые настройки, смонтированные тома, переменные окружения и многое другое. Можно извлечь конкретное значение:

```
docker inspect -f '{{.NetworkSettings.IPAddress}}' wp-nginx
```

Справочник команд управления

Команда	Описание
`docker start <имя>`	Запустить остановленный контейнер
`docker stop <имя>`	Остановить контейнер
`docker restart <имя>`	Перезапустить контейнер
`docker logs <имя>`	Показать логи контейнера
`docker logs -f <имя>`	Следить за логами в реальном времени
`docker exec -it <имя> bash`	Войти внутрь контейнера
`docker exec <имя> <команда>`	Выполнить команду в контейнере
`docker inspect <имя>`	Подробная информация о контейнере

Очистка

Со временем накапливаются остановленные контейнеры и неиспользуемые образы. Полезные команды для очистки:

```
# Удалить все остановленные контейнеры
docker container prune

# Удалить неиспользуемые образы
docker image prune

# Удалить всё неиспользуемое (контейнеры, образы, сети)
docker system prune
```

Docker запросит подтверждение перед удалением.

Практические задания

1. Установите Docker из официального репозитория. Добавьте своего пользователя в группу `docker`. Проверьте установку командой `docker run hello-world`.
2. Скачайте образ nginx (`docker pull nginx`). Запустите контейнер с пробросом порта 8080 → 80. Откройте `http://localhost:8080` через `curl` и убедитесь, что видите стандартную страницу nginx.
3. Запустите nginx с монтированием каталога `~/wordpress-project/site` как корня веб-сервера. Проверьте, что `curl` показывает ваш `index.html`.
4. Добавьте монтирование конфигурации nginx из `~/wordpress-project/config/nginx.conf`. Проверьте, что контейнер использует ваш конфиг (`docker exec <имя> nginx -t`).

5. Создайте `Dockerfile` в `~/wordpress-project`. Соберите образ `wordpress-nginx` и запустите контейнер из него. Убедитесь, что сайт работает без флагов `-v`.
6. Создайте `.dockerignore`, исключающий `logs/`, `.git/`, `*.tar.gz` и `*.swp`. Пересоберите образ.
7. Посмотрите логи работающего контейнера (`docker logs`). Сделайте несколько запросов через `curl` и убедитесь, что они появляются в логах.
8. Войдите внутрь контейнера (`docker exec -it ... bash`). Посмотрите содержимое `/usr/share/nginx/html/` и `/etc/nginx/conf.d/`. Выйдите командой `exit`.
9. Остановите и удалите все контейнеры. Удалите образ `hello-world`. Выполните `docker system prune` для полной очистки.

Итоги

В этой главе вы:

- узнали, зачем нужны контейнеры и чем они отличаются от виртуальных машин;
- разобрали три формата конфигурационных файлов: INI, JSON и YAML (YAML пригодится в следующих главах);
- установили Docker из официального репозитория;
- научились работать с образами и контейнерами (`pull`, `run`, `ps`, `stop`, `rm`);
- запустили nginx в контейнере с пробросом порта и монтированием файлов проекта через volumes;
- написали Dockerfile и собрали собственный образ `wordpress-nginx`;
- освоили управление контейнерами: логи, выполнение команд внутри контейнера, очистка.

В следующей главе мы используем **Docker Compose**, чтобы запустить весь стек — nginx, PHP-FPM, MariaDB и WordPress — в связанных контейнерах одной командой `docker compose up`.

Этап 15. Docker Compose: оркестрация в контейнерах

Зачем нужен Docker Compose

В предыдущей главе мы запускали один контейнер nginx командой `docker run` с несколькими флагами. Для полного стека WordPress нужно **три контейнера**: MariaDB, WordPress (с PHP-FPM) и nginx. У каждого свои порты, volumes, переменные окружения.

Представьте три команды `docker run`, каждая на несколько строк:

```
docker run -d --name db \
  -e MYSQL_ROOT_PASSWORD=secret \
  -e MYSQL_DATABASE=wordpress \
  -v db_data:/var/lib/mysql \
  mariadb:11

docker run -d --name wordpress \
  -e WORDPRESS_DB_HOST=db \
  -e WORDPRESS_DB_PASSWORD=secret \
  --link db \
  -v wp_data:/var/www/html \
  wordpress:6-fpm

docker run -d --name nginx \
  -p 80:80 \
  -v ./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro \
  -v wp_data:/var/www/html:ro \
  --link wordpress \
  nginx:1.27
```

Запускать, останавливать и обновлять три контейнера по отдельности — неудобно и легко ошибиться. **Docker Compose** решает эту проблему: весь стек описывается в одном YAML-файле, а управляется одной командой.

> **Привязка к проекту.** В главе 11 мы разворачивали nginx + PHP-FPM + MariaDB + WordPress вручную — устанавливали пакеты, редактировали конфиги, создавали базу данных. Сейчас мы получим тот же результат одной командой `docker compose up`.

Установка

Docker Compose — это плагин Docker. Установим его:

```
sudo apt install docker-compose-plugin
```

Проверим:

```
docker compose version
```

```
Docker Compose version v2.32.4
```

> **Примечание.** Старая версия инструмента называлась `docker-compose` (через дефис) и устанавливалась отдельно. Новая — `docker compose` (через пробел) — встроенный плагин Docker. В интернете вы ещё встретите старый синтаксис, но используйте новый.

Подготовка структуры проекта

Перестроим каталог `wordpress-project/` для контейнерного деплоя:

```
cd ~/wordpress-project
mkdir -p nginx
```

Итоговая структура будет такой:

```
wordpress-project/
├── docker-compose.yml    ← описание всего стека
├── .env                  ← пароли и настройки (не в Git!)
├── nginx/
│   └── default.conf      ← конфигурация nginx
├── .gitignore
└── .dockerignore
```

WordPress и MariaDB будут работать из готовых образов Docker Hub. Для nginx нужен только кастомный конфиг.

Шаг 1: база данных (MariaDB)

Начнём с одного сервиса — базы данных. Создадим файл `docker-compose.yml`:

```
nano docker-compose.yml
```

```
services:
  db:
    image: mariadb:11
    restart: unless-stopped
    env_file:
      - .env
    environment:
      MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASSWORD}
```

```
      MYSQL_DATABASE: ${DB_NAME}
      MYSQL_USER: ${DB_USER}
      MYSQL_PASSWORD: ${DB_PASSWORD}
    volumes:
      - db_data:/var/lib/mysql

volumes:
  db_data:
```

Разберём ключевые элементы:

Ключ	Описание
`services`	Список сервисов (контейнеров)
`db`	Имя сервиса — будет использоваться как сетевое имя
`image: mariadb:11`	Образ из Docker Hub
`restart: unless-stopped`	Перезапускать контейнер, если он упал (кроме ручной остановки)
`env_file`	Файл с переменными окружения
`environment`	Переменные окружения для контейнера
`${DB_ROOT_PASSWORD}`	Подстановка значения из `.env`
`volumes`	Привязка томов (volumes)
`db_data:/var/lib/mysql`	Именованный том для хранения данных БД

В секции `volumes:` на верхнем уровне мы **объявляем** именованный том `db_data`. Docker создаст его автоматически. Данные в именованном томе сохраняются даже после удаления контейнера — пока вы явно не удалите том.

Где хранятся данные тома

Именованные тома Docker хранит в своём внутреннем каталоге. Найти путь к данным можно командой:

```
docker volume inspect wordpress-project_db_data
```

```
[
    {
        "Name": "wordpress-project_db_data",
        "Mountpoint":
         ↪  "/var/lib/docker/volumes/wordpress-project_db_data/_data",
        ...
    }
]
```

Поле `Mountpoint` — путь к файлам тома на хосте. Однако этот каталог принадлежит Docker, и работать с ним напрямую не очень удобно.

Если вы хотите, чтобы данные лежали прямо в каталоге проекта (например, чтобы скопировать весь проект на другой сервер или на флешку), используйте **локальный путь** вместо именованного тома:

```
    volumes:
      - ./db_data:/var/lib/mysql
```

В этом случае данные БД окажутся в `~/wordpress-project/db_data/`. Достаточно выполнить `docker compose down`, скопировать весь каталог `wordpress-project/` — и на другом сервере `docker compose up -d` поднимет стек с теми же данными.

> **Компромисс.** Именованные тома работают быстрее (особенно на macOS и Windows) и управляются Docker. Локальные пути удобнее для переноса и резервного копирования. В этом курсе мы используем именованные тома, но для своих проектов выбирайте то, что подходит под задачу.

Файл .env

Создадим файл с паролями и настройками:

```
nano .env
```

```
DB_ROOT_PASSWORD=SuperSecret123
DB_NAME=wordpress
DB_USER=wp_user
DB_PASSWORD=WpPassword456
```

> **Важно.** Файл `.env` содержит пароли — он **не должен** попадать в Git. Добавьте его в `.gitignore` (мы разбирали это в главе 13):
>
> ```
> echo ".env" >> .gitignore
> ```

Запуск и проверка

```
docker compose up -d
```

```
[+] Running 2/2
 ✔ Network wordpress-project_default  Created
 ✔ Container wordpress-project-db-1   Started
```

Обратите внимание: Docker Compose автоматически создал **сеть** `wordpress-project_default`. Все сервисы из одного compose-файла попадают в эту сеть и могут обращаться друг к другу **по имени сервиса** (например, `db`). Это работает благодаря встроенному DNS — не нужно знать IP-адреса контейнеров.

Проверим статус:

```
docker compose ps
```

```
NAME                        IMAGE        ...  STATUS          PORTS
wordpress-project-db-1      mariadb:11   ...  Up 10 seconds   3306/tcp
```

Посмотрим логи:

```
docker compose logs db
```

Если видите строку `ready for connections` — MariaDB запустилась успешно.

Шаг 2: WordPress

Добавим сервис `wordpress` в `docker-compose.yml`. Откройте файл и допишите сервис:

```
nano docker-compose.yml
```

```yaml
services:
  db:
    image: mariadb:11
    restart: unless-stopped
    env_file:
      - .env
    environment:
      MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASSWORD}
      MYSQL_DATABASE: ${DB_NAME}
      MYSQL_USER: ${DB_USER}
      MYSQL_PASSWORD: ${DB_PASSWORD}
    volumes:
      - db_data:/var/lib/mysql

  wordpress:
    image: wordpress:6-fpm
    restart: unless-stopped
    depends_on:
      - db
    env_file:
      - .env
    environment:
      WORDPRESS_DB_HOST: db
      WORDPRESS_DB_NAME: ${DB_NAME}
      WORDPRESS_DB_USER: ${DB_USER}
      WORDPRESS_DB_PASSWORD: ${DB_PASSWORD}
    volumes:
      - wordpress_data:/var/www/html

volumes:
  db_data:
  wordpress_data:
```

Что нового:

Ключ	Описание
`wordpress:6-fpm`	Официальный образ WordPress с PHP-FPM (без встроенного веб-сервера)
`depends_on: - db`	WordPress запускается **после** MariaDB
`WORDPRESS_DB_HOST: db`	Адрес базы данных — имя сервиса `db` (внутренний DNS Compose)
`wordpress_data`	Том для файлов WordPress (`/var/www/html`)

> **Почему `wordpress:6-fpm`, а не `wordpress:6`?** Образ `wordpress:6` содержит Apache — встроенный веб-сервер. Нам он не нужен, потому что мы используем nginx. Образ с тегом `fpm` содержит только PHP-FPM, который обрабатывает PHP-файлы и отдаёт результат через FastCGI.

Применим изменения:

```
docker compose up -d
```

```
[+] Running 3/3
 ✔ Container wordpress-project-db-1          Running
 ✔ Container wordpress-project-wordpress-1   Started
```

Docker Compose не перезапустил `db` — он видит, что сервис не изменился. Запустился только новый сервис `wordpress`.

Проверим, что WordPress видит базу данных:

```
docker compose logs wordpress
```

Если нет ошибок подключения к БД — всё работает.

Шаг 3: nginx

Сначала создадим конфигурацию nginx. Он должен:

- отдавать статические файлы (CSS, JS, картинки) самостоятельно;
- передавать PHP-запросы на WordPress через FastCGI.

```
nano nginx/default.conf
```

```
server {
    listen 80;
    server_name localhost;

    root /var/www/html;
    index index.php index.html;

    location / {
        try_files $uri $uri/ /index.php?$args;
    }

    location ~ \.php$ {
        fastcgi_pass wordpress:9000;
        fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
        include fastcgi_params;
    }

    location ~* \.(js|css|png|jpg|jpeg|gif|ico|svg)$ {
        expires 30d;
        access_log off;
    }
}
```

Ключевая строка — `fastcgi_pass wordpress:9000`. Здесь `wordpress` — имя сервиса из `docker-compose.yml`, а `9000` — порт, на котором PHP-FPM слушает соединения внутри контейнера.

Теперь добавим сервис nginx в `docker-compose.yml`:

```
nano docker-compose.yml
```

Финальная версия файла:

```yaml
services:
  db:
    image: mariadb:11
    restart: unless-stopped
    env_file:
      - .env
    environment:
      MYSQL_ROOT_PASSWORD: ${DB_ROOT_PASSWORD}
      MYSQL_DATABASE: ${DB_NAME}
      MYSQL_USER: ${DB_USER}
      MYSQL_PASSWORD: ${DB_PASSWORD}
    volumes:
      - db_data:/var/lib/mysql

  wordpress:
    image: wordpress:6-fpm
    restart: unless-stopped
    depends_on:
      - db
    env_file:
      - .env
    environment:
      WORDPRESS_DB_HOST: db
      WORDPRESS_DB_NAME: ${DB_NAME}
      WORDPRESS_DB_USER: ${DB_USER}
      WORDPRESS_DB_PASSWORD: ${DB_PASSWORD}
    volumes:
      - wordpress_data:/var/www/html

  nginx:
    image: nginx:1.27
    restart: unless-stopped
    depends_on:
      - wordpress
    ports:
      - "80:80"
    volumes:
      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf:ro
      - wordpress_data:/var/www/html:ro

volumes:
  db_data:
  wordpress_data:
```

Что нового в сервисе `nginx`:

Ключ	Описание
`ports: - "80:80"`	Проброс порта 80 хоста на порт 80 контейнера
`./nginx/default.conf:...`	Монтирование нашего конфига (путь на хосте → путь в контейнере)
`wordpress_data:/var/www/html:ro`	Тот же том, что у WordPress — nginx читает статические файлы
`:ro`	Read-only — nginx только читает файлы, не изменяет их

Обратите внимание: сервисы `wordpress` и `nginx` используют **один и тот же том** `wordpress_data`. WordPress записывает PHP-файлы в `/var/www/html`, а nginx читает оттуда статику.

Запуск и проверка

Запустим весь стек:

```
docker compose up -d
```

```
[+] Running 4/4
 ✔ Network wordpress-project_default         Created
 ✔ Container wordpress-project-db-1          Started
 ✔ Container wordpress-project-wordpress-1   Started
 ✔ Container wordpress-project-nginx-1       Started
```

Проверим, что все три контейнера работают:

```
docker compose ps
```

```
NAME                            IMAGE              STATUS          PORTS
wordpress-project-db-1          mariadb:11         Up 5 seconds    3306/tcp
wordpress-project-nginx-1       nginx:1.27         Up 3 seconds
 ↪  0.0.0.0:80->80/tcp
wordpress-project-wordpress-1   wordpress:6-fpm    Up 4 seconds    9000/tcp
```

Проверим через curl:

```
curl -I http://localhost
```

```
HTTP/1.1 302 Found
Location: http://localhost/wp-admin/install.php
...
```

Код 302 и редирект на `/wp-admin/install.php` — WordPress работает и предлагает пройти мастер установки. Это тот же результат, что в главе 11, но вместо ручной установки пакетов, настройки конфигов и создания базы данных — **одна команда** `docker compose up -d`.

Схема взаимодействия сервисов:

```
                    ┌──────────────────────────────────────┐
                    │          Docker Compose сеть         │
                    │                                      │
Браузер ──► nginx:80 ──► wordpress:9000 (PHP-FPM) ──► db:3306 │
                    │                                (MariaDB)│
                    └──────────────────────────────────────┘
```

1. Браузер отправляет запрос на порт 80 хоста.
2. nginx принимает запрос. Если это статический файл (CSS, JS, картинка) — отдаёт сам. Если PHP — передаёт на `wordpress:9000`.

3. PHP-FPM обрабатывает PHP-код WordPress, при необходимости обращается к базе данных `db:3306`.
4. Ответ идёт обратно по цепочке в браузер.

Управление стеком

Основные команды

Команда	Описание
`docker compose up -d`	Запустить все сервисы в фоне
`docker compose down`	Остановить и удалить контейнеры и сети
`docker compose down -v`	То же + удалить тома (все данные!)
`docker compose ps`	Статус сервисов
`docker compose logs`	Логи всех сервисов
`docker compose logs -f nginx`	Следить за логами одного сервиса
`docker compose restart`	Перезапустить все сервисы
`docker compose restart nginx`	Перезапустить один сервис
`docker compose exec wordpress bash`	Войти внутрь контейнера
`docker compose pull`	Обновить образы до последних версий

stop vs. down

- `docker compose stop` — **останавливает** контейнеры, но не удаляет их. При следующем `docker compose start` контейнеры запустятся с прежними данными.
- `docker compose down` — **останавливает и удаляет** контейнеры и сети. Тома (volumes) сохраняются — данные БД не пропадут.
- `docker compose down -v` — удаляет **всё**, включая тома. Используйте с осторожностью: данные базы данных будут потеряны безвозвратно.

Просмотр логов

Логи всех сервисов одновременно:

```
docker compose logs
```

Следить за логами nginx в реальном времени:

```
docker compose logs -f nginx
```

Показать последние 50 строк логов WordPress:

```
docker compose logs --tail 50 wordpress
```

Выполнение команд внутри контейнера

Войти в контейнер WordPress:

```
docker compose exec wordpress bash
```

Проверить версию PHP:

```
docker compose exec wordpress php -v
```

Подключиться к MariaDB:

```
docker compose exec db mariadb -u wp_user -p wordpress
```

Практические задания

1. Установите `docker-compose-plugin`. Проверьте версию командой `docker compose version`.
2. Создайте файл `.env` с переменными `DB_ROOT_PASSWORD`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`. Добавьте `.env` в `.gitignore`.
3. Создайте `docker-compose.yml` с одним сервисом `db` (MariaDB). Запустите его и проверьте логи — найдите строку `ready for connections`.
4. Добавьте сервис `wordpress` с образом `wordpress:6-fpm`. Запустите `docker compose up -d` и убедитесь, что WordPress подключился к базе данных (проверьте логи на отсутствие ошибок).
5. Создайте файл `nginx/default.conf` с конфигурацией для проксирования PHP-запросов. Добавьте сервис `nginx` в compose-файл. Запустите и проверьте `curl -I http://localhost`.
6. Посмотрите логи всех сервисов (`docker compose logs`). Зайдите внутрь контейнера WordPress и проверьте версию PHP (`docker compose exec wordpress php -v`).
7. Остановите стек командой `docker compose down` (без `-v`). Запустите снова. Убедитесь, что данные сохранились — WordPress не предлагает повторную установку.
8. Выполните `docker compose down -v`. Запустите стек снова. Убедитесь, что WordPress снова показывает мастер установки — данные были удалены вместе с томами.

Итоги

В этой главе вы:

- узнали, зачем нужен Docker Compose и как он упрощает управление несколькими контейнерами;
- пошагово собрали `docker-compose.yml` из трёх сервисов: MariaDB, WordPress (PHP-FPM) и nginx;
- научились хранить пароли в файле `.env` и не допускать их попадания в Git;
- разобрали, как контейнеры взаимодействуют через внутреннюю сеть Docker Compose (обращение по имени сервиса);
- освоили управление стеком: запуск, остановка, логи, выполнение команд внутри контейнеров.

Весь стек из главы 11 — nginx + PHP-FPM + MariaDB + WordPress — теперь запускается одной командой `docker compose up -d` и может быть развёрнут на любом сервере с Docker.

В следующей главе мы настроим **CI/CD на GitLab** — автоматическую сборку и деплой, чтобы при каждом push в репозиторий стек обновлялся автоматически.

Этап 16. CI/CD: автоматизация сборки и деплоя

Зачем автоматизировать

В предыдущих главах мы проделали большой путь: написали `docker-compose.yml`, настроили nginx, WordPress, MariaDB и отправили проект в GitLab. Но каждый раз, когда нужно обновить сайт, мы действуем вручную:

1. Изменяем файлы на локальной машине.
2. Делаем `git push`.
3. Заходим на сервер по SSH.
4. Выполняем `git pull`.
5. Запускаем `docker compose up -d`.

Пять шагов — и ни один нельзя пропустить. Забыли `git pull` — на сервере старая версия. Ошиблись в конфиге — сайт лёг, а вы узнали об этом от пользователей.

CI/CD решает эту проблему: после `git push` всё остальное происходит автоматически.

> **Привязка к проекту.** В главе 13 мы отправили `wordpress-project` в GitLab, в главе 15 описали весь стек в `docker-compose.yml`. Теперь мы свяжем эти два шага: каждый push в GitLab будет запускать автоматическую проверку и деплой нашего WordPress-стека.

Что такое CI/CD

CI/CD — это две связанные практики:

CI — Continuous Integration (непрерывная интеграция): автоматическая проверка кода после каждого push. Цель — как можно раньше обнаружить ошибку.

CD — Continuous Delivery / Deployment (непрерывная поставка / развёртывание): автоматический деплой проверенного кода на сервер. Цель — чтобы изменения попадали в продакшен быстро и без ручных шагов.

```
                    CI/CD-конвейер (pipeline)

git push ──→ [ Сборка ] ──→ [ Тестирование ] ──→ [ Деплой ]
                 │                  │                  │
            Собрать образ     Проверить, что     Обновить стек
            Docker            конфиг корректен   на сервере
```

Каждый блок — это **этап** (stage). Если этап упал — конвейер останавливается, следующие этапы не запускаются. Разработчик получает уведомление и может исправить ошибку до того, как сломанный код попадёт на сервер.

CI/CD-системы

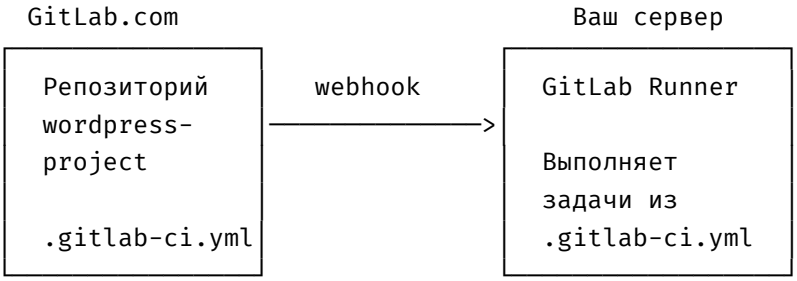
Существует много инструментов для CI/CD: Jenkins, GitHub Actions, GitLab CI/CD, Drone, CircleCI и другие. Мы будем использовать **GitLab CI/CD**, потому что наш репозиторий уже хранится в GitLab (глава 13) — ничего дополнительного устанавливать не нужно.

> **Принцип тот же.** Если вы позже перейдёте на GitHub Actions или Jenkins — концепции останутся прежними: конвейер, этапы, задачи, переменные окружения. Изменится только синтаксис конфигурационного файла.

Как устроен GitLab CI/CD

GitLab CI/CD состоит из двух частей:

1. **Файл `.gitlab-ci.yml`** в корне репозитория — описывает, что и в каком порядке делать.
2. **GitLab Runner** — программа, которая выполняет задачи из конвейера. Runner может работать на том же сервере, где развёрнут проект, или на отдельной машине.



При каждом `git push` GitLab читает `.gitlab-ci.yml` и отправляет задачи свободному Runner. Runner клонирует репозиторий, выполняет команды и сообщает результат обратно в GitLab.

Установка GitLab Runner

GitLab Runner устанавливается на сервер, где будет выполняться конвейер. В нашем случае — на тот же сервер, где работает Docker Compose.

Шаг 1: добавление репозитория и установка

```
curl -L
↪  "https://packages.gitlab.com/install/repositories/runner/gitlab-runner/script.deb.sh"
↪  | sudo
↪  bash
sudo apt install gitlab-runner
```

Проверим установку:

```
gitlab-runner --version
```

```
Version:      17.7.0
Git revision: 12345abc
Git branch:   main
GO version:   go1.22.5
Built:        2025-01-15T10:00:00+0000
OS/Arch:      linux/amd64
```

Шаг 2: регистрация Runner в GitLab

Runner нужно связать с вашим проектом. Для этого:

1. Откройте проект `wordpress-project` на gitlab.com.
2. Перейдите в **Settings → CI/CD → Runners**.
3. Нажмите **New project runner**.
4. Выберите **Linux** как платформу.
5. В поле **Tags** введите `deploy` (по этому тегу мы будем обращаться к Runner из конвейера).
6. Нажмите **Create runner** — GitLab покажет токен регистрации.

На сервере выполните:

```
sudo gitlab-runner register
```

Runner задаст несколько вопросов:

```
Enter the GitLab instance URL (for example, https://gitlab.com/):
https://gitlab.com/

Enter the registration token:
<вставьте токен из GitLab>

Enter a description for the runner:
wordpress-deploy

Enter tags for the runner (comma-separated):
deploy

Enter optional maintenance note for the runner:
<нажмите Enter>

Enter an executor:
docker
```

> **Почему executor `docker`?** Runner с Docker-executor выполняет каждую задачу внутри отдельного контейнера. Это обеспечивает изоляцию: задачи не могут повлиять на хост-систему и не зависят от установленных на хосте пакетов.

Когда Runner спросит образ по умолчанию:

```
Enter the default Docker image:
docker:27
```

Проверим, что Runner зарегистрирован и работает:

```
sudo gitlab-runner list
```

```
Listing configured runners          ConfigFile=/etc/gitlab-runner/config.toml
wordpress-deploy                    Executor=docker Token=glrt-...
 ↪  URL=https://gitlab.com/
```

Шаг 3: доступ Runner к Docker

Наш конвейер будет запускать команды Docker (собирать образы, запускать `docker compose`). Для этого Runner должен иметь доступ к Docker. Самый простой способ — подключить Docker-сокет хоста.

Откроем конфигурацию Runner:

```
sudo nano /etc/gitlab-runner/config.toml
```

Найдите секцию `[runners.docker]` и добавьте строку `volumes`:

```
[runners.docker]
  image = "docker:27"
  privileged = false
  volumes = ["/var/run/docker.sock:/var/run/docker.sock", "/cache"]
```

> **Внимание.** Монтирование Docker-сокета даёт Runner полный доступ к Docker на хосте. Это допустимо для учебного сервера, но в продакшене стоит рассмотреть более изолированные варианты (см. врезку ниже).

> ### Врезка. DooD и DinD — два способа дать CI/CD доступ к Docker
>
> Когда Runner с Docker-executor выполняет задачу, он запускает контейнер. Но что если задаче самой нужно вызывать `docker` — собирать образы или запускать `docker compose`? Контейнер по умолчанию не имеет доступа к Docker. Есть два подхода.
>
> **Docker-outside-of-Docker (DooD)** — монтирование сокета хоста:
>
> ```
> volumes = ["/var/run/docker.sock:/var/run/docker.sock"]
> privileged = false
> ```
>
> Контейнер Runner подключается к Docker-демону, который работает на хосте. Команды `docker build`, `docker compose up` выполняются тем же демоном, что обслуживает и сам Runner. Контейнеры, созданные из задачи, — это обычные контейнеры на хосте, они видят друг друга и делят ресурсы с остальными.
>
> *Плюсы:* просто настроить, быстро работает (нет накладных расходов на вложенный демон), образы кешируются на хосте.
>
> *Минусы:* задача получает полный доступ к Docker на хосте — может увидеть и удалить чужие контейнеры. Для доверенного учебного сервера это приемлемо, для общего CI-кластера — нет.

Docker-in-Docker (DinD) — отдельный демон внутри контейнера:

```
privileged = true
```

```yaml
# .gitlab-ci.yml
services:
  - docker:27-dind
variables:
  DOCKER_HOST: tcp://docker:2376
  DOCKER_TLS_CERTDIR: "/certs"
```

Runner запускает рядом с задачей дополнительный контейнер (`docker:27-dind`), в котором работает собственный Docker-демон. Задача обращается к нему по сети, а не через сокет хоста. Каждый запуск конвейера получает чистый Docker — контейнеры и образы от предыдущих запусков не остаются.

Плюсы: полная изоляция — задача не видит контейнеры хоста.

Минусы: требует `privileged: true` (контейнеру разрешается управлять ядром), образы не кешируются между запусками (каждый раз скачиваются заново), сложнее в настройке.

Что мы используем в курсе. DooD — монтирование сокета. Для учебного проекта с одним Runner на собственном сервере это самый простой и быстрый вариант. В продакшене с общими Runner чаще выбирают DinD или rootless Docker — чтобы задачи разных проектов не могли влиять друг на друга.

Перезапустим Runner, чтобы он подхватил новый конфиг:

```
sudo systemctl restart gitlab-runner
```

Проверим статус:

```
sudo systemctl status gitlab-runner
```

```
● gitlab-runner.service - GitLab Runner
     Active: active (running)
```

Первый конвейер

Начнём с простого конвейера, чтобы убедиться, что всё работает.

Перейдём в каталог проекта и создадим файл `.gitlab-ci.yml`:

```
cd ~/wordpress-project
nano .gitlab-ci.yml
```

```yaml
stages:
  - test

check-config:
  stage: test
```

```
  image: docker:27
  tags:
    - deploy
  script:
    - docker compose config --quiet
    - echo "Конфигурация корректна"
```

Разберём каждую строку:

Ключ	Описание
`stages`	Список этапов конвейера в порядке выполнения
`check-config`	Имя задачи (job) — произвольное
`stage: test`	К какому этапу относится задача
`image: docker:27`	Образ Docker, внутри которого выполняется задача
`tags: - deploy`	Запускать только на Runner с тегом `deploy`
`script`	Список команд, которые выполнятся последовательно
`docker compose config --quiet`	Проверяет `docker-compose.yml` на синтаксические ошибки

Отправим изменения в GitLab:

```
git add .gitlab-ci.yml
git commit -m "Добавить CI/CD: проверка конфигурации"
git push
```

Теперь откройте проект на gitlab.com и перейдите в **Build → Pipelines**. Вы увидите запущенный конвейер:

```
Pipeline #1       check-config ✓       passed       1 minute ago
```

Нажмите на имя задачи, чтобы увидеть лог выполнения. Если всё настроено правильно — задача пройдёт успешно, и вы увидите зелёную галочку.

> **Совет.** Если конвейер не запускается, проверьте: 1) Runner зарегистрирован и онлайн (Settings → CI/CD → Runners); 2) тег `deploy` в `.gitlab-ci.yml` совпадает с тегом Runner; 3) файл `.gitlab-ci.yml` находится в корне репозитория.

Полный конвейер: проверка и деплой

Теперь создадим конвейер из двух этапов: **test** (проверка) и **deploy** (развёртывание).

```
nano .gitlab-ci.yml
```

```
stages:
  - test
  - deploy

# --- Этап 1: проверка ---
validate:
  stage: test
  image: docker:27
  tags:
    - deploy
```

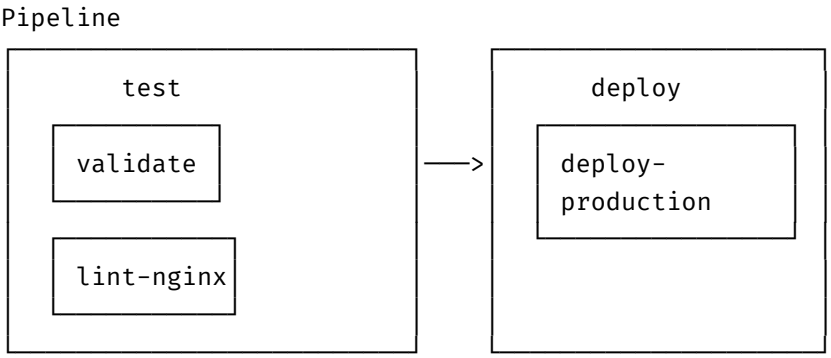
```yaml
  script:
    - docker compose config --quiet
    - echo "docker-compose.yml — OK"

lint-nginx:
  stage: test
  image: nginx:1.27
  tags:
    - deploy
  script:
    - cp nginx/default.conf /etc/nginx/conf.d/default.conf
    - nginx -t
    - echo "nginx.conf — OK"

# --- Этап 2: деплой ---
deploy-production:
  stage: deploy
  image: docker:27
  tags:
    - deploy
  only:
    - main
  script:
    - docker compose down
    - docker compose pull
    - docker compose up -d
    - docker compose ps
  environment:
    name: production
```

Разберём новые элементы:

Два этапа

```
stages:
  - test       ← выполняется первым
  - deploy     ← выполняется, только если test пройден
```

Задачи внутри одного этапа запускаются **параллельно** (если есть свободные Runner). Задачи следующего этапа — только после успешного завершения всех задач предыдущего.



Проверка конфигурации nginx

Задача `lint-nginx` использует образ `nginx:1.27` — в нём есть утилита `nginx -t`, которая проверяет конфигурацию на ошибки, не запуская сам сервер. Если в конфиге опечатка — конвейер остановится до деплоя.

Деплой только из main

```
only:
  - main
```

Это значит: задача `deploy-production` выполняется **только** при push в ветку `main`. Если вы работаете в feature-ветке — проверки запустятся, но деплой нет. Это важная защита: сырой код из экспериментальной ветки не попадёт на сервер.

Окружение (environment)

```
environment:
  name: production
```

GitLab запоминает деплои и показывает их на странице **Operate → Environments**. Вы видите, когда и какая версия была развёрнута — удобно для отслеживания.

Отправим обновлённый конвейер:

```
git add .gitlab-ci.yml
git commit -m "CI/CD: добавить проверку nginx и деплой"
git push
```

В GitLab вы увидите конвейер с двумя этапами:

```
Pipeline #2    validate ✓  lint-nginx ✓  │  deploy-production ✓    passed
```

Переменные окружения и секреты

Файл `.env` содержит пароли — он не должен храниться в Git (мы добавили его в `.gitignore` в главе 15). Но тогда как конвейер получит пароли для деплоя?

GitLab CI/CD Variables — безопасное хранилище для секретов.

Настройка переменных

1. Откройте проект на gitlab.com.
2. Перейдите в **Settings → CI/CD → Variables**.
3. Нажмите **Add variable** и создайте четыре переменные:

Ключ	Значение (пример)	Настройки
`DB_ROOT_PASSWORD`	`SuperStr0ng!Root`	Masked, Protected
`DB_NAME`	`wordpress`	Protected
`DB_USER`	`wp_user`	Protected
`DB_PASSWORD`	`Wp_S3cur3_Pass!`	Masked, Protected

Флаги:

- **Masked** — значение скрыто в логах конвейера (вместо пароля отображается `[MASKED]`).
- **Protected** — переменная доступна только в protected-ветках (по умолчанию `main`). Это ещё один уровень защиты — из случайной ветки пароли не достать.

  Важно. Никогда не выводите секреты через `echo` в `script`. Даже если переменная masked — лучше не рисковать. Относитесь к секретам как к паролям: чем меньше мест, где они видны, тем лучше.

Использование переменных в конвейере

GitLab автоматически передаёт переменные как переменные окружения внутрь каждой задачи. Docker Compose, в свою очередь, умеет подставлять переменные окружения вместо `${...}` в `docker-compose.yml`.

Обновим задачу деплоя — создадим файл `.env` из переменных GitLab прямо перед запуском:

```
deploy-production:
  stage: deploy
  image: docker:27
  tags:
    - deploy
  only:
    - main
  script:
    - |
      cat > .env << EOF
      DB_ROOT_PASSWORD=${DB_ROOT_PASSWORD}
      DB_NAME=${DB_NAME}
      DB_USER=${DB_USER}
      DB_PASSWORD=${DB_PASSWORD}
      EOF
    - docker compose down
    - docker compose pull
    - docker compose up -d
    - docker compose ps
  environment:
    name: production
```

Конструкция `cat > .env << EOF ... EOF` создаёт файл `.env` из переменных, которые GitLab передал задаче. Файл существует только на время выполнения задачи — после завершения контейнер Runner удаляется вместе с файлом.

Конвейер с уведомлениями

Полезно знать, успешно ли прошёл деплой, не заходя каждый раз в GitLab. Добавим задачу, которая отправляет уведомление после деплоя:

```
notify-success:
  stage: deploy
  image: alpine:3.20
  tags:
    - deploy
  needs:
    - deploy-production
  script:
    - echo "Деплой завершён успешно"
    - echo "Коммит — $CI_COMMIT_SHORT_SHA"
    - echo "Автор — $CI_COMMIT_AUTHOR"
    - echo "Сообщение — $CI_COMMIT_MESSAGE"
```

Здесь мы используем **встроенные переменные** GitLab CI/CD:

Переменная	Описание
`CI_COMMIT_SHORT_SHA`	Короткий хеш коммита (например, `a1b2c3d`)
`CI_COMMIT_AUTHOR`	Автор коммита
`CI_COMMIT_MESSAGE`	Текст коммита
`CI_PIPELINE_URL`	Ссылка на конвейер в GitLab
`CI_PROJECT_NAME`	Имя проекта
`CI_COMMIT_BRANCH`	Имя ветки

> **Примечание.** В реальных проектах уведомления обычно отправляют в Telegram, Slack или по email. GitLab также умеет отправлять email-уведомления автоматически — это настраивается в **Settings → Integrations**.

Ключевое слово needs и граф зависимостей

По умолчанию задачи одного этапа запускаются параллельно, а следующий этап ждёт завершения всех задач предыдущего. Ключевое слово `needs` позволяет точнее описать зависимости:

```
notify-success:
  needs:
    - deploy-production
```

Это означает: `notify-success` запустится сразу после `deploy-production`, не дожидаясь остальных задач этапа `deploy` (если бы они были). Для нашего проекта разница небольшая, но в крупных конвейерах `needs` позволяет существенно сократить время выполнения.

Финальная версия конвейера

Соберём всё вместе — полный файл `.gitlab-ci.yml`:

```
nano .gitlab-ci.yml
```

```yaml
stages:
  - test
  - deploy

# --- Этап 1: проверка ---
validate:
  stage: test
  image: docker:27
  tags:
    - deploy
  script:
    - docker compose config --quiet
    - echo "docker-compose.yml — OK"

lint-nginx:
  stage: test
  image: nginx:1.27
  tags:
    - deploy
  script:
    - cp nginx/default.conf /etc/nginx/conf.d/default.conf
    - nginx -t
    - echo "nginx.conf — OK"

# --- Этап 2: деплой ---
deploy-production:
  stage: deploy
  image: docker:27
  tags:
    - deploy
  only:
    - main
  script:
    - |
      cat > .env << EOF
      DB_ROOT_PASSWORD=${DB_ROOT_PASSWORD}
      DB_NAME=${DB_NAME}
      DB_USER=${DB_USER}
      DB_PASSWORD=${DB_PASSWORD}
      EOF
    - docker compose pull
    - docker compose up -d
    - docker compose ps
  environment:
    name: production
```

Почему нет `docker compose down` перед `up`? Команда `docker compose up -d` достаточно умна: если образ обновился — контейнер будет пересоздан; если нет — останется работать. Явный `down` вызывает кратковременный простой, который можно избежать. Для нашего учебного проекта разница невелика, но это хорошая привычка.

Отправим финальную версию:

```
git add .gitlab-ci.yml
git commit -m "CI/CD: финальный конвейер с проверкой и деплоем"
git push
```

Что происходит после push

Проследим путь кода от `git push` до работающего сайта:

```
1. Вы делаете git push в ветку main
          │
2. GitLab получает изменения и читает .gitlab-ci.yml
          │
3. GitLab отправляет задачи Runner
          │
4. Runner запускает этап test:
   ├── validate:  проверяет docker-compose.yml
   └── lint-nginx: проверяет конфигурацию nginx
          │
5. Все проверки пройдены → Runner запускает этап deploy:
   └── deploy-production:
       ├── Создаёт .env из переменных GitLab
       ├── Скачивает свежие образы (docker compose pull)
       └── Обновляет контейнеры (docker compose up -d)
          │
6. Сайт обновлён. GitLab показывает зелёный статус ✓
```

Если на шаге 4 проверка упадёт (например, опечатка в `nginx/default.conf`), конвейер остановится — деплой не случится. Вы увидите красный статус ✗ и сможете прочитать лог ошибки прямо в GitLab.

Работа с ветками

CI/CD особенно полезен при работе в команде. Типичный сценарий:

```
# Создаём ветку для новой функции
git switch -c feature-caching

# Вносим изменения в nginx/default.conf
nano nginx/default.conf

# Коммитим и пушим
git add nginx/default.conf
git commit -m "Добавить кэширование статики"
git push -u origin feature-caching
```

После push GitLab запустит конвейер для ветки `feature-caching`. Этап `test` выполнится — вы узнаете, корректен ли новый конфиг. Этап `deploy` будет пропущен (он срабатывает только для `main`).

Когда проверки пройдены, вы создаёте **Merge Request** (запрос на слияние) в GitLab:

1. Откройте проект на gitlab.com.
2. GitLab предложит создать Merge Request для свежей ветки — нажмите **Create merge request**.
3. Заполните описание и нажмите **Submit**.
4. После ревью нажмите **Merge**.

GitLab сольёт ветку в `main` и автоматически запустит конвейер деплоя. Конфиг, проверенный в feature-ветке, попадёт на сервер без единого ручного шага.

> **Merge Request vs. Pull Request.** В GitLab используется термин Merge Request (MR), в GitHub — Pull Request (PR). Суть одна: предложить изменения для слияния в основную ветку.

Отладка конвейера

Конвейеры не всегда работают с первого раза. Типичные проблемы и способы их решения:

Runner не подхватывает задачу

Симптом: задача висит в статусе «Pending» с пометкой «This job is stuck because the project doesn't have any runners online».

Проверьте:

```
sudo gitlab-runner status
sudo gitlab-runner list
```

Если Runner не запущен — запустите:

```
sudo systemctl start gitlab-runner
```

Если Runner работает, но задача висит — проверьте совпадение тегов. Тег в `.gitlab-ci.yml` (`tags: - deploy`) должен совпадать с тегом, указанным при регистрации Runner.

Ошибка «docker: not found»

Runner с Docker-executor запускает задачи в контейнере. Если Docker-сокет не подключён — внутри контейнера не будет доступа к Docker.

Проверьте `/etc/gitlab-runner/config.toml`:

```
volumes = ["/var/run/docker.sock:/var/run/docker.sock", "/cache"]
```

После изменения перезапустите Runner:

```
sudo systemctl restart gitlab-runner
```

Ошибка в .gitlab-ci.yml

GitLab проверяет синтаксис файла при получении push. Если в файле ошибка — конвейер не запустится.

Проверить синтаксис можно заранее, не делая push. В GitLab есть встроенный инструмент: **Build → Pipeline editor** → вкладка **Validate**.

Или через API:

```
curl --request POST \
  --header "PRIVATE-TOKEN: <ваш-токен>" \
  "https://gitlab.com/api/v4/ci/lint" \
  --form "content=@.gitlab-ci.yml"
```

Справочник: структура .gitlab-ci.yml

Ключевое слово	Описание
`stages`	Список этапов в порядке выполнения
`stage`	К какому этапу относится задача
`image`	Docker-образ для выполнения задачи
`tags`	На каком Runner запускать (по тегам)
`script`	Команды для выполнения
`before_script`	Команды, которые выполняются перед `script`
`after_script`	Команды, которые выполняются после `script` (даже при ошибке)
`only`	В каких ветках запускать задачу
`except`	В каких ветках **не** запускать
`needs`	Зависимости от других задач (для ускорения)
`environment`	Имя окружения для отслеживания деплоев
`variables`	Локальные переменные задачи
`artifacts`	Файлы, которые нужно сохранить после задачи
`when`	Условие запуска: `on_success` (по умолчанию), `on_failure`, `always`, `manual`

Практические задания

1. Установите GitLab Runner на сервер. Зарегистрируйте его в своём проекте с тегом `deploy`. Убедитесь, что Runner отображается как онлайн в Settings → CI/CD → Runners.
2. Создайте `.gitlab-ci.yml` с одной задачей, которая выводит `echo "Hello CI/CD"`. Сделайте push и убедитесь, что конвейер запустился и прошёл успешно.
3. Добавьте задачу `validate`, которая проверяет `docker-compose.yml` командой `docker compose config --quiet`. Проверьте, что при корректном файле задача проходит.
4. Внесите намеренную ошибку в `docker-compose.yml` (например, неверный отступ). Сделайте push и убедитесь, что конвейер упал. Посмотрите лог ошибки в GitLab. Исправьте ошибку и отправьте исправление — конвейер должен пройти.
5. Добавьте задачу `lint-nginx`, которая проверяет конфигурацию nginx командой `nginx -t`. Убедитесь, что обе задачи этапа `test` запускаются параллельно.
6. Создайте переменные `DB_ROOT_PASSWORD`, `DB_NAME`, `DB_USER` и `DB_PASSWORD` в Settings → CI/CD → Variables. Включите флаги Masked и Protected.
7. Добавьте этап `deploy` с задачей `deploy-production`, которая создаёт `.env` из переменных GitLab и запускает `docker compose up -d`. Ограничьте задачу веткой `main` с помощью `only`.
8. Создайте ветку `feature-test`, внесите изменение в `nginx/default.conf`, сделайте push. Убедитесь, что проверки прошли, а деплой был пропущен. Создайте Merge Request и выполните слияние — убедитесь, что деплой запустился автоматически.
9. Сделайте `curl -I http://localhost` после деплоя и убедитесь, что WordPress отвечает. Поздравляем — ваш сайт разворачивается автоматически!

Итоги

В этой главе вы:

- узнали, что такое CI/CD и зачем нужна автоматизация сборки и деплоя;
- установили и зарегистрировали GitLab Runner на сервере;
- написали `.gitlab-ci.yml` — конфигурацию конвейера с двумя этапами: проверка (валидация `docker-compose.yml` и конфига nginx) и деплой (`docker compose up -d`);
- научились хранить секреты (пароли базы данных) в переменных GitLab CI/CD, а не в репозитории;
- увидели, как работа с ветками и Merge Request позволяет проверить изменения до того, как они попадут на сервер.

Теперь весь путь от изменения в коде до работающего сайта автоматизирован: вы делаете `git push` — GitLab проверяет конфигурацию и обновляет стек на сервере. Ручные шаги — SSH, `git pull`, `docker compose up` — больше не нужны.

Это последняя глава курса. Давайте оглянемся назад.

В главе 1 вы впервые подключились к Linux-серверу по SSH и не знали ни одной команды. Сейчас вы умеете работать с файловой системой, редактировать конфигурации, управлять пользователями и сервисами, настраивать сеть и файрвол. Вы развернули полноценный веб-сайт на nginx + PHP-FPM + WordPress — сначала вручную, потом упаковали его в Docker-контейнеры, описали в Docker Compose и настроили автоматический деплой через CI/CD.

Это фундамент. Дальше — мониторинг (Prometheus, Grafana), оркестрация контейнеров (Kubernetes), инфраструктура как код (Ansible, Terraform), облачные платформы. Но все эти инструменты опираются на то, что вы уже знаете: командную строку, Linux, контейнеры и автоматизацию.

Удачи!